



Università degli Studi di Perugia

Dipartimento di Chimica, Biologia e Biotecnologie

Tesi di Laurea Magistrale in Scienze Chimiche

Titolo:

*Machine learning techniques for molecular
electrodynamics simulations in real time*

Laureando

SIOW JIA SHENG

Relatore

Prof.ssa Paola Belazoni

Correlatore

Dott. Leonardo Belpassi

Anno Accademico 2024/2025

Contents

Introduction	4
1 RT-TD Electronic Structure Theory	6
1.1 Introduction	6
1.2 Theory of real-time time-dependent method	8
1.2.1 Schrödinger equation and Time evolution operator	8
1.2.2 Magnus expansion and the midpoint method	9
1.2.3 From Schrödinger equation to Time-Dependent Equations for approximate wave functions	12
1.2.4 Absorption in real-time time-dependent electronic structure methods	19
1.2.5 Comparing with Real-time with Linear response	21
1.3 Application of real-time methods	22
1.3.1 UV / Vis Spectroscopy	22
1.3.2 X-ray Absorption Spectroscopy	23
1.3.3 Excited State Absorption and Emission	23
1.3.4 Nonlinear properties	24
1.4 Current and future challenges	24
2 Machine learning Introduction	26
2.1 Theory of Machine Learning	26
2.1.1 Introduction	26
2.1.2 Types of Learning	26
2.1.3 Machine Learning Workflow	27
2.2 General applications in chemistry	28
2.3 Application in real-time electron dynamics	29
2.4 Ordinary least square	30
2.5 Neural Network	32
2.5.1 Forward pass	36
2.5.2 Backward pass	36
2.5.3 Activation function	38
2.5.4 Optimizer	39
2.5.5 Loss Function	40
2.6 Python and Machine learning frameworks	41

3	Methodology	42
3.1	Data generation and scaling of the data	42
3.2	Scaling of data	44
3.3	Measurements of performance metrics	45
3.3.1	R^2 value	45
3.3.2	Mean squared error and root mean squared error	46
3.3.3	Mean Absolute Percentage Error (MAPE)	47
3.4	Multiple Linear Regression	47
3.5	Neural Network	47
3.6	Implementation	48
4	Results and Discussion	59
4.1	Linear regression	59
4.1.1	Effect of different scaling method	59
4.1.2	Effect of train/test ratio	60
4.1.3	Effect of increasing dataset size	65
4.1.4	Effects of increasing basis size	67
4.1.5	Effect changing orientation of LiH	70
4.1.6	Effect of including/removing delta kick values	70
4.1.7	Effect of removing zeros	76
4.2	Neural Network method	80
4.2.1	Effect of changing number of hidden layers	81
4.2.2	Effect of changing number of neurons	82
4.2.3	Effect of different types of optimizer	84
4.2.4	Grid search	87
4.2.5	Effect of removing zeros	89
4.3	Discussion	92
	Conclusion	97
	Bibliography	99
A	Machine learning program details	106
A.0.1	Input parameters	106
B	Propagation details	108
B.1	Molecule coordinates	108
B.2	Details of the simulation	109

Introduction

In recent years, we have seen a paradigm shift in computational quantum chemistry. Traditional methods of computational chemistry are being supplemented, extended and replaced by faster and more effective approaches, such as new algorithms that take advantage of improved hardware. The progress in computer hardware, which is also caused by the current increase in the development of machine learning, brings new opportunities for scientific development. Machine learning is gaining ground in several branches of science, including quantum chemical calculations. Therefore, it seems to be a time to investigate the possibility of merging the two branches of science in order to provide quantum chemists with new tools for research. The real-time electron structure theory, which requires computationally intensive explicit time propagation, has regained motivation in research in recent years due to advances in computing power and numerical algorithms. Real-time methods are important for spectroscopy, the description of the interaction of electron spin interaction with perturbations and for the simulation of various chemical phenomena beyond linear response regime. In this work we want to investigate how machine learning can be applied in molecular electrodynamic simulations.

In particular, we will consider here the simplest ab-initio method for electronic structure theory based on the mean-field approximation, namely the real time time-dependent Hartree-Fock (RT-TDHF). For each time step of an RT-TDHF calculation, the Fock matrix must be iteratively analysed and manipulated, which is the most time-consuming step. From the representation of the Fock matrix, we suggest that there exists a linear relationship between the Fock matrix and the density matrix. Therefore, it should be possible to replace the expensive calculation of two-electron integrals by the machine-learned parameters. In this work, we will try to find and exploit the relationship between the density matrix and the Fock matrix using machine learning algorithms. The methods we investigate are simple machine learning methods such as regression analysis and the more complex neural network.

Currently, there are few academic publications in the field of machine learning techniques applied to electrodynamic propagation. The only existing papers dealing with machine-learned propagation of the Hartree-Fock Hamiltonian are by Bhat and his co-workers [1, 2, 3]. However, in these pioneering papers, the authors did not address the quality of the predicted Fock matrix and how accurately it can be compared to the true matrix. Furthermore, they did not compare their method with different machine learning methods. In this work, we will investigate the prediction of Fock matrix using density matrices obtained along the real-time simulation and compare

the performance of different machine learning algorithms. Due to the limitation of the existing literature and research approach, this thesis will focus exclusively on the RT-TDHF scheme since it is the simplest approximation method and we can apply the findings to other methods, including density functional theory. Moreover, we restrict ourselves to investigate simple molecular systems with relatively small basis sets since increasing the hierarchy of basis sets increases the computational resource requirements.

There are many types of machine learning methods. We will use supervised machine learning methods because we can obtain the data (Fock and density matrices) during time propagation at each time step. For a first analysis, we will use the simple linear regression method. Other variants of regression models, such as the ridge regression technique, are outside the scope of this work, which focuses on investigating the effectiveness of a simple regression model in this framework. We will also apply the simple neural network method to investigate whether it is an effective method for solving our problem. Different combinations of hyperparameters for the neural network will be investigated to find the best neural network architecture that gives us the best results. To check how effective the machine-learned Fock matrix is compared to the actual value, we introduce different performance metric measurement tools commonly used in machine learning studies.

This study aims to complement the usual electrodynamic simulation, i.e. to find out whether it is possible to mix the current method with machine learning analysis instead of using machine learning as a substitute for the current method. Therefore, we will use the data generated during time propagation, which are the Fock matrices and the density matrices generated during propagation, as a complete dataset. This complete dataset will then be split into a training dataset that are used for the learning step of the model, and a test dataset that are used for testing the quality of model. We use this problem as a testbed to develop strategies that can be generalised to other methods in quantum dynamics, such as the post-Hartree-Fock method and time-dependent density functional theory. We expect that this study will successfully provide us with new insights for effective electrodynamic simulation using a machine-learned Fock matrix, and hopefully to reduce computation time allowing for longer propagations, which are crucial for increasing the accuracy of the derived observables. The implementation of machine learning is done in Python programming language. We used several popular Python machine learning modules such as scikit-learn, TensorFlow which are available freely online. Those machine learning modules are easy to use, regularly maintained by developers, and come with detailed documentation and tutorials.

This thesis work is organized as follows. First, in Chapter 1, we will introduce the basic theory of the real-time electronic structure propagation, from the time-dependent Schrödinger equation to the quantum Liouville-von Neumann equation within the Hartree-Fock theory. Successively, in Chapter 2, we will describe the concepts of machine learning and its components that are useful in this project. We will then proceed in Chapter 3 to the illustration of methods and tools that will be used to develop and implement the Python code used this work. Finally, in Chapter 4, the results and discussions will be presented.

Chapter 1

Real-time Time-Dependent Electronic Structure Theory

1.1 Introduction

The real-time electronic structure theory considers the time dependence of the quantum electronic system by evolving the time-dependent Schrödinger equation in the time domain.

This let us to predict and simulate the response of a molecular system or material to any combination and strength of external electro-magnetic field, which including weak or strong fields. The real-time methods explicitly consider the time-dependence of the quantum system, and its interactions with an environment. Differently to other time-dependent approaches, such as for instance the time-dependent linear-response theory, in real-time methods a numerical approximation to the underlying time-dependent Schrödinger equation has to be devised. Therefore, one must develop robust numerical techniques, to explicitly propagate the quantum system in time.

Standard algorithms to solve time-dependent partial differential equation, such as Runge-Kutta methods, are not always suitable to describe the time-evolution of the TDSE. In particular in those cases in which TDSE is approximated via a mean-field approach, one can introduce the Magnus expansion of the time evolution operator to propagate the quantum system.

Since we require large number of time steps for accurate calculation of the time evolution, the computational cost required in this kind of simulations is typically very high. Furthermore, in order to obtain accurate observables obtained by Fourier transform, one has the necessity of very long total propagation time. In this project, we will investigate the use of some machine learning methods to speed up propagation of the system forward in time. For example, in the work of Harish S. Bhat,[1], they developed a computational method to learn the molecular Hamiltonian matrix from matrix-values time series of the electron density.

In the simplest mean-field approach, in order to propagate the system forward in time, we have to calculate the Hamiltonian matrix $\mathbf{H}$ or Fock matrix in Hartree-Fock scheme iteratively, during each time step in a self-consistent manner, to obtain the

new density matrix. Since the calculation of Fock matrix include the evaluation of two electron potential, which is assembling density matrix and two-electron integrals. This computation is scaled by $O(M^4/8)$ with chosen basis size M and we have to calculate self-consistently.

The application of real-time electronic structure theory, span the field of spectroscopy including photoionization, non-linear optical response, etc. Furthermore, this approach can be used to study molecular conductance, excited state dynamics, ionization, and non-linear optical effects. This real-time technique, allow us to go beyond the traditional energy, or frequency domain based response theories.[4] We will dedicate a section below to dive into the details of these applications. We will briefly discuss the the theoretical basis for real-time propagation in the next section.

1.2 Theory of real-time time-dependent method

In this section, we will briefly introduce the theory of real-time time-dependent electronic structure theory.

1.2.1 Schrödinger equation and Time evolution operator

The time-dependent Schrödinger equation is :

$$i\hbar \frac{\partial}{\partial t} \Psi(x, t) = \hat{H} \Psi(x, t) \quad (1.1)$$

where i is the imaginary number, $\hbar$ is the Planck's constant divided by 2π , and H is the non-relativistic Hamiltonian operator:

$$\hat{H} = \frac{\hbar^2}{2m} \nabla^2 + V \quad (1.2)$$

where V is the potential function.

Since the time-dependent Schrödinger equation is linear in time, we can suppose that exists an operator that describes the dynamics of the wavefunction:

$$\Psi(t) = \hat{U}(t, t_0) \Psi(t_0) \quad (1.3)$$

where we have defined $\hat{U}$ as the time-evolution operator that evolve the quantum system as a function of time, and this is a solution of the time-dependent Schrödinger equation. The propagator $\hat{U}$ acts on the wavefunction to the right, to propagate the system forward in time from t_0 to t . In the simple case that the Hamiltonian is time-independent, this time-evolution propagator has the form :

$$\hat{U}(t, t_0) = \exp \left(\frac{-iH(t - t_0)}{\hbar} \right) \quad (1.4)$$

Below we can see the time-evolution operator obeys several properties. The wavefunction must be continuously differentiable in time. An obvious property is that $\hat{U}$ should do nothing when $t' = t$, thus:

$$\hat{U}(t, t) = 1 \quad (1.5)$$

Besides, the $\hat{U}$ must be unitary, such that so it preserves the norm of the states

$$\hat{U}(t, t_0) \hat{U}(t_0, t) = 1 \quad (1.6)$$

Therefore,

$$\hat{U}^{-1}(t, t_0) = \hat{U}(t_0, t) \quad (1.7)$$

Furthermore, $\hat{U}(t'; t)$ satisfies the composition property

$$\hat{U}(t, t_0) = \hat{U}(t, t') \hat{U}(t', t_0) \quad (1.8)$$

by which, in order to evolve a state from t_0 to t , we evolve the state from t_0 to t' and then evolve the state from t' to t .

It is worth noting some relationships that are important in evaluating the action of exponential operators:

1. The Baker-Hausdorff relationship:

$$\exp(i\hat{G}\lambda)\hat{A}\exp(-i\hat{G}\lambda) = \hat{A} + i\lambda[\hat{G}, \hat{A}] + \left(\frac{i^2\lambda^2}{2!}\right)[\hat{G}, [\hat{G}, \hat{A}]] + \dots \quad (1.9)$$

2. If $\hat{A}$ and $\hat{B}$ do not commute, but $[\hat{A}, \hat{B}]$ commutes with $\hat{A}$ and $\hat{B}$, then:

$$e^{\hat{A}+\hat{B}} = e^{\hat{A}}e^{\hat{B}}e^{-\frac{1}{2}[\hat{A}, \hat{B}]} \quad (1.10)$$

$$e^{\hat{A}}e^{\hat{B}} = e^{\hat{B}}e^{\hat{A}}e^{-[\hat{A}, \hat{B}]} \quad (1.11)$$

1.2.2 Magnus expansion and the midpoint method

For the time-independent Hamiltonian, the Schrödinger equation is:

$$i\hbar\frac{\partial}{\partial t}\Psi(x, t) = \hat{H}\Psi(x, t) \quad (1.12)$$

and we can also rewrite it, in the case that the hamiltonian is time-independent, as in the form of propagator:

$$\Psi(x, t) = \exp\left(\frac{-i\hat{H}t}{\hbar}\right)\Psi(x, 0). \quad (1.13)$$

However, it is not true when Hamiltonian is time-dependent. If we simply write the time- dependent Hamiltonian as:

$$\Psi(x, t) = \exp\left(-i\int_0^t \frac{\hat{H}(t')dt'}{\hbar}\right)\Psi(x, 0) \quad (1.14)$$

it will not be correct, because from the Taylor expansion we have,

$$\begin{aligned} \Psi(x, t) = & \Psi(x, 0) - \frac{i}{\hbar}\int_0^t \hat{H}(t')dt'\Psi(x, 0) \\ & - \frac{1}{2\hbar^2}\int_0^t\int_0^t \hat{H}(t')\hat{H}(t'')dt'dt''\Psi(x, 0) + \dots \end{aligned} \quad (1.15)$$

This expression indicates that $\hat{H}(t')$ always operates after $\hat{H}(t'')$, regardless of whether $t' > t''$ or not, but the differential form of the Time-Dependent Schrödinger Equation, shows that the Hamiltonian at different times, must operate on the wavefunction in chronological order. Therefore, we need to write the integral representation of time-dependent Hamiltonians as:

$$\Psi(x, t) = \hat{\mathcal{T}}\exp\left(-i\int_0^t \frac{\hat{H}(t')dt'}{\hbar}\right)\Psi(x, 0) \quad (1.16)$$

where the $\hat{\mathcal{T}}$ here is the time-ordering operator, to order operators in the Taylor series chronologically. The propagator in this equation is just a shorthand notation for the Dyson series:

$$\hat{U}(t, t_0) = \hat{\mathcal{T}} \exp \left(-i \int_{t_0}^t \hat{H}(t') dt' \right) \quad (1.17)$$

Where Dyson series is the most general form of expression for the time-evolution operator:

$$\hat{U}(t, t_0) = \sum_{n=1}^{\infty} \frac{(-i)^n}{n!} \int_0^t dt_1 \int_0^{t_1} dt_2 \cdots \int_0^{t_{n-1}} dt_n \hat{H}(t_1) \hat{H}(t_2) \cdots \hat{H}(t_n), \quad (1.18)$$

$$t_1 > t_2 > \cdots > t_n$$

The goal of the real-time electronic structure theory is to explicitly solve the time-dependent Schrödinger equation given above. However, the time dependence in the Hamiltonian $\hat{H}(t)$ (or Fock operator $\hat{F}(t)$ within Hartree Fock theory) does not have a general closed-form solution. The above integration cannot be done, unless the corresponding Hamiltonian $\hat{H}(t)$ commute among themselves for all values of time.[5] One can introduce the Magnus expansion. It can be used to find a general solution for the time-dependent wave function, in the case where the $\hat{H}$ is time-dependent and does not commute with itself at different times:

$$[\hat{H}(t_1), \hat{H}(t_2)] \neq 0 \quad (1.19)$$

The Magnus expansion, can let us find a numerical solution of the time-dependent Schrödinger equation. This can be done by the propagator $\hat{U}(t, t_0)$ in the exponential form, that can relate wave functions or density matrices at different times:

$$\Psi(t) = \hat{U}(t, t_0) \Psi(t_0) \quad (1.20)$$

Where $\hat{U}$ is defined as:

$$\hat{U}(t, t_0) = \exp(\Omega(t, t_0)) \quad (1.21)$$

Then, we can cast the equation as:

$$\frac{d}{dt} \hat{U}(t, t_0) \Psi(t_0) = \tilde{H}(t) \hat{U}(t, t_0) \Psi(t_0) \quad (1.22)$$

where we have defined

$$\tilde{H}(t) \equiv \frac{-i}{\hbar} \hat{H}(t) \quad (1.23)$$

In the Magnus expansion $\Omega(t, t_0)$ is written as a power-series,

$$\Omega(t, t_0) = \int_{t_0}^t \tilde{H}_1 dt_1 + \frac{1}{2} \int_{t_0}^t dt_1 \int_{t_0}^{t_1} dt_2 [\tilde{H}_1, \tilde{H}_2] + \dots \quad (1.24)$$

To see clearly the expansion, we can write down the three first term of the series:

$$\Omega_1(t) = \int_{t_0}^t \tilde{H}_1 dt_1 \quad (1.25)$$

$$\Omega_2(t) = \frac{1}{2} \int_{t_0}^t dt_1 \int_{t_0}^{t_1} dt_2 [\tilde{H}_1, \tilde{H}_2] \quad (1.26)$$

$$\Omega_3(t) = \frac{1}{6} \int_{t_0}^t dt_1 \int_{t_0}^{t_1} dt_2 \int_{t_0}^{t_2} dt_3 \left([\tilde{H}_1, [\tilde{H}_2, \tilde{H}_3]] + [\tilde{H}_3, [\tilde{H}_2, \tilde{H}_1]] \right) \quad (1.27)$$

where $[\tilde{H}_a, \tilde{H}_b]$ is the matrix commutator of the terms, and we write $\tilde{H}_k = \tilde{H}(t_k)$ for simplicity.

For simplicity we can also express the propagator in the form of

$$\hat{U}(t, 0) = \exp(A(t)), \quad A(t) = A_1(t) + A_2(t) + A_3(t) + \dots \quad (1.28)$$

Each order of the series is a correction for the time ordering of the Hamiltonian, and higher-order expansion allows for larger time steps. The application of Magnus expansion is popular in the context of the mean-field approximation, where the Hamiltonian (Fock matrix) depends itself from the density matrix at the future time.[5] In this case this propagator is used in combination with different predictor schemes which conserve time-reversibility, unitarity and the numerical stability of the propagation.[6]

Recall the propagator equation $\hat{U}$ from 1.21, and the simplest propagator based on the Magnus expansion, is just the first term of the power series 1.24,

$$\Omega(t, t_0) \approx \int_{t_0}^t \tilde{H}_1 dt_1 \quad (1.29)$$

This equation can be numerically integrated with a forward-Euler-like time integrator, but we can have more accurate methods based on second-order methods. A popular second order method, approximate the first term in equation 1.24 by the mid-point rule, so that we have $\mathcal{O}(\Delta t^2)$ time integrator

$$\psi(t_{k+1}) = \exp\left(\Delta t \tilde{H}(t_{k+1/2})\right) \psi(t_k) \quad (1.30)$$

where Δt is the time step, and subscript k is the time index, also we can write it as in the matrix form of operator:

$$\mathbf{U}(t + \Delta t) \approx \exp\left[-i\mathbf{H}\left[\mathbf{P}_{(t+\frac{\Delta t}{2})}\right]\Delta t\right] \quad (1.31)$$

This method is also called as the second-order midpoint Magnus propagator, which is unitary and $\mathbf{H}$ is Hermitian, also the matrix exponential is evaluated directly by diagonalization.

The $\mathbf{H}$ matrix at time $t + \Delta t/2$ depend on the density matrix $\mathbf{P}_{(t+\frac{\Delta t}{2})}$ but t is not available at time, $t + \frac{\Delta t}{2}$. Thus we can obtain it using an iterative series of extrapolations and interpolations at each time. If the predictor / corrector procedure is converged by self-consistent, then the second-order Magnus propagator preserves the time reversal symmetry.[4]

1.2.3 From Schrödinger equation to Time-Dependent Equations for approximate wave functions

Solving the real-time Schrödinger equation is successful in explaining many experimental results, but we resort to an approximate method because it is computationally prohibitive to solve systems with more than two electrons. We will discuss, in the following, the use of variational principle to generate equations of motion for approximate wave functions. The theory is described in details by Tannor and McLachlan[7, 8]. We will main focused on the use of McLachlan variational principle used to derive the working equations for the real-time TD-Hartree-Fock method, which will be extensively applied in this thesis.

The problem to solve is to find an approximate solution of the time-dependent Schrödinger equation. First from the time-dependent Schrödinger equation,

$$i\hbar \frac{\partial}{\partial t} \Psi = \hat{H} \Psi \quad (1.32)$$

and with small increment Δt , we can express Ψ at time $(t + \Delta t)$ as:

$$\Psi(t + \Delta t) = \Psi(t) - \frac{i}{\hbar} \theta(t) \Delta t \quad (1.33)$$

where we define $\theta(t)$ as an unknown function:

$$\theta(t) = i\hbar \frac{\partial}{\partial t} \Psi \quad (1.34)$$

In variational principle, we write a functional that is positive defined and that is zero for the exact wavefunction. We suppose that $\Psi(t)$ is given, but not the rate of change. If Ψ were an exact solution to the Time-Dependent Schrödinger Equation, then $\theta = \hat{H} \Psi$. Since this is not true, the θ should vary. Then we can define a variational functional called McLachlan functional $\mathcal{J}$:

$$\mathcal{J} = \int |\theta(t) - H\Psi(t)|^2 d\mathcal{V} = \int |\chi|^2 d\mathcal{V} \quad (1.35)$$

and we minimize the equation subject to variation in θ . The $d\mathcal{V}$ is the volume $dq_1 dq_2 \dots dq_N$, where N is the number of degrees of freedom, and $\chi = \theta(t) - H\Psi(t)$. The equation should be a minimum for all allowed variations of the function θ . By this method, we can find values of $\partial\Psi/\partial t$ at successive instants of time, and build the time variation of Ψ . Since $\mathcal{J}$ is always positive, the variational principle always leads to a definite minimum, and $\mathcal{J}$ will only vanish for the exact solution of Eq.1.32.

By varying θ , we can get

$$\delta\mathcal{J} = \int \delta\theta^* (\theta(t) - H\Psi(t)) d\mathcal{V} + \int (\theta(t) - H\Psi(t))^* \delta\theta d\mathcal{V} = 0 \quad (1.36)$$

for all variations of θ .

Now we can look at the application of the above derivation. We can use this McLachlan variational functional to obtain the from of the Time-dependent Schrödinger

equation in a finite basis. We choose a set of N functions, $\{\Psi_n\}$ and expand Ψ and θ in them because of the linear combination of finite orthogonal fixed functions Φ_n :

$$\Psi(t) = \sum_{m=1}^N c_m(t) \Phi_m \quad (1.37)$$

$$\theta(t) = \sum_{n=1}^N d_n(t) \Phi_n \quad (1.38)$$

We can also introduce another set of function, $\{\Phi_\alpha\}$ such that the union, or Φ_n $\{\Phi_\alpha\}$ together form a complete set. We then use the McLachlan principle and find that :

$$\theta - H\Psi = \sum_{n=1}^N \left(d_n(t) - \sum_{m=1}^N H_{mn} c_m(t) \right) \Phi_n - \sum_{m=1}^N \sum_{\alpha} H_{\alpha m} c_m(t) \Phi_\alpha \quad (1.39)$$

The variation $\mathcal{J}$ is therefore

$$\delta\mathcal{J} = \sum_{n=1}^N \delta d_n^* \left(d_n - \sum_{m=1}^N H_{mn} c_m \right) + c.c. = 0 \quad (1.40)$$

We can see that the following equation is obtained, because we can treat δd_n and δd_n^* as independent quantities, as the variations of d_n are completely unrestricted.

$$d_n(t) = \sum_{m=1}^N H_{mn} c_m(t) \quad (1.41)$$

The above equation follow the variational principle,

$$i\hbar \frac{\partial c_n}{\partial t} = \sum_{m=1}^N H_{mn} c_m(t) \quad (1.42)$$

which represents the time-dependent Schrödinger equation in a finite basis.

More important here, we can also use McLachlan variational functional in time-dependent self-consistent field method, to solve multidimensional problems. Suppose we want to approximate a many-particle wavefunction, as a product of single particle wavefunctions:

$$\Psi(q_1, q_2, \dots, q_N, t) = \psi_1(q_1, t) \psi_2(q_2, t) \dots \psi_N(q_N, t) \quad (1.43)$$

where q_i is the coordinates of the system i . For the Hamiltonian describing two-body interactions,

$$\hat{H} = \hat{H}_0 + \hat{V} = \sum_{i=1}^N \hat{H}_i(q_i) + \sum_{i=1}^N \sum_{j>i}^N \hat{V}_{ij}(q_i, q_j) \quad (1.44)$$

To find the optimal choice of $\{\psi_i\}$, first we can vary each of the one-particle wavefunctions (orbitals) independently,

$$\psi_i(q_i, t + \Delta t) = \psi_i(q_i, t) - \frac{i}{\hbar} \theta_i(q_i, t) \Delta t \quad (1.45)$$

The change or total time variation Θ , in the many-body wavefunctions is then given by

$$\Theta(q_1, q_2, \dots, q_N, t) = \sum_{i=1}^N \{\psi_1(q_1, t) \dots \theta_i(q_i, t) \dots \psi_N(q_N, t)\} \quad (1.46)$$

From the equation 1.36 above , :

$$\delta \mathcal{J} = \int \delta \theta^* \chi d\mathcal{V} + \text{complex conjugate} \quad (1.47)$$

where the χ is written as

$$\begin{aligned} \chi &= \theta - \hat{H} \Psi \\ &= \sum_{i=1}^N \{\psi_1(q_1, t) \dots [\theta_i(q_i, t) - \hat{H}_i \psi_i(q_i, t)] \dots \psi_N(q_N, t)\} \\ &\quad - \sum_{i=1}^N \sum_{j>i}^N \{\psi_1(q_1, t) \dots [\hat{V}_{ij}(q_i, q_j) \psi_i(q_i, t) \psi_j(q_j, t)] \dots \psi_N(q_N, t)\} \end{aligned} \quad (1.48)$$

Setting $\Delta \mathcal{J} = 0$ leads to a set of N equations for the N unknown ψ_i , the variational principle leads to equation:

$$i\hbar \frac{\partial \psi_i}{\partial t} = (\hat{H}_i + \hat{G}_i + \lambda_i) \psi_i \quad (1.49)$$

where G_i is the average potential, and λ_i is a number. Both G and λ vary with time,

$$\hat{G}_i(q_i, t) = \sum_{j \neq i=1}^N \int \psi_j(q_j, t) \hat{V}_{ij}(q_i, q_j) \psi_j(q_j, t) dq_j \quad (1.50)$$

$$\lambda_i(t) = \sum_{k \neq i=1}^N \langle \hat{H}_k - \epsilon_k \rangle + \sum_{k \neq i}^N \sum_{j>k \neq i}^N \langle \hat{V}_{kj}(q_k, q_j) \rangle \quad (1.51)$$

where the brackets represent the averages, and the ϵ_i is

$$\epsilon_i(t) = i\hbar \int \psi_i^* \frac{\partial \psi_i}{\partial t} dq_i \quad (1.52)$$

from the equation 1.49 , 1.51 and 1.52, we can get the following relations

$$\begin{aligned}
\epsilon_i &= \langle H_i + G_i + \lambda_i \rangle \\
\sum_{i=1}^N \epsilon_i &= \langle H \rangle \\
\sum_{i=1}^N \lambda_i &= -\langle V \rangle
\end{aligned} \tag{1.53}$$

The presence of λ_i in equation 1.49 is inconvenient, since it depends on the integrals over all the coordinates. But the λ_i here are arbitrary, since changing the pair of them does not change the solution, as long as the sum, $\sum \lambda_i$ remains constant.

We can show this by example, we can change any pair, so that the sum $\sum_i \lambda_i$ remains constant, and the transformation

$$\begin{aligned}
\lambda_i &\rightarrow \lambda_i + \lambda, & \lambda_j &\rightarrow \lambda_j + \lambda, \\
\psi_i &\rightarrow \psi_i \exp(-is/\hbar), & \psi_j &\rightarrow \psi_j \exp(-is/\hbar) \\
s(t) &= \int \lambda(t) dt
\end{aligned} \tag{1.54}$$

will give an equivalent solution of 1.49. So instead of solving 1.49, we may solve:

$$i\hbar \frac{\partial \phi_i}{\partial t} = (\hat{H}_i + \hat{G}_i) \phi_i \tag{1.55}$$

by taking new individual functions $\phi_i(q_i, t)$ which satisfy the time-dependent self-consistent field equations, and we can construct the total wave function that is:

$$\begin{aligned}
\Psi(q_1, q_2, \dots, q_n, t) &= \phi_1(q_1, t) \cdots \phi_N(q_N, t) \exp[-iS(t)/\hbar] \\
&= \left(\prod_{i=1}^N \phi_i(q_i, t) e^{-\frac{i}{\hbar} S(t)} \right)
\end{aligned} \tag{1.56}$$

where

$$\hat{S}(t) = \int \sum_i \lambda_i(t') dt' = - \int \langle \hat{V}(t') \rangle dt' \tag{1.57}$$

Equation 1.49 (or 1.55) with equation 1.43 (or 1.56) together formed the time-dependent Hartree (TDH) or time-dependent self-consistent field (TDSCF) method. TDH means that all particles in the system are identical, and hence $\hat{H}_i$ and $\hat{V}_{ij}$ have the same form.

Then from 1.55 we may write a time-dependent variant of the Hartree-Fock equation, in terms of the Fock operator in the Hartree-Fock scheme as:

$$\hat{F}(r, t) \phi(r, t) = i\hbar \frac{\partial \phi(r, t)}{\partial t} \tag{1.58}$$

this $\hat{F}(r, t)$ is the time-dependent Fock operator, which is same as time independent Fock operator $\hat{F}(r) = \sum_i^n \hat{f}(r)$, but contains the time-dependent single-particle potential $\hat{V}(r, t)$. Since the single particle wave-functions $\phi(r, t)$ are now time-dependent, we now also have the coulomb and exchange operator (in the matrix form) acquire a time dependence.

The Fock operator has the form :

$$\hat{F} = \hat{h} + \hat{G}(P) \quad (1.59)$$

as an effective Hamiltonian $\hat{H}(t)$ in Liouville von Neumann equation, in atomic orbital basis set where $\hat{h}$ is one electron contribution to Fock operator, and $\hat{G}(P)$ as two electron operator. Since we know that the density matrix $\mathbf{P}(t)$ is complex, we can also expect propagated Fock matrices to be complex, because a complex P is used to form a two-electron $\hat{G}(P)$ part of the Fock operator.[5] For each time step, the Fock matrix is reconstructed, by using the current external field and electron density distribution.

The two electron operator depends on the density matrix, and can be written as:[5]

$$\hat{G}(P)_{\mu,\nu} = \sum_{\lambda,\sigma} \langle \mu, \lambda || \nu, \sigma \rangle \mathbf{P}_{\lambda,\sigma} = \hat{J}_{\mu,\nu} - \hat{K}_{\mu,\nu} \quad (1.60)$$

where the coulomb operator $\hat{J}_{\mu,\nu}$ and exchange operator $\hat{K}_{\mu,\nu}$ are:

$$\begin{aligned} \hat{J}_{\mu,\nu} &= \sum_{\lambda,\sigma} \langle \mu, \lambda | r_{12}^{-1} | \nu, \sigma \rangle \mathbf{P}_{\lambda,\sigma} \\ \hat{K}_{\mu,\nu} &= \sum_{\lambda,\sigma} \langle \mu, \lambda | r_{12}^{-1} | \sigma, \nu \rangle \mathbf{P}_{\lambda,\sigma} \end{aligned} \quad (1.61)$$

Since the time-dependent molecular orbitals, ϕ_i are often a linear combination of the basis function χ_u , so that we can write it as:

$$\phi_i = \sum_{\mu} c_{\mu,i}(t) \chi_{\mu} \quad (1.62)$$

such that the $c_{\mu,i}(t)$ is the time-dependent coefficients. Then we can write the Hartree-Fock density matrix, $\mathbf{P}$ in this basis by

$$\mathbf{P}_{\mu\nu}(t) = \sum_p f_p c_{\mu,p}^*(t) c_{\nu,p}(t) \quad (1.63)$$

and the f_p is the occupation of orbital ϕ_p , we have $f_p = 2$ if it is a restricted closed shell system, or 0 if it is unoccupied.

Now we look at the derivation of the expression of Liouville-von Neumann equation. Starting from the equation 1.58 and from now on, we have the equation in orthonormal basis and drop the t for simplicity,

$$\hat{F}(r) \phi_v(r) = i \frac{\partial \phi_v(r)}{\partial t} \quad (1.64)$$

and the molecular orbitals in an orthonormal basis in M basis function is expressed as

$$\phi_i(r) = \sum_{\mu=1}^M c_{\mu,i} \chi_{\mu} \quad (1.65)$$

Substituting the expression using index ν ,

$$i\hbar \frac{\partial}{\partial t} \left(\sum_{\nu} c_{\nu,i} \chi_{\nu}(r) \right) = \hat{F} \left(\sum_{\nu} c_{\nu,i} \chi_{\nu}(r) \right) \quad (1.66)$$

multiplying by $\phi_{\mu}^*(r)$ on the left and integrating, we will get the equation

$$i\hbar \frac{\partial c_{\nu,i}}{\partial t} = \sum_{\nu} \mathbf{F}'_{\mu,\nu} c_{\nu,i} \quad (1.67)$$

where $c_{\nu,i}$ is in its vector form and $\mathbf{F}'_{\mu,\nu}$ the Fock matrix. Therefore, for i in 1 to M basis functions, we can instead write as matrix equation

$$i\hbar \frac{\partial \mathbf{C}}{\partial t} = \mathbf{F}' \mathbf{C} \quad (1.68)$$

The complex conjugate form of this matrix equation is

$$\begin{aligned} \left(i\hbar \frac{\partial \mathbf{C}}{\partial t} \right)^{\dagger} &= (\mathbf{F}' \mathbf{C})^{\dagger} \\ -i\hbar \frac{\partial \mathbf{C}^{\dagger}}{\partial t} &= \mathbf{C}^{\dagger} \mathbf{F}'^{\dagger} \end{aligned} \quad (1.69)$$

and they both have the same dimension $M \times M$.

Since we know that $\mathbf{P}' = \mathbf{C} \mathbf{C}^{\dagger}$, we can write the first equations multiply by the $\mathbf{C}^{\dagger}$

$$\begin{aligned} i\hbar \left(\frac{\partial \mathbf{C}}{\partial t} \right) \mathbf{C}^{\dagger} &= \mathbf{F}' \mathbf{C} \mathbf{C}^{\dagger} \\ &= \mathbf{F}' \mathbf{P}' \end{aligned} \quad (1.70)$$

In the same way with its conjugated equation, we can show that

$$\begin{aligned} -i\hbar \mathbf{C} \frac{\partial \mathbf{C}^{\dagger}}{\partial t} &= \mathbf{C} \mathbf{C}^{\dagger} \mathbf{F}'^{\dagger} \\ &= \mathbf{P}' \mathbf{F}' \end{aligned} \quad (1.71)$$

where the $\mathbf{F}'^{\dagger} = \mathbf{F}'$ because $\mathbf{F}'$ is Hermitian.

When we subtract the Equation 1.71 from Equation 1.70,

$$\begin{aligned}
 i\hbar \left(\frac{\partial \mathbf{C}}{\partial t} \right) \mathbf{C}^\dagger + i\hbar \mathbf{C} \left(\frac{\partial \mathbf{C}^\dagger}{\partial t} \right) &= \mathbf{F}' \mathbf{P}' - \mathbf{P}' \mathbf{F}' \\
 &= [\mathbf{F}', \mathbf{P}'] \\
 i\hbar \frac{\partial \mathbf{P}'}{\partial t} &= [\mathbf{F}', \mathbf{P}']
 \end{aligned} \tag{1.72}$$

Here we write orthonormal basis $\mathbf{P}$ as $\mathbf{P}'$, and the Fock matrix $\mathbf{F}'$ for TDHF, put $\hbar$ in atomic units and hence:

$$\begin{aligned}
 i \frac{\partial \mathbf{P}'(t)}{\partial t} &= [\mathbf{F}'(t), \mathbf{P}'(t)] \\
 &= \mathbf{F}'(t) \mathbf{P}'(t) - \mathbf{P}'(t) \mathbf{F}'(t)
 \end{aligned} \tag{1.73}$$

Where the $\mathbf{F}'(t), \mathbf{P}'(t)$ are the time-dependent density and Fock matrices respectively, and i is the imaginary unit.

We can compute the entire Fock matrix (or Hamiltonian) at the mid point of the step, using a modified mid-point algorithm, called as midpoint unitary transformation (MMUT) equation :

$$\mathbf{U}(t_k) = \exp(-2i\Delta t \mathbf{F}(t_i)), \tag{1.74}$$

$$\mathbf{P}'(t_{k+1}) = \mathbf{U}(t_k) \mathbf{P}'(t_{k-1}) \mathbf{U}^\dagger(t_k) \tag{1.75}$$

where the time-evolution matrix $\mathbf{U}(t_k) = \exp[-i2\Delta t \mathbf{H}(t_k)]$ can be constructed using various methods, such as direct diagonalization, or power series. For small Hamiltonian matrices, we can construct the time evolution matrix, using the eigenvectors $\mathbf{C}(t_k)$ and eigenvalues $\epsilon(t_k)$ of the matrix representation of the Hamiltonian at time t_k :

$$\mathbf{C}^\dagger(t_k) \cdot \mathbf{F}'(t_k) \cdot \mathbf{C}(t_k) = \epsilon(t_k) \tag{1.76}$$

$$\mathbf{U}(t_k) = \mathbf{C}(t_k) \cdot \exp[-i2\Delta t \epsilon(t_k)] \cdot \mathbf{C}^\dagger(t_k) \tag{1.77}$$

We define $\mathbf{W} = -i2\Delta t \mathbf{F}(t_k)$, and write MMUT equation in the matrix form,

$$\mathbf{P}'(t_{k+1}) = e^{\mathbf{W}_k} \mathbf{P}'(t_{k-1}) e^{-\mathbf{W}_k} \tag{1.78}$$

and we can use BCH expansion to evolve the density matrix as:

$$\begin{aligned}
 \mathbf{P}'(t_{k+1}) &= \mathbf{P}'(t_{k-1}) + \frac{1}{1!} [\mathbf{W}_k, \mathbf{P}'(t_{k-1})] \\
 &+ \frac{1}{2!} [\mathbf{W}_k, [\mathbf{W}_k, \mathbf{P}'(t_{k-1})]] \\
 &+ \frac{1}{3!} [\mathbf{W}_k, [\mathbf{W}_k, [\mathbf{W}_k, \mathbf{P}'(t_{k-1})]]] + \dots
 \end{aligned} \tag{1.79}$$

The BCH expansion has been shown to have superior convergence properties, as compared to simple power series expansion.

The main challenge to implement the real-time methods, involves the development of numerically stable and computationally efficient time evolution propagator. This is because we want to propagate the Fock matrix many times at each time step.

In this thesis we focus on the TDHF, because we know by theory the computation of the entire Hamiltonian. We can also generate training data for large molecular systems. Also, the Hartree-Fock theory is the first model within a sequence of increasingly accurate models, that leads to full configuration interaction. Therefore, by studying a machine learning problem from TDHF, we can produce insights and generalize it to more accurate models for quantum dynamics.[9]

Real-time method can be very efficient to compute spectra in molecules and materials with a high density of states. This is because we can compute an entire absorption spectrum from a single real-time simulation. Therefore, this requires computing the frequency-dependent response of the system, via Fourier transform (FT) of the dipole moment, following interaction with laser pulse with sufficient bandwidth to cover a spectral region of interest.

Usually, we use a narrow-in-time pulse, or a delta function. However, we can see the disadvantages in the real-time methods:

1. Firstly, the spectra generated from the dipole moment, do not contain any information about the molecular orbitals, or excited states involved in each transition, which is where the interpretation of spectra takes place.
2. Secondly, long simulation times are needed to resolve spectra via Fourier Transform, with denser spectra requiring longer simulations.

1.2.4 Absorption in real-time time-dependent electronic structure methods

We can derive an expression for the absorption cross section in terms of time-dependent observables, to compute spectroscopic observables. We define the absorption cross section $\sigma(\omega)$ as the ratio of the average number of photons absorbed ($\Delta N(\omega)$), to the average number of incident photons per unit area ($N(\omega)/A$) at a given frequency ω

$$\sigma(\omega) = \frac{\delta N(\omega)}{N(\omega)} A \quad (1.80)$$

The average rate of photons absorbed, dN/dt is expressed by

$$\frac{dN}{dt} = \frac{2}{\hbar} \text{Im} \left[\mathbf{E}(t) \cdot \sum_n \langle \psi_n(t) | \boldsymbol{\mu}_{n0} | \psi_0(t) \rangle \right] \quad (1.81)$$

$$\frac{dN}{dt} = \frac{2}{\hbar} \text{Im}[\mathbf{E}(t) \cdot \mathbf{D}(t)] \quad (1.82)$$

and we specify that $\mathbf{D}(t) = \sum_n \langle \psi_n(t) | \boldsymbol{\mu}_{n0} | \psi_0(t) \rangle$, as the time-dependent transition dipole moment.

To find the average total number of photons absorbed, we can integrate the above equation to get

$$\Delta N = \frac{2}{\hbar} \text{Im} \int_{-\infty}^{\infty} \mathbf{E}(t) \cdot \mathbf{D}(t) dt = \frac{4\pi}{\hbar} \text{Im} \int_{-\infty}^{\infty} \tilde{\mathbf{E}}(\omega) \cdot \tilde{\mathbf{D}}(\omega) d\omega \quad (1.83)$$

the tilde notation indicates the Fourier transform pairs. We use the antisymmetric definition of the Fourier transform, such that the forward Fourier transform,

$$f(t) = \int_{-\infty}^{\infty} \tilde{f}(\omega) e^{-i\omega t} d\omega \quad (1.84)$$

and the backward Fourier transform is,

$$\tilde{f}(\omega) = \frac{1}{2\pi} \int_{-\infty}^{\infty} f(t) e^{+i\omega t} dt \quad (1.85)$$

For the absorption cross section, we can look at single frequencies ω , so that the average number of photons absorbed for a given frequency of incident light is given by:

$$\Delta N(\omega) = \frac{4\pi}{\hbar} \text{Im}[\tilde{\mathbf{E}}(\omega) \cdot \tilde{\mathbf{D}}(\omega)] \quad (1.86)$$

From the classical electrodynamics, we know that for a given frequency, the average number of photons incident per unit area, $N(\omega)/A$, is given by

$$N(\omega)/A = \frac{|\tilde{\mathbf{E}}(\omega)|^2 c}{\hbar \omega} \quad (1.87)$$

and combining the equations, our absorption cross section will be

$$\sigma(\omega) = \frac{4\pi\omega}{c} \frac{\text{Im}[\tilde{\mathbf{E}}(\omega) \cdot \tilde{\mathbf{D}}(\omega)]}{|\tilde{\mathbf{E}}(\omega)|^2} \quad (1.88)$$

We would consider a suitable electric field shape, which is a delta-impulse function, to access $\sigma(\omega)$. The delta-impulse function is

$$\mathbf{E}(t) = \delta(t - t_0) \boldsymbol{\kappa} \quad (1.89)$$

where the $\boldsymbol{\kappa}$ collects the intensity of the applied field. Here, the Fourier transform of $\mathbf{E}(t)$ is just $\tilde{\mathbf{E}}(\omega) = \frac{1}{2\pi} \boldsymbol{\kappa}$.

If we consider an impulsive field along a single axis i , then the equation 1.88 takes the form

$$\sigma_{ii}(\omega) = \frac{4\pi\omega}{c} \frac{\text{Im}[\tilde{D}_i(\omega)]}{\kappa_i} \quad (1.90)$$

so that when we have three pulses along each Cartesian axis, we can obtain the rotational isotropic average of:

$$\sigma(\omega) = \frac{4\pi\omega}{3c} \sum_{i=1}^3 \frac{\text{Im}[\tilde{D}_i(\omega)]}{\kappa_i} \quad (1.91)$$

This means that the absorption cross section is the imaginary component of the Fourier transform of the time-dependent transition dipole moment, scaled by the field intensity. In the real-time time-dependent electronic structure theory, if the system is initially in its ground state, and we have weak perturbing field, the $\tilde{\mathbf{D}}(\omega)$ is given by:

$$\tilde{\mathbf{D}}(\omega) \approx \frac{1}{2\pi} \int_{-\infty}^{\infty} (\boldsymbol{\mu}(t) - \boldsymbol{\mu}(0)) e^{i\omega t} dt \quad (1.92)$$

since $\boldsymbol{\mu}(t) = \text{Tr}(\mathbf{P}(t) \cdot \boldsymbol{\mu})$, and the time-dependent density matrix $\mathbf{P}(t)$ is given to the first-order perturbation theory as:

$$P(t) = |\psi(t)\rangle \langle \psi(t)| \approx |\psi_0\rangle \langle \psi_0| + \sum_n (c_n(t) e^{-i(\omega_n - \omega_0)t} |\psi_n\rangle \langle \psi_0| + c.c) \quad (1.93)$$

The first term gives the static dipole moment when traced with $\boldsymbol{\mu}$, and the second term gives the time-dependent transition dipole moment when traced with $\boldsymbol{\mu}$.

The main drawback of real-time approaches over the frequency domain methods, is we require long simulation times to adequately converge a spectrum, by the Fourier transform. This is because of the Fourier uncertainty principle. This issue is more prominent for high energy spectra with high spectral densities, such as those relevant to X-ray absorption.

1.2.5 Comparing with Real-time with Linear response

Light matter interaction can be classified as weak or strong interactions. Weak interaction, is interaction with the field which is smaller than the magnitude of intermolecular interaction. The excitation is weak, and results in small perturbation away from the ground state. In this regime, we can use perturbation theories such as linear-response or frequency domain density functional theory (LR-TDDFT) to describe the linear-properties of molecules and materials. While strong interaction is otherwise, and it requires methodologies beyond linear response.[10]

Therefore, we can apply TDDFT in two approaches. First is the transition-based linear-response TDDFT (LR-TDDFT), and the second is the real-time formalism (RT-TDDFT). The LR-TDDFT method can give us the energies and transition densities of the excitations, but it requires the calculation of large number of empty electron states that makes it cumbersome for large systems. On the contrary, RT-TDDFT works without the evaluation of empty orbitals, which is advantageous to use in large systems.[11]

RT-TDDFT is an alternative approach traditional response formulation, that allow us to capture the effects of external fields of molecules, and materials in both weak and strong interaction regimes.[10]

Even though RT-TDDFT requires long simulation times when compared with LR-TDDFT for the calculation of absorption spectra in small systems, it is effective for the calculation of wide absorption spectra of large complexes, as well as systems with very high density of states. This is an advantage when compared to LR approach, where it is impractical because of very large number of excitations that must be considered.[10]

1.3 Application of real-time methods

The real-time methods, are efficient for computing the spectra in molecules and materials with a high density of states, so that the entire absorption spectrum can be computed from a single real-time simulation. This is done by using the delta function to have sufficient bandwidth to cover a spectral region of interest. It has been used extensively in spectroscopy and in the study of electronic responses. Below we give a brief review about the applications of different real-time electronic structure methods, so that we can appreciate more about the importance of this project.

1.3.1 UV / Vis Spectroscopy

Ultraviolet-visible (UV-vis) spectroscopy is used to obtain the absorbance spectra of a compound in a solution or solid. We measure the absorbance of light energy or electromagnetic radiation, that excite the electron from ground state to excited state.

This real-time spectroscopic technique, has the power to provide important information on the evolution of fast dynamic processes, such as electronic transitions and compositional changes that are not easily seen by conventional UV-vis spectroscopy that require wavelength tuning.[12] It also has the advantage to solve the lowest energy eigenstates because it can simulate the linear (UV-vis) absorption spectrum of a system depends on the implementation, the requested number of states, the desired resolution of the spectrum, and time-step for numerical propagation.

We can obtain the UV-vis liner absorption spectrum, by the Fourier transform of the field-free time-dependent dipole moment, after perturbation by a weak, off-resonant perturbation. This is often an electric field applied as a delta function.

Yabana and Bertsch computed the optical spectra for the benzene molecule. They have demonstrated the utility of real-time approach, using the electromagnetic response of conjugated carbon molecules. In their work, they successfully inspected the optical absorption spectra of simple aromatic molecules using time-dependent local-density approximation (TDLDA) calculations, which means adding a density-dependent contact term to the Hartree Hamiltonian.[13] Rubio and coworkers[14] performed the first principles calculations of the optical response of green fluorescent protein (GFP) using a combined QM/MM and TDDFT approach. They have measured the main excitation peaks, cationic conformation spectrum, and those calculated values are with good

agreement with experiment values. This is important to the first-principle description of the excited-state dynamic of important biological photoreceptors.

1.3.2 X-ray Absorption Spectroscopy

X-ray absorption spectroscopy (XAS) is a method to investigate atomic local structure and electronic states. An X-ray strikes the atom, and excites a core electron that can either be promoted to an unoccupied level, or ejected from the atom. It is absolutely necessary in the experimental techniques in materials and chemical sciences, due to the recent advancement in synchrotron technology that enable us to harness improved temporal and energy resolution of X-ray photons. We can get many insights about the specific chemical processes by inspecting the local electronic and binding environment, from the excitation of core electrons to unoccupied bound orbitals.

The presence of saw-tooth-like features with a sharp rise at discrete energies, are called absorption edges. They occur near the ionization energy of the inner shell electrons.[15] There are K-, L- and M-edge spectra generated from the XAS excitation of electrons in principle quantum number $n=1$, $n=2$, $n=3$ orbitals, respectively. Many ab-initio methods, have been developed to model core excitations, and most approaches operate within the frequency domain. We can get the K-edge spectra from linear response TDDFT, and many other methods. The L-edge XAS spectra, can be computed from the relativistic two-component LR-TDDFT. XAS involves the transition of a core-level electron to either bound or continuum states, and can be broadly recognized as pre-edge, X-ray absorption near-edge structure (XANES).

Currently, Fernando and coworkers have used RT-TDDFT approach to XAS to compute the near-edge and post-edge X-ray absorption spectrum of insulators.[16] They have developed an alternative approach to post-edge XANES using real-time time-dependent DFT with atom-centered basis sets. They want to describe accurately the excitations near and above the edge of wide-gap materials, without any parameterization against experiment. They have successfully use RT-TDDFT to model the X-ray absorption of α -quartz to capture both near- and above-ionization XANES features without input from experiment.

1.3.3 Excited State Absorption and Emission

RT-TDDFT has been shown to be an efficient prediction method to compute molecular absorption and photoelectron spectra. Transient absorption spectroscopy is used to probe the dynamics of excited states, it is one of the most common experimental techniques for studying ultrafast electronic dynamics. This technique works by splitting a laser pulse into two components, where a pump to excite, and a delayed probe to measure absorption. We study the dynamics of electronic excitations by varying the time delay.[17] For the ground-state absorption of large systems, real-time time-dependent density functional theory is an efficient approach for obtaining the spectrum. TDDFT is able to simulate the behaviour of the system in time-dependent external fields. From the pump probe experiment by De Giovannini and coworkers[18], they

showed the extended techniques to study time-resolved pump probe experiments, which a system response to a probe pulse is measured in excited state.

1.3.4 Nonlinear properties

Real-time approaches has the great advantage to model the non-linear response of a system to electromagnetic radiation. These techniques, allow us to control the pulse shape and field strengths during experiments. The electrodynamic simulation can go beyond the perturbative regime, and record all orders of response simultaneously. Therefore, these real-time approaches are great to describe the highly non-linear process and properties.

High Harmonic generation (HHG) is a technique that produce spatially and temporally coherent light pulse as short as hundred attoseconds, as well as the extreme-ultraviolet (EUV) lights. HHG is an extremely non-linear effect that occurs when an intense pulsed laser beam is focused into atomic, molecular or solid system, and emits photons at high frequencies. The intensity of the laser, is chosen so that its electric field amplitude is comparable to the atom electric field. This electric field can detach the electrons from atoms by tunnel ionization. The detached electron, is accelerated in the field, and has high probability to hit the ion left when it returns. This collision result in emission of high energy photons. The emission of photons are at frequency of integer multiples of the driving field frequency.[19]

We can obtain dynamical and structural information from the HHG spectra, to reveal the laser- and correlation-driven effects in strong field dynamics. For example, the study of different ionisation and recombination channels, molecular imaging, electrodynamics of organic molecules and biomolecules. The electron dynamics involved in the HHG process can be rather complex, and the many-electron dynamics under an influence pulse is obtained by the TDSE. We also have approximated methods developed to study HHF in atoms, molecules and solids.[20]

1.4 Current and future challenges

The time propagation of the wave-function or the density matrix, is the main challenges in real-time techniques. This is because the time scales of the propagation ranging from atto to femto, to even picoseconds. This require large number of steps in the time propagation. Meanwhile, the construction of Hamiltonian, is also crucial in real-time approaches. Time-acceleration approaches, are still a major obstacle, that limits the application of quantum electrodynamics in practice, across all real-time methodologies.

Real-time electronic structure codes, could take advantage of the GPU in high performance computers. Besides, machine learning approaches like artificial neural network, have been demonstrated to estimate efficiently the quantum dynamics propagator for some simple model systems. With reduced scaling approaches, GPU hardware enhancement, the improve of technical aspects have led to faster Hamiltonian construction, but real-time propagation still rely on the sequential time-propagation. Therefore,

we can incorporate machine learning methods to first principle electronic structure theories in the future. Next, we will introduce the basic concepts in machine learning.

Chapter 2

Machine learning Introduction

2.1 Theory of Machine Learning

2.1.1 Introduction

Machine Learning (ML) is defined as the computation paradigm, where the computer can learn the ability to solve the given problem using previous examples.[21] The previous examples that are use to build the capacity are called training examples, and the process of building capacity is called learning. Once the model is trained using data, the model can be used “inexpensively” to predict the same kind of properties for a set of new but similar structures, and pave the way to simulations that can be compared with experiments. High computational cost of quantum mechanic calculations cause limitation of time and length scales accessible to simulations. We may expect that the emergence of ML incorporating physical insights, machine learning can lift the limitations to model the complex materials and molecules in realistic conditions.[22] In the following sections, we will present a general overview at the basics of machine learning.

2.1.2 Types of Learning

There are several ways one can develop machine learning model for different applications. We include some examples of supervised, unsupervised and reinforcement learning. [23] Supervised learning is the learning type that assumed all training examples are labeled. Labeled data is raw data that has been assigned labels to give them context or meaning, to give the machine algorithm to a guide to learn. This is a learning paradigm of building capacity to solve problems based labels of training examples. The machine learning model perform an either regression or classification task, to map a set of known inputs $\mathcal{X}$ and outputs $\mathcal{Y}$. We can map a regression problems like energy and polarizability onto a continuous space, while we can also map classification problem like acidity or base, conductor or insulator into categorical value.[24] The goal is to deduce the relationship between inputs and outputs. The supervised learning is the most popular types of learning, and is usually used to apply in the data classifications and regressions.[21] Classification is the process of classifying items

into predefined categories. In materials science as example, we can classify between insulators and metals when identifying characteristics of the electronic structure.[24] Regression is the process of estimating the continuous values based on the one or more independent features, or input. It finds the relationships between dependent variables, which are the variable we try to predict, and the independent variables which are the input variables that influence the prediction.[25]

We have also unsupervised learning, which the training examples are unlabelled. Unlabelled examples or data are learned depending on their similarities. Data clustering is a typical task of unsupervised learning algorithms.[21] Clustering is the process of separating an entire group into subgroups, based on their similarities among items. The goal is to recover the underlying structure of the data, to gain higher level of understanding. For example, we can use clustering algorithm like k -means algorithm to identify the conformers by grouping data based on common patterns.[24] K -means clustering is a partition method that treats the observations in the data as objects that have distance to each other, and partition the object into k mutually exclusive clusters from its centroid. In the work of D. Ashok kumar and coworker, they have used k -means algorithm to group data such as soil pH, soil electrical conductivity and organic carbon into clusters by finding similarities between chemical properties.[26]

The reinforcement learning, is an autonomous agent.[21] The reinforcement learning involve defining an agent (decision maker), that learns by receiving feedback in the form of punishments and rewards. The agent can learn through trial and error, to maximize cumulative rewards by interacting with environment.[27] Yoon and co-workers have introduced a new framework to predict the kinetic energy pathways to the catalyst surface segregation under reaction, using deep reinforcement learning (DRL, deep learning + reinforcement learning) with catalysis knowledge. The DRL agent repeatedly attempts to explore nearby local minima with changed surface compositions, to explore and learn based on the properties of local minima and the transition states the agent discovered.[28]

2.1.3 Machine Learning Workflow

Here we summarize the overall machine learning process. The typical machine learning workflow is the following:

1. Gathering and preparing the data
2. Choosing a numerical representation (i.e., features / descriptors)
3. Train the model
 - 3.1. Select and train the most appropriate model
 - 3.2. Evaluation of model
 - 3.3. Tune the hyperparameter
4. Testing the model out of the sample

The machine learning models can be seen as parameterization of data sets, which means defining the necessary parameters that define the model. The reference data set, becomes the backbone that define the model’s effectiveness. Therefore, we need the data that represents the problem at hand. When the data that are not properly captured or with bias and noise, it will make the model incomplete and behave unpredictably. If the sampling method is narrow, we will not see much errors during cross-validation, and the model will fail when applied to real problem. Comprehensive data sets will give us robust models, and it is better to have scientific knowledge or intuition about specific problems to have efficient machine learning solution. [24]

To use machine learning, the data set needs to be encoded into a numerical representation, such as features and descriptors that allow the machine learning to extract meaningful patterns and regularities. Molecular descriptors are formally mathematical representations of a molecule obtained by algorithm. This is challenging for atoms of the same type for multiple atoms together, because they are indistinguishable to each other, so we have to impose some kind of order to assign identity to them. Furthermore, the physical systems can be translated and rotated in space, without changing many attributes. We need to find representations that can adapt to those transformations to solve the learning efficiently.[24]

The training process is the key step to combine the data set and model architecture. When we choose the model architecture, we defined a function space of possible solutions, which is then conditioned on the training data set by selecting suitable parameters. When optimizing the task, we want to have the loss function that gives us good fit of data, meanwhile keeping the parameterization general such that the trained model can be used in new data.[24]

2.2 General applications in chemistry

Now we look at some applications of machine learning in chemistry. Kang and coworkers have used random forest regression method to predict the excitation energies and associated oscillator strengths of a given molecule.[29] Random forest is an ensemble learning method operates by constructing multiple decision trees during training, and averaging the prediction values of all trees to produce final prediction. Each decision tree starts from a root node that encompass all data points, and the nodes will divide into two new subnodes by randomly chosen feature. The splitting are decided by minimizing the mean square error. In their work, they used half million DFT results to train the model, and have high accuracy prediction for the maximum oscillator strength and corresponding excitation energy of a molecule. They have also identified the molecular fragments and molecular descriptors that are important for determining the oscillator strength of a molecule.

Retrosynthesis is also known as synthetic planning, which is the process of planning synthetic routes for a target product from readily available starting materials. This is a challenge in chemistry to understand synthetic pathways to desired molecules. Machine learning based retrosynthesis approaches to improve traditional computer-aided retrosynthetic approaches have been introduced. For example, a program Chematica

implemented a new modules that combines network theory, high-power computing, artificial intelligence and chemical knowledge to design synthetic pathways. A scoring function is used to promote synthetic brevity, and penalize the reactivity conflicts that is difficult for human to identify. [24, 30]

Besides, we can apply the concept of machine learning in catalysis. It is found that machine learning process has been assisting in making better models, understanding catalytic mechanism, and providing insights in catalytic design. In the homogeneous catalysts, we can apply linear regression method to establish the quantitative relationship between the structural descriptors and catalytic properties. For example, the multiple linear regression method was applied to make catalytic activity prediction of different analogues of bis(arylimino)pyridine metal complexes.[31]

2.3 Application in real-time electron dynamics

This section describes the applications of machine learning in real-time electron dynamics. Bhat and coworkers developed a computational method to learn a molecular Hartree-Fock matrix, from matrix-valued time series of the electron density.[1] The learned Fock matrix can be used for evolution of electron density, and produce highly accurate result when propagate for 1000 steps beyond the training data. The method they used is the reduction to least square optimization, to solve the minimization of $\tilde{\beta}^* = \arg \min_{\tilde{\beta}} \mathcal{L}(\tilde{\beta})$ for which $\mathcal{L}(\tilde{\beta})$ is the sum of squared loss function of prediction with actual value. The loss function is used to measure the model performance, by calculating the model's prediction from the true value. We will describe in detail for loss function in the neural network section.

Boyer and coworkers demonstrated the use of moment propagation theory (MPT) with machine learning techniques (MPT-ML) for real systems through the use of RT-TDDFT simulation.[32] The moment propagation theory, describes a single-particle wave function in terms of increasing orders of moments, instead of propagating the wave function via Schrödinger-like equation. The coefficients of the time derivative of the moments are to be machine learned from the RT-TDFT simulation.

Secor and coworkers have used artificial neural networks (ANN) to accelerate molecular simulations.[33] They have implemented ANN as propagators of the time-dependent Schrödinger equation to simulate the nuclear quantum dynamics of wavepackets evolving. They have successfully developed two types of ANN propagators to show the proton transfer system. One type of ANN propagators are trained for a specific time-independent potential, while another type of ANN are trained for general diverse potential that can be time-dependent. The first one achieves high level of accuracy for a wide range of time steps, while the latter achieve reasonable accuracy with suitable choice of time step.

From the motivation of exploring possible applications of ML in the electronic structure calculations, we decided to start with simple ML scheme, from linear regression to simple neural network to see that the real-time simulation is compatible with machine learning applications.

2.4 Ordinary least square

One common machine learning method is the linear regression. The most commonly used procedure in the regression analysis, is the ordinary least squares (OLS). This OLS procedure, minimizes the sum of squared residuals.

The regression model is defined as:

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i \quad (2.1)$$

where X_i is the independent variables, and Y_i is the dependent variable. The OLS aims to determine the coefficients intercept β_0 and gradient β_1 in this linear equation. The ϵ represents the error term, that is the deviation of the observed values from the predicted values. For the general model with k independent variables (multilinear regression), we can write it as:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_k X_{ki} + \epsilon_i \quad (2.2)$$

and choosing different β s that minimize the sum of squared residuals. We want to obtain an estimated regression equation,

$$\tilde{Y}_i = \tilde{\beta} + \tilde{\beta}_1 X_i \quad (2.3)$$

We can start to derive the OLS estimators. The residual ϵ is the difference between the observed value Y and the predicted value $\tilde{Y}$,

$$\epsilon_i = Y_i - \tilde{Y}_i = Y_i - (\beta_0 + \beta_1 X_i) \quad (2.4)$$

and we can get the sum of squares

$$S(\beta_0, \beta_1) = \sum_{i=1}^n (Y_i - \beta_0 - \beta_1 X_i)^2 \quad (2.5)$$

To find the estimators $\tilde{\beta}_0$ and $\tilde{\beta}_1$, we can minimize Equation 2.5 with respect to β_0 and β_1 , by taking the partial derivatives of S with respect to β_0 and β_1 , setting them to zero:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_{i=1}^n (Y_i - \beta_0 - \beta_1 X_i) = 0 \quad (2.6)$$

$$\frac{\partial S}{\partial \beta_1} = -2 \sum_{i=1}^n (Y_i - \beta_0 - \beta_1 X_i) X_i = 0 \quad (2.7)$$

Solving the normal equations simultaneously, we can get the OLS estimators. First from Equation 2.6,

$$\sum_{i=1}^n (Y_i - \tilde{\beta}_0 - \tilde{\beta}_1 X_i) = 0 \quad (2.8)$$

Rearranging this expression, and using the fact that $\sum_{i=1}^n Y_i = n\bar{Y}$, we can write as

$$n\tilde{\beta}_0 = n\bar{Y} - n\beta_1\bar{X} \quad (2.9)$$

where we have the $\bar{X}$ as the mean of independent variables and $\bar{Y}$ as mean of dependent variables. Divide the equation by n , we will get the estimator for the intercept,

$$\beta_0 = \bar{Y} - \beta_1\bar{X} \quad (2.10)$$

Then we consider solving for Equation 2.7. Removing the -2 and rearranging, we will get

$$\sum_{i=1}^n X_i Y_i - \beta_0 X_i - \beta_1 X_i^2 = 0 \quad (2.11)$$

substituting 2.10, we will get

$$\sum_{i=1}^n X_i Y_i - (\bar{Y} - \beta_1\bar{X})X_i - \beta_1 X_i^2 = 0 \quad (2.12)$$

distributing the sum,

$$\sum_{i=1}^n X_i Y_i - \bar{Y} \sum_{i=1}^n X_i - \beta_1 \bar{X} \sum_{i=1}^n X_i - \beta_1 \sum_{i=1}^n X_i^2 = 0 \quad (2.13)$$

using the fact that $\sum_{i=1}^n Y_i = n\bar{Y}$, and $\sum_{i=1}^n X_i = n\bar{X}$, solving for $\tilde{\beta}_1$ from 2.13,

$$\sum_{i=1}^n X_i Y_i - n\bar{Y}\bar{X} - \beta_1 \sum_{i=1}^n \frac{1}{n} X_i X_i - \beta_1 n^2 \bar{X}^2 = 0 \quad (2.14)$$

rearranging,

$$\tilde{\beta}_1 = \frac{\sum_{i=1}^n (X_i Y_i - n\bar{X}\bar{Y})}{\sum_{i=1}^n (X_i^2 - n\bar{X}^2)} \quad (2.15)$$

The numerator can be written as:

$$\begin{aligned} & \sum_{i=1}^n (X_i Y_i - n\bar{X}\bar{Y}) \\ &= \sum_{i=1}^n (X_i Y_i - n\bar{X}\bar{Y} - n\bar{X}\bar{Y} + n\bar{X}\bar{Y}) \\ &= \sum_{i=1}^n X_i Y_i - \bar{Y} \sum_{i=1}^n \bar{X} - \bar{X} \sum_{i=1}^n \bar{Y} + \sum_{i=1}^n \bar{X}\bar{Y} \\ &= \sum_{i=1}^n (X_i Y_i - X_i \bar{Y} - \bar{X} Y_i + \bar{X}\bar{Y}) \\ &= \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y}) \end{aligned} \quad (2.16)$$

For the denominator,

$$\begin{aligned}
& \sum_{i=1}^n (X_i^2 - n\bar{X}^2) \\
&= \sum_{i=1}^n (X_i^2 - 2n\bar{X}\bar{X} + n\bar{X}^2) \\
&= \sum_{i=1}^n X_i^2 - 2n\bar{X} \sum_{i=1}^n X_i + \sum_{i=1}^n \bar{X}^2 \\
&= \sum_{i=1}^n (X_i^2 - 2\bar{X}X_i + \bar{X}^2) \\
&= \sum_{i=1}^n (X_i - \bar{X})^2
\end{aligned} \tag{2.17}$$

Therefore, we have the estimators :

$$\tilde{\beta}_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} \tag{2.18}$$

$$\tilde{\beta}_0 = \bar{Y} - \beta_1 \bar{X} \tag{2.19}$$

This show that $\tilde{\beta}_1$ is the covariance of X and Y , divided by the variance of X , meanwhile $\tilde{\beta}_0$ is the intercept obtained by adjusting $\bar{Y}$ by the product $\tilde{\beta}_1 \bar{X}$.

2.5 Neural Network

A Neural Network is a machine learning model that mimic the complex function of the human brain. A biological neuron receive signals from other neurons, it fires its own signals. Each neurons typically connected to thousands of other neurons, the interconnected nodes or neurons, process data and learn patterns, that enable it to perform tasks such as pattern recognition and decision making.[34, 35]

The building block of a neural network, is a single computational unit, a single neuron. A unit takes a set of input, performing computation and produce an output.[36] A single neuron model is represented by its weights, bias and activation function. The weights ($w_0 \cdots$ to w_n), defines the impact of its associated input on the neuron's output. The weights are initialized before training, and updated during training. Meanwhile, the bias b is a constant, and we can view it as the intercept that offset the results. It is also initialized before training, and updated during training. The activation function will define the output of the node in terms of the weighted sum.

The neural network without hidden layers is just a linear model. This can be illustrated as Figure 2.1, which is a single neuron with example for the linear equation $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$. The output of this model is $\tilde{y} = b + (w_0 x_0) + (w_1 x_1)$, without applying activation function or with linear activation function. Therefore, if our task is linear fitting, we do not actually need any hidden layers.

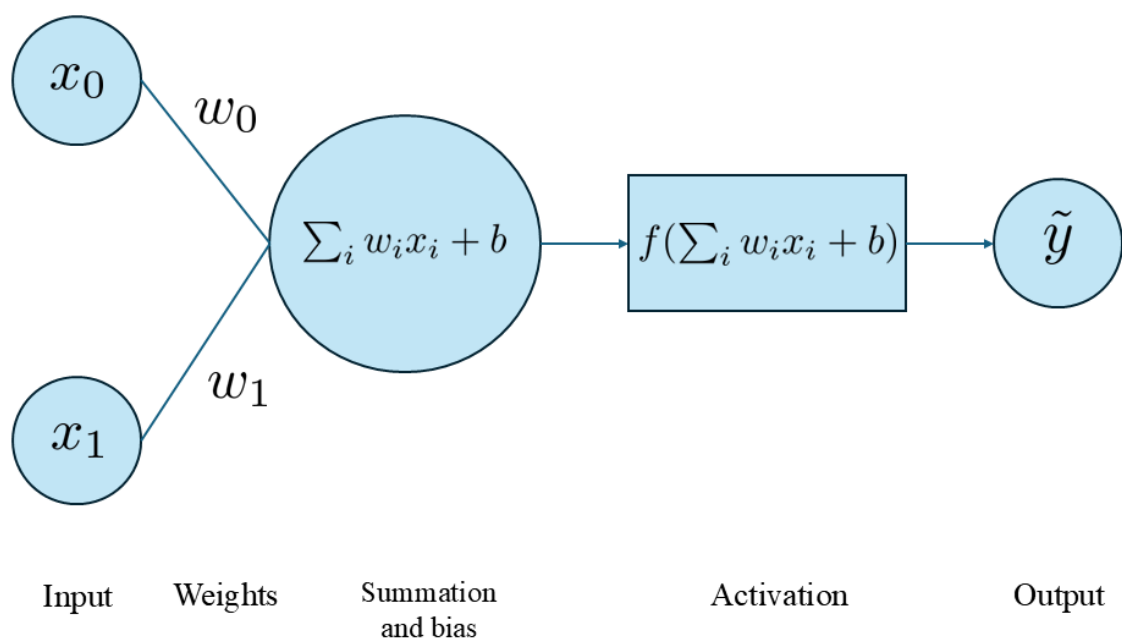


Figure 2.1: A single neuron in a neural network

We will use activation function in the hidden layer to introduce non-linearity. There are different types of activation function. Some of the most popular activation functions are linear, ReLU (Rectified Linear Unit), sigmoid and tanh function. It is important to know the function's codomain, to define appropriately the functions for model output. For example, the ReLU function's codomain is the set of positive numbers.[37] We will explain more about activation function in Section 2.5.3.

For a set of nodes having the same input, it will become a layer. The nodes of the same layer, generally have the same activation function. We have the input layer, which is used to define the number of features (shape) of the model's input. A succession of layers, will become a multi-layer model. Each node of the current layer, is connected to all nodes of the previous / next layer. This is called a fully connected network. In a model, there are three types of layers, called input layer, output layer, and hidden layers. The input layer is the first layer, which is the model's input. The output layer, is the last layer, that its output is the whole model output. We have to choose the types of activation of the layer, for the needed output. The hidden layers, are the layers between the input and the output layers. Figure 2.2 shows a three layer neural network with two hidden layers. When we count the number of layers, we only count the hidden layers and the output layers. We can also use the " a " symbol to represent the function in the neuron.

During the training, the model will find the weights that fit the best for training data. It is the process of updating weights, so that the difference between the predicted value, and the ground truth is small. This is called optimization.

Batch is the number of samples that are used to update the weights. During the training, the data are divided into batches, and the weights are updated according to each batch. An Epoch is when all training data are used, and number of epochs will define how many times the weights are updated along all the training data. It is one of the condition that stops training.

The process of training include the following steps:

1. Initialize the weights and biases.
2. Forward pass, compute the model output.
3. Measure the error between the target output, using the loss function.
4. Backward pass, propagating the error, and update the weights and bias using an optimizer.
5. Repeat steps 2 to 4 for every batch.
6. Repeat the steps 2 to 5 for number of epochs.

The optimizer updates the weights. For example, we have SGD (stochastic Gradient Descent) and Adam optimizers. The Loss function measures how well the model is able to fit the training set, by measuring the difference between the predicted values and ground truth values during training. For regression problem, we can use MSE (mean square error) or MAE (mean absolute error)

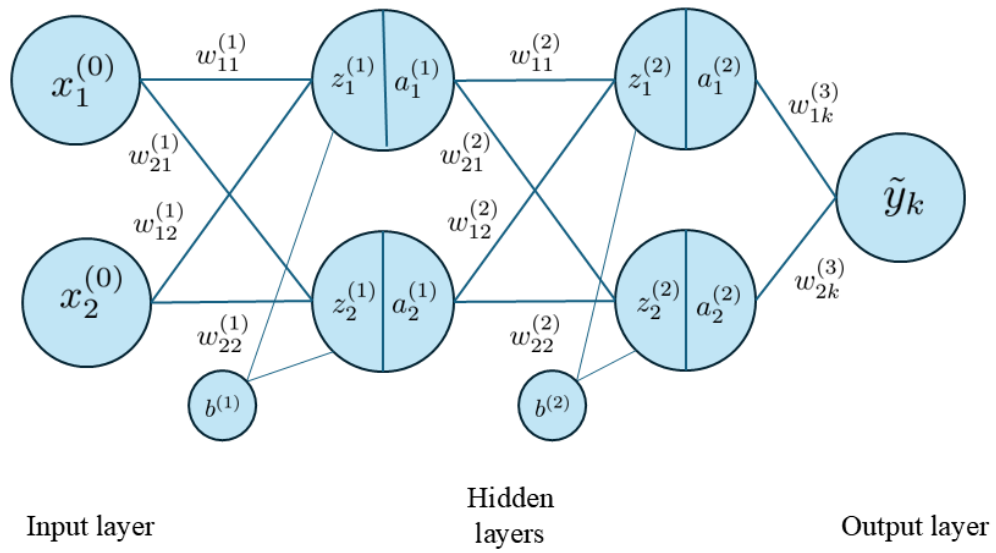


Figure 2.2: A fully connected neural network with 3 layers. The a represents the activation function in the node.

The learning rate will control how the weights are affected by the propagated error. Below we will introduce some of these important concepts in detail.

2.5.1 Forward pass

The forward propagation, is the process where the neural network transforms input data into predictions or outputs. It is a sequential calculation, that moves the data from the input layer to the hidden layers, and finally the output layer. During the process, the data will be transformed by weighted connections and activation function, so that the network can capture complex patterns. We can view the forward propagation as follow:

1. Calculate the hidden layer before apply activation function:

$$z^{(1)} = \mathbf{x} \cdot \mathbf{w}^{(1)T} + b^{(1)} \quad (2.20)$$

2. Apply the activation function to get the hidden layer outputs:

$$a^{(1)} = f_1(z^{(1)}) \quad (2.21)$$

3. Calculate the output layer before activations:

$$z^{(2)} = a^{(1)} \cdot \mathbf{w}^{(2)T} + b^{(2)} \quad (2.22)$$

4. Apply activation function to get the final output, which is the prediction values:

$$a^{(2)} = f_2(z^{(2)}) \quad (2.23)$$

Where $\mathbf{x}$ as the input vector, $\mathbf{w}$ is the weight matrices, b as bias and f for activation functions for weighted sum z nodes, in a 2 hidden layer neural network representation, The symbols are represented for example w_{jk}^l , with superscript l representing the layer number, j and k represent the next node layer and previous node layer, respectively. We can refer to the Figure 2.2 for illustration.

2.5.2 Backward pass

Backward propagation, or backpropagation is the most fundamental building block in a neural network, as it is an algorithm to train a neural network using chain rule. The goal of backpropagation is to update each of the weights and biases in network. After each forward pass through a network, the backpropagation performs a backward pass, while adjusting the parameters such as weights and biases to minimize the cost function C . Usually, we define the loss function as a function defined on a datapoint or a single training example, and measure the penalty; while cost function can be a sum of loss function over the training set.[38]. These terms can be used interchangeably and not strict. The cost function can be mean squared error for regression models, cross entropy for classification models, or other cost functions. [39]

To compute the gradients, we will use the chain rule.[39] For calculating the gradient of the cost function with respect to weight w_{jk}^l with connection from the k^{th} node in the $(l-1)^{th}$ to the j^{th} node of layer l^{th}

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{jk}^l} \quad (2.24)$$

where the node is

$$z_j = \sum_{k=1}^m w_{jk}^l a_k^{l-1} + b_j^l \quad (2.25)$$

which m is the number of neurons in $(l-1)^{th}$ layer. By using this type of index, it agrees with typical (row, column) matrix indexing. Calculating the derivative of the node with respect to the weight, we can write

$$\frac{\partial z_j^l}{\partial w_{jk}^l} = a_k^{l-1} \quad (2.26)$$

So that the equation 2.24 can be written as

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} a_k^{l-1} \quad (2.27)$$

Using similar approach for the derivative of cost function with respect to j^{th} node's bias of layer l , b_j^l we can get

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l} \quad (2.28)$$

and the derivative of z is:

$$\frac{\partial z_j^l}{\partial b_j^l} = 1 \quad (2.29)$$

so that equation 2.28 can be written as

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} 1 \quad (2.30)$$

using the gradients, we can find optimal solution of the weights and bias until a termination condition:

$$w := w - \alpha \frac{\partial C}{\partial w} \quad (2.31)$$

$$b := b - \alpha \frac{\partial C}{\partial b} \quad (2.32)$$

where α is the learning rate that determines the influence of the gradient, that controls how much to change the model in response to the estimated error each time the weights are updated. It is the size of each step, and we need to choose the right learning rate. A large learning rate can cause the model to converge to quickly to a suboptimal option, while a learning that is too small can cause the process stuck in long convergence.[40]

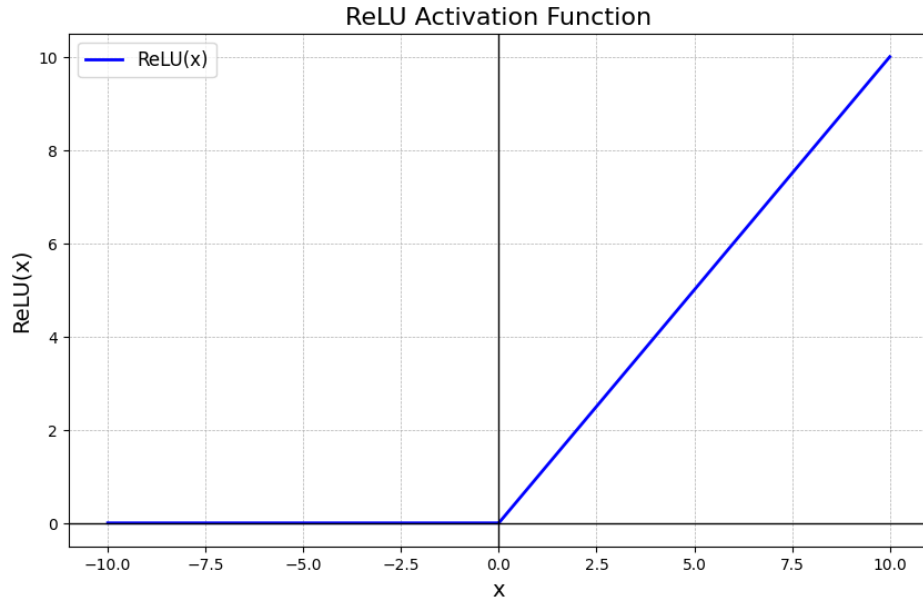


Figure 2.3: A ReLU function. Figure adapted from [43]

2.5.3 Activation function

Activation function is a mathematical function applied to the output of a neuron, that introduces non-linearity into the model. This will allow the network to learn complex patterns in the data. Without introducing non-linearity, the network will only behave like a linear regression model with polynomial degree one, no matter how many layers it has.[41] The most commonly used activation functions, are non-linear activation functions, because the real-world data often exhibits non-linear patterns.

There are numerous types of activation functions. Firstly, we have linear activation function, that resembles a straight line define by $f(x) = x$. It is also known as “no activation” or “identity function”, because it does not do anything to the weighted sum of the input, and simply spits out the given value. Linear activation function is only used as output layer.[41, 42]

Examples of the popular non-linear activation functions are sigmoid, tanh and ReLU activation functions. In this work, we only focus on ReLU activation function. The ReLU is a popular activation function, and it is the default choice in many architectures. In basic terms, the ReLU activation is a piecewise linear function, that outputs the input directly if it is positive, while outputs zero otherwise.[43]

Mathematically, the ReLU activation can be written as:

$$f(x) = \max(0, x) \quad (2.33)$$

where x is the input to the neuron. The function implies that it returns x when x is greater than 0, while returns 0 when x is less than or equal to 0. Therefore, when our input is negative value, it will returns zero as output. We can see the ReLU function as in Figure 2.3 .

ReLU is simple and maintains the non-linearity without complicated transformations, to allow models to learn more effectively. ReLU is a non-linear function, that allows model to learn more complex data patterns. To see how ReLU introduce non linearity, we can view a multiple combination of ReLU as a continuous piecewise linear function approximation (CPWL). This is a continuous function with multiple linear portions, that change slope at transition points, as shown in Figure 2.4 .

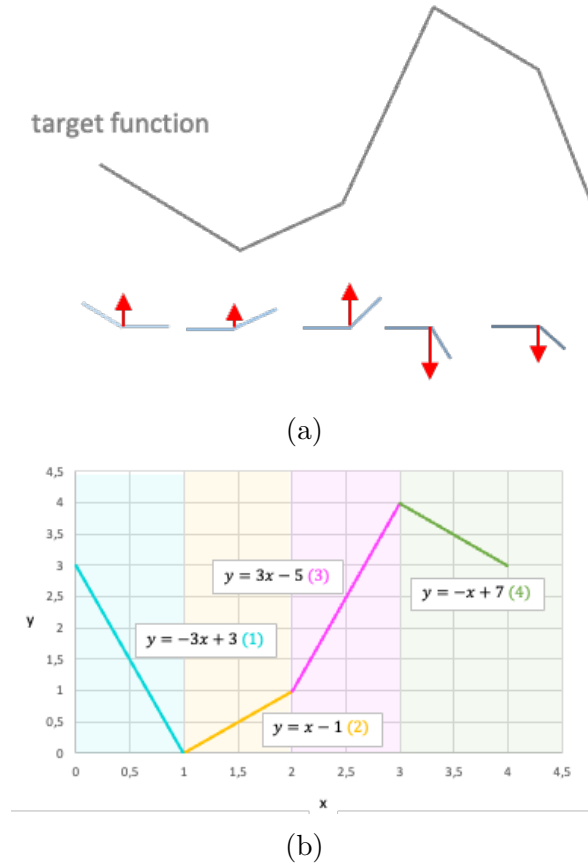


Figure 2.4: (a) A CPWL function. At each transition point, a new ReLU added to/subtracted from the input to increase / decrease the slope. (b) Example of piecewise linear function. Figure adapted from [44]

The target function in Figure 2.4 (b) can be represented by a neural network with 1 hidden layer of 4 units and a linear layer of output. The first segment, which is unit or node 0, we can choose the weight as -3 and bias as 3 so that the first segment is $z_0^{[1]} = -3x_0 + 3$, and the ReLU function disables the $z_0^{[1]}$ after $x_0 = 1$. Similar to the other segment, the ReLU function disables the output at a certain point. With more transition points, the smoother and model target function is better.

2.5.4 Optimizer

Optimizers are the algorithms that are responsible to update the weights of the neural network during the training process, and find the optimal set of weights that minimize

the loss function. The choice of the optimizers depends on the complexity of the neural network, and the type of the data used for training. Usually we try multiple optimizers and select the best optimizers that gives performance.[45] In this study, we use SGD and Adam optimizer to perform the training, because they are the two most common types of optimizers.

Stochastic Gradient Descent (SGD) is, a variant of gradient descent, that updates weights using a single data point or small batch, making it faster and more scalable. It uses a constant learning rate that applies to all parameter updates. This will cause slow convergence.[46] Gradient descent is an algorithm that trains machine learning models by minimizing errors between predicted and actual results, by making small adjustments in the right direction using derivative (slope) until local or global minimum.[47]

Adam is the short for Adaptive Moment Estimation, is an adaptive learning rate algorithm. It adjust the learning rate for each individual parameter within a model, instead of using a single global learning rate. Therefore, it navigates the loss landscape more efficient, and converges much faster.[48] Adam has faster training times than SGD. In this result, we can also say that Adam converges effectively within the same number of epochs. Interested readers can refer to several articles that provide mathematical background for SGD and Adam optimizers.[49, 50]

2.5.5 Loss Function

A loss function is a mathematical function that compares the target and predicted output to measure how well the model trains the data. During the training, we want to minimize the loss between the true and predicted values.[51] The smaller the number, the better the model is performing.

A loss function and performance metric serve different purposes. A loss function is used during training, to optimize the model's parameters. Meanwhile, a performance metric is used to evaluate the model after training, to see how well the model generalize to new data.[52]

There are many types of loss functions that are suitable for different tasks. We can divide it as classification loss and regression loss function. Popular regression loss functions include mean square error (MSE) loss , mean absolute error (MAE) loss.[53] In this work, we use mean squared error in the deep learning algorithm. Mean square error is a common loss function for regression tasks. It evaluate the average squared difference between predicted and actual values,

$$L(y, \tilde{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \tilde{y}_i)^2 \quad (2.34)$$

where N is the number of samples, y and $\tilde{y}$ are true values and predicted values respectively.[54]

2.6 Python and Machine learning frameworks

There are several popular production-ready tools in Python programming that we can use to perform machine learning.[35] Scikit-Learn is open-source library that is easy to use and beginner friendly, and implements many machine learning algorithms efficiently. The library is built upon NumPy, Scipy and Matplotlib. Scikit-learn provides some of the most popular models such as linear regression, decision tree, clustering, ensemble methods, feature selection, etc.

TensorFlow is a more complex library for distributed numerical computation. It can train and run large neural networks efficiently by distributing its computations across multiple GPU servers. It support wide range of application, such as including natural language processing(NLP), computer vision(CV), time series forecasting and reinforcement learning.

We also have Keras, which is a high level Deep Learning application programming interface (API) that can train and run neural network with ease. An API defines how the different software components can communicate and exchange data or services. Keras is a wrapper for TensorFlow, for rapid experimentation. TensorFlow have its own implementation of this API called *tf.keras*, to provide support for more advanced TensorFlow features, such as loading data efficiently. Keras have full access to the scalability and cross-platform capabilities of TensorFlow. It offers simple interfaces and minimize the actions required for user.

PyTorch is another open-source deep learning framework similar to TensorFlow. The programming is pythonic and easy, and has string ecosystem and community. PyTorch is ideal for research and small projects, while TensorFlow is ideal for large scale projects that require high-performance and scalable models.

JAX is a Python library alternative to NumPy, for optimized scientific computing. It is a library for array-oriented numerical computation, with automatic differentiation and JIT (just-in-time) compilation to enable high-performance machine learning research. It also support efficient evaluation of gradient by using its automatic differentiation transformations. The advantage of JAX is it can be extremely fast, compared to libraries mentioned above, and use more efficient use of GPU.

In this project, we will use scikit-learn library and TensorFlow with Keras API to perform machine learning.

Chapter 3

Methodology

In this chapter, we will introduce the methods we have developed in this thesis. First, we will introduce the process to obtain the data, and the scaling, which is the change of value range on the data. We will introduce the methods used for the scaling. Then we will also introduce the performance metrics to analyse the quality of the machine learning predictions. We will also discover the machine learning libraries and python packages used in this thesis, and how do we implement machine learning methods on the real-time data matrices we have obtained from the propagation. The overall work flow is presented in this section.

3.1 Data generation and scaling of the data

To develop machine learning model, the first step is to acquire the data for training and testing. The data is generated from the real-time Hartree-Fock calculation using PySCF code, and the code can be found in the GitHub repo. https://github.com/BERTHA-4c-DKS/ML_Realtime We have run through several electrodynamic simulation calculations to obtain different datasets for 3 selected closed-shell molecules H_2 , LiH , and H_2O . The molecules used for the development are small molecules with simple basis. The first basis used is STO-3G with propagation of 10000 time steps (au) and 20000 time steps (au) in the electrodynamic simulation with a time step of 0.1 au, with a delta function electric field perturbation of maximum amplitude 0.02 au in z-direction molecular axis. By running the propagation using PySCF, the density matrix and Fock matrix for each time step are extracted and stored in CSV files. These molecules are aligned along the z-axis, with the coordinates are shown in the appendix. We have used STO-3G and 6-31G basis set with the same procedure. All input files for the data generation and the CSV files can be also be found on in the GitHub repo. As an example of the simulation, we can see in the Figure 3.1 and Figure 3.2 that we have obtained the induced dipole moment and the absorption spectra for the real-time simulation of the water molecule propagation with STO-3G basis set for 10000 steps.

Since the density and Fock matrix are Hermitian, we only extract out the upper triangle of the matrices, and flatten them into diagonal, real and imaginary part of the matrix in an array respectively for the value of each time steps. The values are

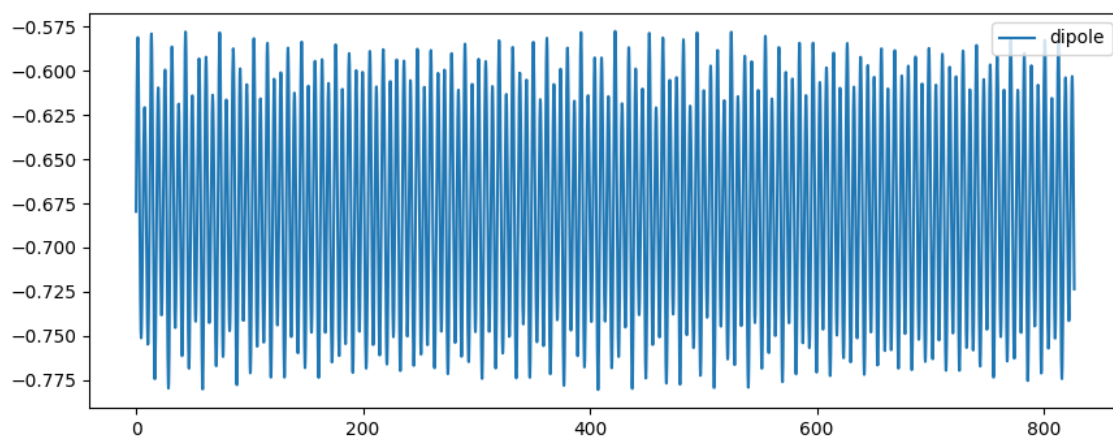


Figure 3.1: The dipole moment of H₂O of the real-time simulation.

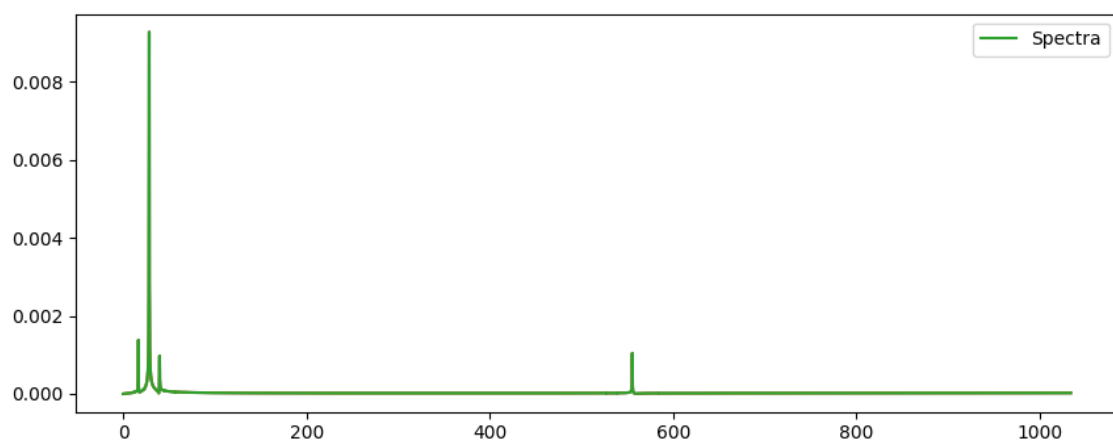


Figure 3.2: The absorption spectra of H₂O of the real-time simulation.

arranged in time evolution sequence using numpy array from t_0 to t_n ,

For linear regression and neural network, the density matrices are served as independent features and the Fock matrices are the dependent features. The density matrices have same size with the Fock matrices. If the Fock matrix has size $M \times M$ for basis size M , there are $M(M+1)/2$ elements of real values (including diagonal values in this thesis), plus $M(M-1)/2$ of imaginary values.

Next, we recall the method of linear regression and refer to the work done by Bhat [1]. For linear model, we formulate the relationship where the Fock matrix or Hamiltonian h is dependent on p density matrix, $\mathbf{h}(\mathbf{p})$:

$$\mathbf{h} = \beta_0 + \beta_1 \mathbf{p} \quad (3.1)$$

where the parameters β_0 has size $M^2 \times 1$, and β_1 has size of $M^2 \times M^2$.

The machine learning framework will find the optimal parameters (β_0, β_1) for the model prediction. The input data is split into training set and testing set, with adjustable size. We set the size of training/testing ratio from 90/10 % to 10/90% to see the effect for different training size.

The datasets used for the linear regression are generated with propagation of 10000 time steps and 20000 time steps. Then the parameters such as types of molecules, train / test percentage, include or exclude delta kick function value can be tuned to observe the trends and improvements in prediction. The observed performance metrics are R^2 , number of Fock matrix elements that have R^2 less than 0.99 with respect to number of features, root mean square error (RMSE) and mean absolute percentage error (MAPE). The performance metrics will be explained in the coming sections. We will adjust several parameters to see their effect on the predictions.

Before passing on to describe our implementation in the next section, we present a brief description to analyse the methods to scale the data, and the typical measures to analyse the performance of an optimized model. Next, we will try to see the effects on different size of training and testing on the dataset. There is no fixed rule to determine the amount of data of train test splitting. Therefore, we will look at the trend where is the optimal splitting of training and testing set that gives us the optimal result over train/test ratio. Also, we are interested to see the effect of removing values that experienced a delta kick in the beginning of the simulation. Therefore, we removed first 500 data points to compare its effect. We will also look at the effects of using larger dataset, to see whether the accuracy will increase by including we expand the basis set for simulation in future.

3.2 Scaling of data

Scaling of data is important because, this will enhance the machine learning model performance. Scaling is necessary because algorithms that use gradient descent such as linear regression, logistic regression, and neural networks, are sensitive to difference in feature scales.[55] First, we try to use different scaling methods on the dataset. This includes no scaling on the dataset, using min-max scaling, and also standard scaling.

The min-max scaling is the scaling of data usually into the range of $[0, 1]$, which is also called normalization. This scaling method is suitable for the data that does not follow a normal distribution, or when we need to scale the data for algorithms like decision trees, k-nearest neighbour. It is sensitive to extreme values, so it will give best result when the dataset do not have outliers.

The scaling is done to a fixed range by subtracting the minimum values, and divide by the range between maximum and minimum value, to ensure all feature values lies within specified range.

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (3.2)$$

where X is the original value, X_{min} is the smallest value of each feature, and X_{max} is the maximum value of the feature. In this work, each columns will be scaled independently into 0 to 1. The largest vale in the column will become 1, while the smallest value in the column will become 0.

The standard scaling is another feature scaling technique, which follows the standard normal distribution, and we call this standardization. It transforms the data, so that the mean of the data becomes 0, and the standard deviation becomes 1. It is suitable to use for machine learning algorithms like support vector machine (SVM), logistic regression, and neural networks, which assume that the data are normally distributed. It is represented by

$$X_{scaled} = \frac{X - \mu}{\sigma} \quad (3.3)$$

where X is the original value, μ is the mean of the feature, and σ is the standard deviation, where the standard deviation is

$$\sigma = \sqrt{\frac{\sum(X - \mu)^2}{N}} \quad (3.4)$$

for N is the number of samples.

3.3 Measurements of performance metrics

To quantify how good the machine learning predictions are, we need some methods to measure their performance. Here we introduce some of the most common metrics that will be used in this project.

3.3.1 R^2 value

The R-squared metric or coefficient of determination (R^2) is used to measure how well a model fits a data. It is a statistical measure that show us how much of the variation of the dependent variable (Fock matrix) is explained by an independent variable (density matrix) in a regression model. This let us know how well the variation in the data can be explained by the model. The closer the R^2 value is to one, the better the model

fits the data. This will evaluate the quality of a regression model. The value of R^2 lies between 0 and 1. If the R^2 value is 0.80, it means that 80% of the model variance explains the variability of the data points.

We can know the amount of variation to be explained by the regression by using total sum of squares (TSS) where y_i is the observation and $\bar{y}$ is the sample mean:

$$\sum_{i=1}^n (y_i - \bar{y})^2 \quad (3.5)$$

Rewriting $y_i - \bar{y}$ as

$$y_i - \bar{y} = (\tilde{y}_i - \bar{y}) + (y_i - \tilde{y}_i) \quad (3.6)$$

and we can rewrite the total variation as:

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (\tilde{y}_i - \bar{y})^2 + \sum_{i=1}^n \epsilon_i^2 \quad (3.7)$$

The explained sum of squares (ESS) represents the explained variation, and it measures the total variation of $\tilde{y}_i$ from $\bar{y}$. The residual sum of squares (RSS) represents the unexplained variation.

The sum of squares due to regression, measure how well the regression model represents the data used for modelling. Meanwhile, the total sum of squares measures the variation in the observed data.

Therefore, we can represent the ratio as the coefficient of determination (R^2):

$$\frac{ESS}{TSS} = 1 - \frac{RSS}{TSS} = \frac{\sum_{i=1}^n \epsilon_i^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3.8)$$

and the R^2 value lies between 0 and 1. It measure the percentage of variation of y around $\bar{y}$, that is explained by the regression equation. The closer the observed points are to the estimated regression line, the better the fit, the higher the R^2 value. The value of R^2 will stay the same, no matter with the scale of our data.

3.3.2 Mean squared error and root mean squared error

The mean squared error (MSE) is calculated as follow:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2 \quad (3.9)$$

where n is the number of data points, y_i is the observed values and $\tilde{y}_i$ is the predicted value. A lower MSE means that the model's predictions are closer to the actual values, showing better accuracy. MSE is scale dependent, the different scale of the data will affect the MSE values.

Root mean squared error (RMSE) is the square root of MSE. It provides a unit of same scale as the data.

3.3.3 Mean Absolute Percentage Error (MAPE)

MAPE is represented as:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \tilde{y}_i}{y_i} \right| \times 100 \quad (3.10)$$

where n is the number of data points, y_i is the actual value of the i^{th} data point, and $\tilde{y}_i$ is the predicted value of the i^{th} data point. The lower the values of MAPE, the higher the accuracy. The numerical value implies the predictions compared to the actual values, which means that MAPE of 5% indicates that the predictions are 5% off from the actual values on average. It is also scale dependent.

Below we will introduce two machine learning methods which are the linear regression and neural network that have been used in this thesis.

3.4 Multiple Linear Regression

For multiple linear regression, the generalized formula is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n \quad (3.11)$$

where the x_i is an independent variable, and the β_i is the coefficient. The function fits a linear model with coefficients $\beta = (\beta_1, \dots, \beta_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

This function performs ordinary least squares Linear Regression. Mathematically, this function solves a problem of the form

$$\min_{\beta} \|X\beta - y\|_2^2 \quad (3.12)$$

The ordinary least squares linear regression is carried out using the scikit-learn [56] Linear Regression model, `LinearRegression()`. This inbuilt function with name `LinearRegression()` will take in its `fit` method arrays X , y and store the coefficients β of the linear model, in its `coef_` member. If we use `model.coef_`, we can print the coefficients of our linear regression. These coefficients define the model for predictions.

3.5 Neural Network

To investigate whether the deep learning method is a suitable method for learning the propagation, we also use the neural network method to perform machine learning on the density matrices to learn the Fock matrices. We performed grid search to find the best hyperparameter to tune the network. Then, the best hyperparameters are used to perform the machine learning on the dataset.

In this project, we use Keras [57], which is a deep learning API written in Python, that run on top of the machine learning platform TensorFlow. Keras enable us to have

full access to the scalability and cross-platform capabilities on TensorFlow. Keras was developed with focus on enabling fast experimentation, it is easy to get started and learn advanced workflows with practice.

To perform neural networks, we have to adjust some hyperparameters to configure the neural network architecture. The hyperparameters used to tune the neural network are number of hidden layers, number of neurons and types of optimizers. We assume that we do not know the best parameters when manually changing these parameters, thus the first guess by intuition is applied to the number of hidden layer, number of neurons, etc. Since there are many parameters that we can tune in a neural network, we only focus on these few most common parameters. To run through each combination, we can use grid search to find the best combination of parameters.

3.6 Implementation

Now we describe how to use the program. The best interface to execute the program is Jupyter notebook. The Jupyter Notebook is an open-source web platform, that allows developers create narrative text, live code, visualizations and equations. We use Jupyter notebook because it is suitable for data visualization, data cleaning and transformation as well as machine learning. It is easy to prototype the program and share the analysis, which is better than the Python script. The Python code snippets examples used in the Jupyter notebook will be presented in each paragraph that describes the implementation of the code.

We use Python programming language because it is simple syntax, can be easily read and reduce time to code. This lets us focus on the task to solve rather than writing complex code that can reduce errors. Python is ideal for machine learning development, because it has vast library ecosystem. Libraries like TensorFlow, Keras, PyTorch and scikit-learn are designed to do machine learning tasks, so that we do not need to write machine learning algorithms from scratch. Those libraries also has strong and active community of developers, we can find useful tutorials and documentations to develop our work. There are several popular packages to perform computational calculation in Python. PySCF is a free and open source quantum chemistry code. In this work, the propagation of the electrodynamics will be performed in PySCF code. During the propagation, the density and Fock matrices will be extracted to use in the machine learning.

The overall process for the part of machine learning implementation is presented in the following pseudocode Algorithm 1 below. The whole machine learning process involved the following important functions described below, and how we integrate it in our code. The list of libraries that enable us to use some of those functions are presented in Table 3.1. To facilitate the future development and revision, their version and installation method used in this thesis are also included. The code is available on the same Github repo. Next, we will look at the details of the implementation.

In our implementation scheme, we assume that the dataset is available and ready to perform machine learning. The data has been generated using a real-time simulation using a local version of RT-TDHF method based on PySCF. The code is available

Algorithm 1 The machine learning implementation

```

if Neural Network is true then
  if Grid search is true then
    Optimize by grid search
  else
    print(Not perform grid search)
  end if
  Perform Neural Network
else
  Perform Linear Regression
end if

```

Library	Version	Installation
NumPy	1.26.4	pip install numpy
TensorFlow	2.19.0	pip install tensorflow
Scikit-learn	1.4.2	pip install scikit-learn
Matplotlib	3.8.4	pip install matplotlib
scikeras	0.13.0	pip install scikeras

Table 3.1: The libraries implemented in the code for machine learning.


```

data_myfock = np.loadtxt("myfocklist-Copy1(H2)-sto3g-10000.
                        csv", delimiter = ',')

elif molecule == 'LiH-sto3g-10000':
    print("Current dataset: LiH-sto3g-10000")
    data_density = np.loadtxt("densitylist-Copy1(LiH)-sto3g-10000
                             .csv", delimiter = ',')
    data_myfock = np.loadtxt("myfocklist-Copy1(LiH)-sto3g-10000.
                             csv", delimiter = ',')

# ... add other molecules

else:
    print('No dataset selected!')
    sys.exit()
return data_density, data_myfock

```

We have added a feature to check the entries that are identical to zero. We calculate the mean of the absolute value each density matrix elements throughout the time step, and if the value is less than 1×10^{-13} , we will remove the element of density and Fock matrices along the time series from the machine learning algorithm. We need to calculate the absolute value because the data contain a mix of positive and negative values. Since the Python uses 64 bit floats which have 15 to 17 decimal digits of precision. [58] We can assume that those values near this limit are, in fact, zeros.

```

if check_noise == True:
    mean_row_density = np.mean(np.abs(data_density), axis=0)
    print("length mean row:", len(mean_row_density))

    mean_row_fock = np.mean(np.abs(data_myfock), axis=0)
    print("length mean row:", len(mean_row_fock))

if filter_noise == True:
    mean_row_density = np.mean(np.abs(data_density), axis=0)
    mean_row_fock = np.mean(np.abs(data_myfock), axis=0)

    index_location_density = np.where( mean_row_density < threshold)
    index_location_fock = np.where( mean_row_fock < threshold)

    data_density = np.delete(data_density, index_location_density, 1)
    data_myfock = np.delete(data_myfock, index_location_fock, 1)

```

By using the `MinMaxScaler()` and `StandardScaler()` provided by scikit-learn machine learning library, we can scale the data before perform machine learning. Then, we can use `inverse.transform()` function to transform back our original data representation. We need to make sure our performance metrics measurements are calculated on actual values. It is not necessary to scale the target value which are the Fock matrix, the results will be the same. However, in this thesis we scaled all the values together. In regression tasks, sometimes scaling the target variable helps but it depends on the model. For example, neural network is sensitive to output scale, and unscaled large target values can cause vanishing or exploding gradients.[59]

```

if scaler_type == 'MinMax':
    scalar = MinMaxScaler()
    print('Using MimMax Scaler')
    data_density = scalar.fit_transform(data_density)
    A = data_density
    B = scalar.fit_transform(data_myfock)

elif scaler_type == 'Std':
    scalar = StandardScaler()
    print('Using Std Scaler')
    data_density = scalar.fit_transform(data_density)
    A = data_density
    B = scalar.fit_transform(data_myfock)

else:
    print('No scaler')
    A = data_density
    B = data_myfock

```

The first steps of the simulation are expected to be influenced by the initial perturbation (delta-kick). We try to exclude some values from the beginning to see if there are delta kick effects applied on the training and predictions. In the code, it is implemented to possibly remove these values with the defined amount. First, the user input the number of values to remove, and the code will select all the values except the n values to exempt. For example, if we select to remove first 10 values, then the data obtained by the first 10 time steps of the propagation of density matrices and Fock matrices will be removed from training and testing.

```

print("delta kick sample is excluded, take sample exclude first n
      values")
A = A[kick_size : sample_size ]
# print("A shape removed kick: ", A.shape)
B = B[kick_size : sample_size ]
# print("B shape removed kick: ", B.shape)
test_size = sample_size - training_size - kick_size # test size

```

The linear regression method can be carried out by using the function from scikit-learn `LinearRegression()` to create a linear regression model. Inside this function, we set `n_jobs = -1` so that we can use all computer processors for the computation. We use `model.fit()` to train the machine learning model, which the inputs of this function are the density matrix and Fock matrix. We then use the function `model.predict()` to make prediction using the parameters obtained from the training. We can use test set of density matrix and apply the obtained parameters to make Fock matrix prediction. The training and prediction process is loop through each individual Fock matrix elements, the number of loops depending the number of Fock matrix elements. We include all the density matrix elements to train the parameters, because we do not know which independent variables or density matrix elements that contribute to the fitting of Fock matrix element. Including more parameters has no effect the accuracy, if it does not related to the dependent variables. The case is same in the neural network implementation in the following, but the neural network takes in all input and output

together instead of iteration through each individual Fock matrices.

```

for i in range(B.shape[1]): # B.shape[1]: number of features
    b = B[:,i] # b = B[1], 2, 3, ... b = true value, to compare with
                # pred value
    b_train = b[0:training_size] # take all training value until
                                # training size
    A_train = A[0:training_size]

    # use second half test the model
    b_test = b[training_size: sample_size] # take testing value
    A_test = A[training_size: sample_size]

    if scikit_LR_option == True:
        # print("Solving using Scikit-learn LR regression")
        # Solve using scikit-learn library
        # creating a regression model
        model = LinearRegression(n_jobs = -1, ) # use all processors
        # fitting the model
        model.fit(A_train, b_train)

        # making predictions
        true_data_train = b_train
        pred_data_train = model.predict(A_train) # make prediction
                                                # on train set

        true_data_test = b_test
        pred_data_test = model.predict(A_test) # make prediction on
                                                # test set

```

To build the neural network, we use sequential model `Sequential()` from TensorFlow to add neural network layers. A sequential model allow us to create models layer-by-layer, stacking together where each layer have one input tensor and one output tensor, which means we only include homogeneous input and output. Multiple inputs and outputs means that we have inputs of images and texts, or outputs of regression and probability. By using sequential model, we can simply add more layers by using `add()`. We add dense layer with `Dense()`, which are fully connected network that connect every input feature to every neuron in the layer. We can add as many layer as we want. Some hidden layers are commented out, because if we want to test with different hidden layers manually, we can uncomment it. Same as the procedure with different number of neurons, we can uncomment it manually in the code. The benefit of using Jupyter notebook is we can make changes and edit the code, to see the effect and output immediately. This method needs to manually change the code, in exchange of we do not need to build a very complex program to investigate the change of number of layers in the code. The learning rate α , which controls the step size of optimization, are set to a constant 0.01 for different optimizers because we want to observe the effect of same learning rate on different optimizers. The number of neuron is set to 256, because we assume that more number of neurons lead to better result, as small number of neurons might lack sufficient capacity to model the data. It is a historical reason to use 2^n number of neurons, as the physical processor is usually power of 2, and it does

not have much effect between even or odd numbers of batch size number of neurons with highly optimized neural network architecture nowadays.

```

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.InputLayer(shape=(A_train.shape[1],)))

# set default as 0 hidden layer      # hidden layer 0
# model.add(tf.keras.layers.Dense(256, activation='relu')) # hidden
#                                     layer 1

model.add(tf.keras.layers.Dense(A_train.shape[1], activation='linear',
                                ))

model.compile(loss='mean_squared_error',
              optimizer = optimizer_select,
              metrics=['mse'])
A_train_np = np.array(A_train)

history = model.fit(A_train_np, b_train,
                    epochs = num_epoch,
                    batch_size = num_batch_size,
                    verbose = 1,
                    validation_split=0.1)

```

To use the grid search, we use `GridSearchCV()` from scikit-learn library. Inside the grid search, we create regression model using `KerasRegressor()`, and loop through different hyperparameters to find the best combination. The grid search evaluate the best combinations using `best_score_` attribute, which provides the best cross-validation score. In the regression case, it returns the R^2 score across mean of the number of folds. We also use `best_params_` to print out the best combinations of hyperparameters that give us the best result, and use in the neural network. We conduct exhaustive search for all hyperparameters, and is time consuming. For example, the combinations of hyperparameters we have defined can formed from three different number of layers, two different optimizers and four different number of neurons. This will produce a total of 24 different combinations, and with cross validation value of 3, it brings the total number of jobs to 72. Each value added to the parameter grid dictionary, can significantly increase the total runtime of the function. To have a intuition of the time for implementation, we briefly implement different techniques with 50% train-test ratio of H_2O with the code used in this thesis in one same computer. The linear regression training and prediction takes about 2 seconds, the neural network takes about 27 seconds, and the grid search with neural network takes about 9.6 minutes. We have defined `cv=3` in the grid search, means that it is a 3-fold cross validation for each combination of parameters. Cross validation works by splitting the datasets into random groups, and hold one group as test, and train model on remaining groups. The dataset will be split into k -equal parts or called folds. Hence we have three iterations of the model, each with different fold as test set and train the other two fold, and the resulting iterations are averaged together to create a final cross validation model.[60]

```

grid = GridSearchCV(estimator=model,

```

```

        param_grid=param_grid,
        n_jobs= -1,
        cv=3,
        verbose = 1)
grid_result = grid.fit(A_train, b_train)
# summarize the results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.
                             best_params_))

```

Then, after the predicted values are transformed back to original scale, the performance of the machine-learned data can be evaluated by using the functions provided by the scikit-learn library. The coefficient of determination, R^2 can be calculated using `r2_score()`. The other metric is the function `mean_squared_error()`, which calculate the mean square error (MSE). The other one, which is the function `mean_absolute_percentage_error()`, gives us the value of mean absolute percentage error (MAPE). Those values are average results metrics for each of the Fock matrix elements. Alternatively, scikit-learn also provide us the function to view each score of the matrix elements, which are called the raw values of the metrics if we intended to. For example, we use this function to find out which matrix elements across time steps have prediction of R^2 value that are less than 0.99, then we can identify whether those matrix elements are small values. Those predefined functions made by the scikit-learn developers are easy to use as we only need to provide inputs of true data and predicted data.

```

def performance_metrics(true_result, pred_result):
    r2_uni_avg = r2_score(true_result, pred_result, multioutput='
                        uniform_average')
    mse = mean_squared_error(true_result, pred_result)
    rmse = np.sqrt(mse)
    mape = mean_absolute_percentage_error(true_result, pred_result,
                                          multioutput='uniform_average')

    print(f"R-squared (uniform average) : {r2_uni_avg:.4f}")
    print(f"Mean squared error (avg): {mse:.4e}")
    print(f"Root mean squared error: {rmse:.4e}")
    print(f"Mean absolute percentage error MAPE (avg):{mape * 100:.4e
        } %")

```

Finally, if we want to visualize the plots of the true data and prediction data across time steps, we can use `plot()` function provided by the Matplotlib library. This is usually done by importing the library using the command `import matplotlib.pyplot as plt` in Python. Matplotlib is one of the most popular plotting library for Python in scientific research, and a tool for data visualization. It can let us create line plots, bar plots, scatter plots and more. It is essential for analyzing data. Matplotlib is designed to work with NumPy arrays, so it is suitable to use it in our thesis. We can see how the predicted value fits well with the true value if they overlap with each other on the plot. We also use the plots to visualize the if there are abnormal behaviour of the propagation, such as the propagation and prediction of identically zero elements. However, for more accurate observation, we still rely on the numerical values of the performance metrics. We have created different plot functions that allow user to choose to observe

the propagation of Fock matrix. The functions available are `plot_train_data` that allow user to view the train data and its prediction, `plot_test_data` that can plot test data and its prediction. The user can define the amount of data to plot within the available range of the time step for train and test data. Besides, we have defined a function called `plot_partial_data`, that allow us to plot the first $x\%$ and last $x\%$ of Fock matrix across time step. We defined this function to observe if the prediction accuracy drift away across time steps, means that to see if the prediction is accurate with true value in the beginning but shift away from true value for long prediction.

```
def plot_partial_data(percentage, true_data, pred_data, iteration,
                    title_name):
    plt.rcParams['figure.figsize'] = [11, 4]
    plt.rcParams.update({'font.size': 8})

    front_part = true_data.shape[0] * percentage // 100
    back_part = - true_data.shape[0] * percentage // 100 # count from
                                                         back

    # front plot
    fig = plt.figure()
    ax1 = fig.add_subplot(121) # Three integers (nrows, ncols,
                                index).
    plt.plot(true_data[: front_part], '-x', color='k', linewidth=1,
              ms=4, label='True value') #
                                         True relationship
    plt.plot(pred_data[: front_part], 'o', color='r', linewidth=1, ms
              =3, label='Prediction')

    plt.xlabel('time step')
    plt.ylabel('value')
    plt.title(f'first {percentage} % ' + title_name[iteration])
    # plt.title(f'Train {iteration}')
    plt.legend()

    # back plot
    ax2 = fig.add_subplot(122)
    plt.plot(true_data[back_part :], '-x', color='k', linewidth=1, ms
              =4, label='True value') # True
                                         relationship
    plt.plot(pred_data[back_part :], 'o', color='r', linewidth=1, ms=
              3, label='Prediction')

    plt.xlabel('time step')
    plt.ylabel('value')
    plt.title(f'last {percentage} % ' + title_name[iteration])
    # plt.title(f'Train {iteration}')
    plt.legend()
```

We have combined different machine learning methods in one code, to simplify the implementation and make sure we use the same data processing techniques. The users can choose different methods to do machine learning. The input parameters needed to adjust are shown in Appendix A, and the overall procedure described above is shown

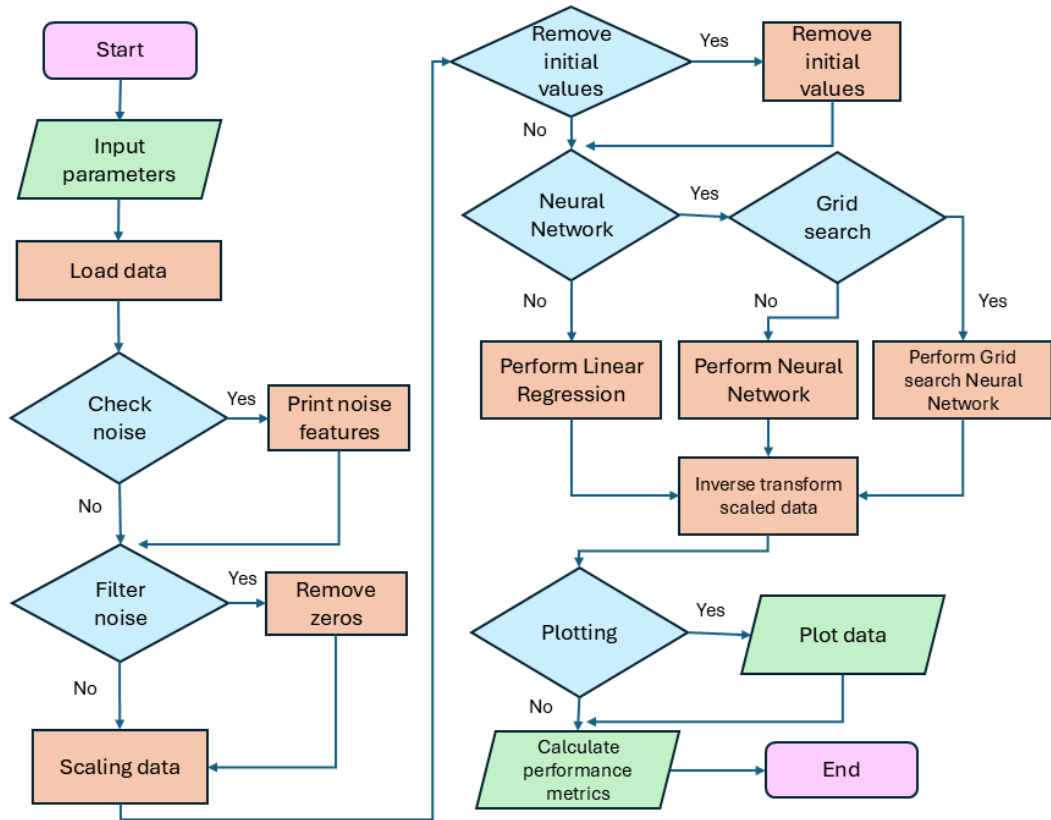


Figure 3.3: The flow chart of the machine learning program

in the Figure 3.3 below. In the figure, if we select the parameters to be “True” before running the program, it means “Yes” in the figure and the function will be executed and proceed to next block of code, else if “False” or “No” is chosen, the function will be skipped and proceed to the next section.

```

scikit_LR = True    # solve using scikit-learn LR method
# scikit_LR = False

# Neural_Network = True    # use NN method
Neural_Network = False

# grid_search = True    # use grid search method
grid_search = False

```

All the density and Fock matrices extraction process, training data and code implemented in this thesis can be found on GitHub: https://github.com/BERTHA-4c-DKS/ML_Realtime

An example of the output is shown below, the result shown is the linear regression for H_2 molecule with STO-3G dataset and 10000 time steps. After we have obtained our output results, we will copy the values to make plots and tables for investigation. The process is same for different datasets.

```

column_name is H2:
+++++ START +++++

kick values are included.(full data)
train / test percentage:  50 % /  50 %
sample_size is: 10000
training_size is: 5000
test_size is: 5000
Current dataset: H2-sto3g-10000
Using Std Scaler

delta kick sample is included, take sample from first
(whole dataset)
  B_result_train_true shape (5000, 4)
  B_result_train_pred shape (5000, 4)
  B_result_test_true shape (5000, 4)
  B_result_test_pred shape (5000, 4)
Plotting training data is off
Plotting testing data is off
B.shape[1] is number of dimension: 4

.....Solving using Scikit-learn LR regression.....
Time train    0.015 s

Transform data back from Std scale

performance metric for TEST set (original scale data):
R-squared (uniform average)  : 1.0000
R-squared (raw values)      :
Number of R2 < 0.99:  0 / 4

Mean squared error (avg): 2.4945e-30
Mean squared error (raw):
Root mean squared error: 1.5794e-15
Mean absolute error (avg): 1.0280e-15
Mean absolute percentage error MAPE (avg):1.9418e-12 %
Mean absolute percentage error MAPE (raw):

Not plotting front-end graph
Not calculating front end metrics
+++++ END +++++

```

Chapter 4

Results and Discussion

In this section we will present the results obtained from the linear regression and neural network methods. We will investigate the effect of changing training size, data size, presence of noise in the data, the orientation of the molecule, and the effect of using different combinations of hyperparameters of neural network. The results will be presented in tables and plots. The shape of the density and Fock matrices used in this project is presented in the Table 4.1. Those data are generated from the real-time propagation of density matrix, and they have the same size. The default data sets used are molecules in STO-3G basis set with 10000 time steps, unless stated for investigation.

4.1 Linear regression

First, we will investigate the linear regression machine learning method. The results are described and tabulated in the following sections. We will look at the results of the different scaling methods. Then, we are interested to know the effect of different size of training and testing data. We then proceed to investigate the effect of including the data that experienced delta kick function. Furthermore, we concerned about the size of the dataset and the size of basis set used. We will also look at the effect of removing small values that present in data. We are also interested to know whether the orientation of linear molecules to the electric field affect our result.

4.1.1 Effect of different scaling method

We have to adjust between the parameters `MinMax`, `Std` and `None` in the parameter `scaler_type` before running the code to change the type of scaler. In this section, we used first half of the full dataset for training and last half for testing. By doing this we use `training_percentage` and fix the training percentage in our code. The total number of data is 10000 time steps. The effect of using different scaler to transform the values of data is shown in Table 4.2 , 4.3, and 4.4. In our data, we have values of different range, with positive and negative values. We can see that using the standard scaler leads to slightly better results. The results of LiH with standard scale has

Data (Density, Fock)	Basis set	Array size (row, columns)
H ₂ (10000 time steps)	STO-3G	(10000, 4)
LiH (10000 time steps)	STO-3G	(10000, 36)
H ₂ O (10000 time steps)	STO-3G	(10000, 49)
H ₂ (20000 time steps)	STO-3G	(20000, 4)
LiH (20000 time steps)	STO-3G	(20000, 36)
H ₂ O (20000 time steps)	STO-3G	(20000, 49)
LiH (10000 time steps)	6-31G	(10000, 121)
H ₂ O (10000 time steps)	6-31G	(10000, 169)

Table 4.1: The size of the dataset used in this thesis.

H ₂	No scale	Std	MinMax
R^2 avg	1.0000	1.0000	1.0000
Number of $R^2 < 0.99$	0 / 4	0 / 4	0 / 4
MSE	2.5033e-30	2.4945e-30	2.4970e-30
RMSE	1.5822e-15	1.5794e-15	1.5802e-15
MAPE (%)	1.9618e-12	1.9418e-12	1.9872e-12

Table 4.2: H₂ data scaling effect

better R^2 value, but we should aware that the cause of lower R^2 is the presence of very small values. Although the current results do not show significant effect with or without scaling, we should also include scaling to our future dataset, because we might not know the distribution of the data. When the molecule becomes larger such as H₂O, there are more data in different ranges, so that the data scaling becomes more important.

It is beneficial to test different types of scales and see which affects the performance of the model. It is known that many machine algorithms perform better when scaled to standard range. Therefore, we will use standard scaler as a default in this the following machine learning study, since many deep algorithms assume normality with a mean of 0 and standard deviation of 1. The results shown below are using the standard scaler.

4.1.2 Effect of train/test ratio

In this section, we have to uncomment the parameter `training_percentage2` and comment out the `training_percentage` parameter. From Table 4.5 , 4.6, we can see

LiH	No scale	Std	MinMax
R^2 avg	0.9137	1.0000	0.9138
Number of $R^2 < 0.99$	2 / 36	0 / 36	2 / 36
MSE	4.6995e-31	1.8323e-31	1.8011e-31
RMSE	6.8553e-16	4.2805e-16	4.2439e-16
MAPE (%)	0.9010	3.1807e-11	3.3518e-11

Table 4.3: LiH data scaling effect

H ₂ O	No scale	Std	MinMax
R^2 avg	0.9870	0.9871	0.9871
Number of $R^2 < 0.99$	11 / 49	11 / 49	11 / 49
MSE	6.1795e-30	4.9697e-30	4.9966e-30
RMSE	2.4859e-15	2.2293e-15	2.2353e-15
MAPE (%)	11.7397	11.7226	11.7217

Table 4.4: H₂O data scaling effect

Train / test (%)	R^2 (avg)	Number of $R^2 < 0.99$	RMSE	MAPE (%)
10 / 90	1.0000	0	1.4110e-15	5.6276e-12
20 / 80	1.0000	0	1.4738e-15	7.5507e-12
30 / 70	1.0000	0	1.5091e-15	3.2098e-12
40 / 60	1.0000	0	1.5549e-15	1.8144e-12
50 / 50	1.0000	0	1.5794e-15	1.9418e-12
60 / 40	1.0000	0	1.5865e-15	2.8009e-12
70 / 30	1.0000	0	1.7155e-15	2.2832e-12
80 / 20	1.0000	0	1.8211e-15	1.7961e-12
90 / 10	1.0000	0	1.8297e-15	2.4494e-12

Table 4.5: Effect of changing train-test ratio for H_2 . Total number of raw R^2 is 4 (4 matrix elements).

that the linear diatomic molecule fits well with the prediction. For the H_2O from Table 4.7, along with the graphs presented in Figure 4.1, we can see that the prediction accuracy increases when the ratio of training set increases. This is not uncommon, because when the algorithm has more data to learn, it performs better on the prediction on test set. After about half the dataset is used, the prediction error converges, which means that we should use at least 50% of training data for prediction. We can also compare the result of prediction on test set to the prediction on training set for the molecule H_2O , as shown in Table 4.8. We can see that when the trained parameters applied on the training set, it always performs well because the parameters obtained from learning the train set and applied on training set itself. The slight increase in inaccuracy might because the model learns more from the training set, thus includes more errors. This is also explain the slight decrease in accuracy of test set when the train / test ratio approach 90% / 10%, when the model overfitting because it learns to includes more errors, the parameters obtained will not generalize well on the remaining test set, thus the accuracy decrease.

Train/ test (%)	R^2 (avg)	Number of $R^2 < 0.99$	RMSE	MAPE (%)
10 / 90	1.0000	0	2.5703e-15	5.9730e-11
20 / 80	1.0000	0	2.5651e-15	9.6785e-11
30 / 70	1.0000	0	4.6838e-16	3.3151e-11
40 / 60	1.0000	0	6.3370e-16	3.1370e-11
50 / 50	1.0000	0	4.2805e-16	3.1807e-11
60 / 40	1.0000	0	4.6337e-16	4.3634e-11
70 / 30	1.0000	0	4.6371e-16	4.2886e-11
80 / 20	1.0000	0	4.7790e-16	2.7191e-11
90 / 10	1.0000	0	4.7687e-16	2.3042e-11

Table 4.6: Effect of changing train-test ratio for LiH. Total number of raw R^2 is 36 (36 matrix elements).

Train / test (%)	R^2 (avg)	Number of $R^2 < 0.99$	RMSE	MAPE (%)
10 / 90	0.7263	14	6.3961e-15	47.8891
20 / 80	0.9300	11	5.2899e-15	19.2694
30 / 70	0.9807	11	2.3555e-15	14.1691
40 / 60	0.9853	11	2.2518e-15	12.3486
50 / 50	0.9871	11	2.2293e-15	11.7226
60 / 40	0.9859	11	2.2284e-15	11.8586
70 / 30	0.9846	11	2.4220e-15	12.1611
80 / 20	0.9833	11	2.5575e-15	12.4601
90 / 10	0.9806	11	2.5998e-15	12.9319

Table 4.7: Effect of changing train-test ratio for H₂O (on test set). Total number of raw R^2 is 49 (49 matrix elements).

Train / test (%)	R^2 (avg)	Number of $R^2 < 0.99$	RMSE	MAPE (%)
10 / 90	0.9909	11	5.4748e-16	4.7156
20 / 80	0.9930	11	6.8027e-16	6.4713
30 / 70	0.9944	11	7.7386e-16	7.4523
40 / 60	0.9937	11	8.6183e-16	8.3613
50 / 50	0.9938	11	9.5769e-16	8.4502
60 / 40	0.9943	11	1.0519e-15	8.4931
70 / 30	0.9943	11	1.1250e-15	8.4330
80 / 20	0.9941	11	1.2301e-15	8.5112
90 / 10	0.9937	11	1.3636e-15	8.7488

Table 4.8: Effect of changing train-test ratio for H₂O (on train set). Total number of raw R^2 is 49 (49 matrix elements).

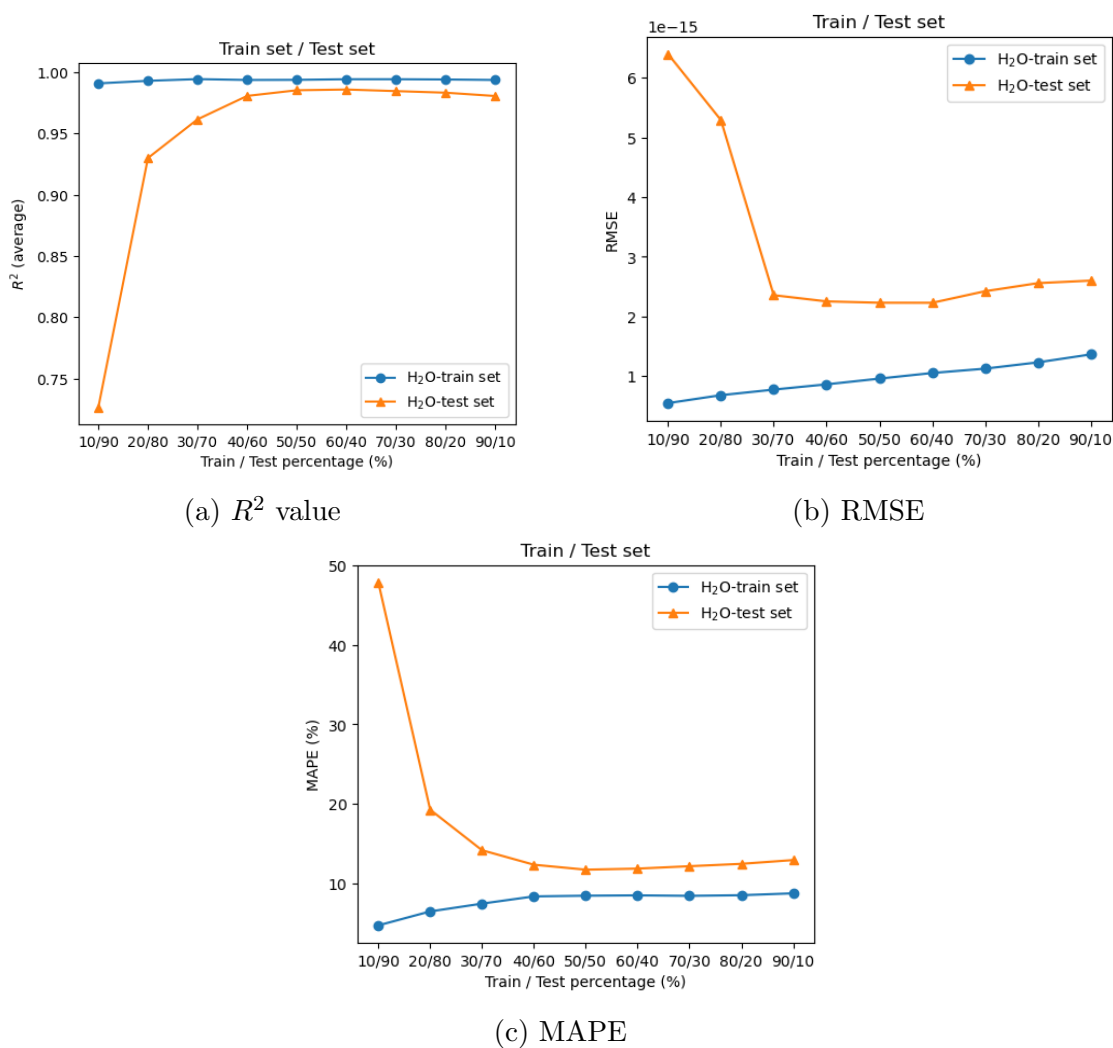


Figure 4.1: Train set and test set plots across different train-test percentage for H₂O molecule dataset

Train / test (%)	R^2	
	10000 time steps	20000 time steps
10 / 90	0.7263	0.9140
20 / 80	0.9300	0.9771
30 / 70	0.9807	0.9895
40 / 60	0.9853	0.9938
50 / 50	0.9871	0.9954
60 / 40	0.9859	0.9961
70 / 30	0.9846	0.9964
80 / 20	0.9833	0.9964
90 / 10	0.9806	0.9966

Table 4.9: R^2 values for effect of increase dataset size of H₂O

4.1.3 Effect of increasing dataset size

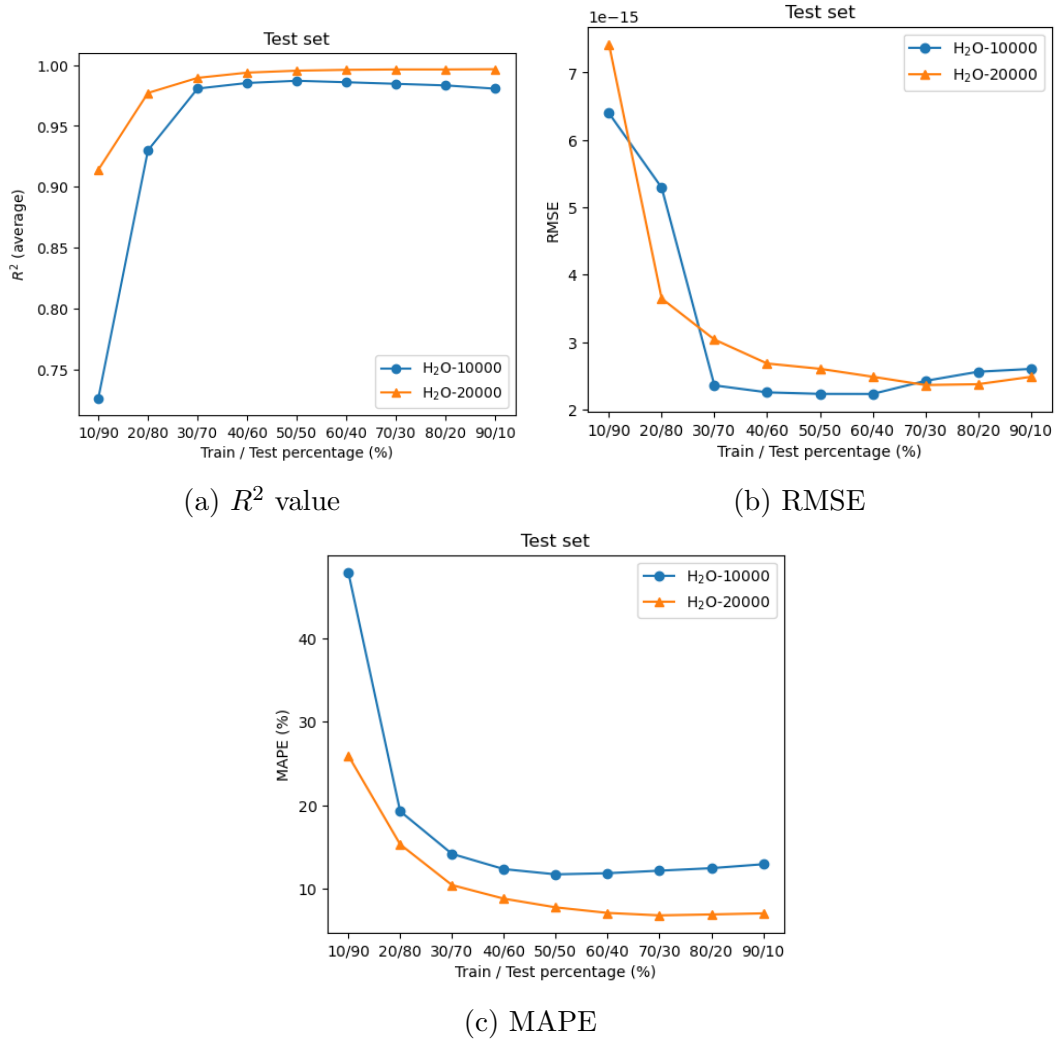
We have generated different datasets with different amount of time steps. The prediction accuracy increases when we include more time steps in the H₂O dataset. The results are shown in the Table 4.9, 4.10, and 4.11 with plots shown in Figure 4.2 . By including twice the results of time steps, we include also double the training data in each train-test ratio, and longer propagation results. We can see that longer propagation lowers the inaccuracy of prediction.

Train / test (%)	RMSE	
	10000 time steps	20000 time steps
10 / 90	6.3961e-15	7.4209e-15
20 / 80	5.2899e-15	3.6447e-15
30 / 70	2.3555e-15	3.0357e-15
40 / 60	2.2518e-15	2.6802e-15
50 / 50	2.2293e-15	2.6018e-15
60 / 40	2.2284e-15	2.4839e-15
70 / 30	2.4220e-15	2.3607e-15
80 / 20	2.5575e-15	2.3730e-15
90 / 10	2.5998e-15	2.4823e-15

Table 4.10: RMSE values for effect of increase dataset size of H₂O

Train / test (%)	MAPE (%)	
	10000 time steps	20000 time steps
10 / 90	47.8891	25.9440
20 / 80	19.2694	15.3066
30 / 70	14.1691	10.4344
40 / 60	12.3486	8.8079
50 / 50	11.7226	7.7699
60 / 40	11.8586	7.1045
70 / 30	12.1611	6.8094
80 / 20	12.4601	6.9226
90 / 10	12.9319	7.0532

Table 4.11: MAPE values for effect of increase dataset size of H₂O

Figure 4.2: Plots for effect of increase dataset size of H₂O

4.1.4 Effects of increasing basis size

We have also generated data file using different basis. The results of prediction with different basis set size is shown in Table 4.12, 4.13, 4.13 and the plots are shown in Figure 4.3. The increase of basis results in the analysis of 49 number of features increase to 169 number of features. With the increase of basis size, we expect to see the prediction more prone to error. However, the study shows the value of R^2 remains similar with increasing size of basis. The MAPE decrease with increase basis size. This might because basis is not the main factor for error.

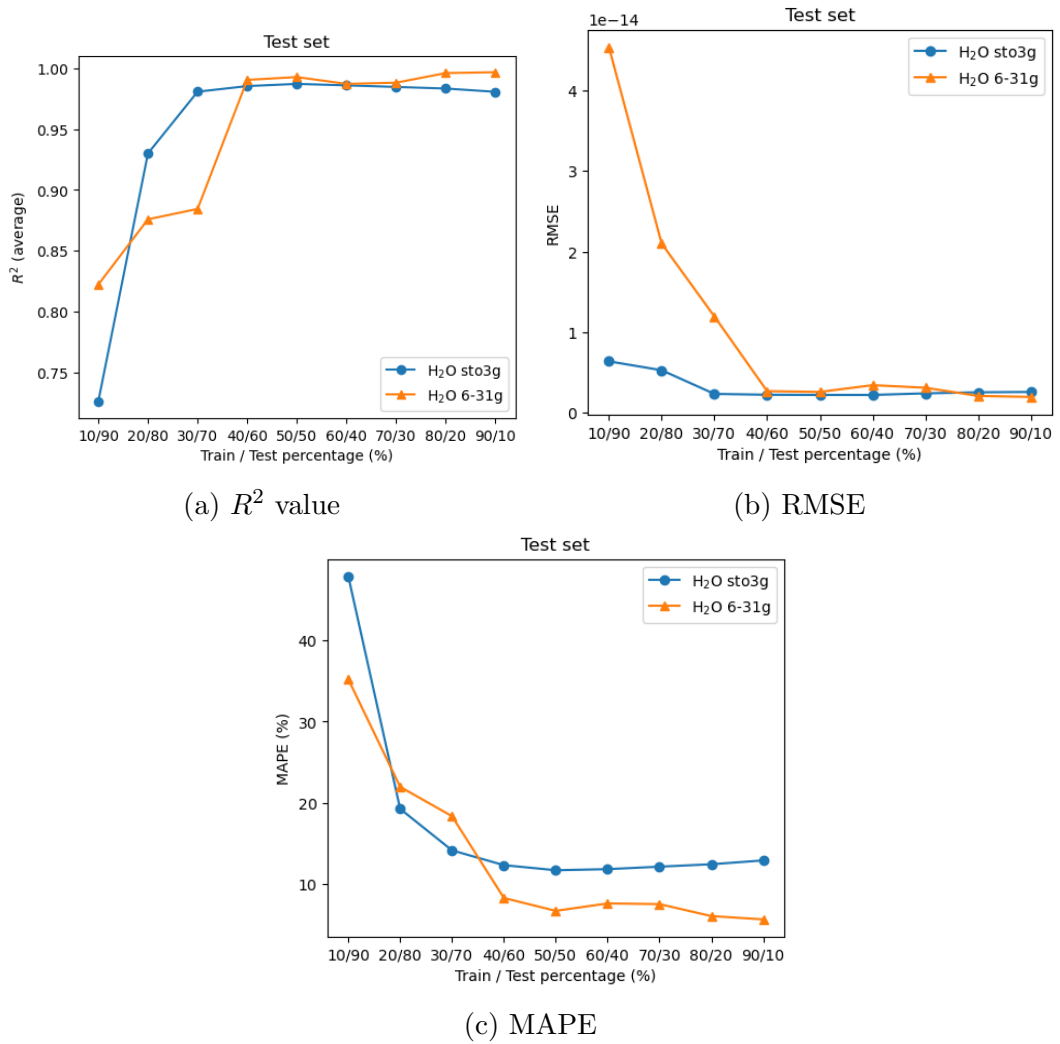
Train / test (%)	R^2	
	STO-3G	6-31g
10 / 90	0.7263	0.8223
20 / 80	0.9300	0.8759
30 / 70	0.9807	0.8844
40 / 60	0.9853	0.9904
50 / 50	0.9871	0.9927
60 / 40	0.9859	0.9871
70 / 30	0.9846	0.9881
80 / 20	0.9833	0.9960
90 / 10	0.9806	0.9967

Table 4.12: R^2 values for effect of increasing H₂O basis size

Train / test (%)	RMSE	
	STO-3G	6-31G
10 / 90	6.3961e-15	4.5343e-14
20 / 80	5.2899e-15	2.0986e-14
30 / 70	2.3555e-15	1.1961e-14
40 / 60	2.2518e-15	2.6978e-15
50 / 50	2.2293e-15	2.6006e-15
60 / 40	2.2284e-15	3.4412e-15
70 / 30	2.4220e-15	3.1247e-15
80 / 20	2.5575e-15	2.1073e-15
90 / 10	2.5998e-15	1.9800e-15

Table 4.13: RMSE values for effect of increasing H₂O basis size

Train / test (%)	MAPE (%)	
	STO-3G	6-31G
10 / 90	47.8891	35.2094
20 / 80	19.2694	21.9828
30 / 70	14.1691	18.3691
40 / 60	12.3486	8.3126
50 / 50	11.7226	6.7272
60 / 40	11.8586	7.6422
70 / 30	12.1611	7.5591
80 / 20	12.4601	6.0854
90 / 10	12.9319	5.6798

Table 4.14: MAPE values for effect of increasing H₂O basis sizeFigure 4.3: Plots for effect of increasing basis size H₂O

Train / test (%)	R^2		
	z-axis	y-axis	x -axis
10 / 90	1.0000	1.0000	1.0000
20 / 80	1.0000	1.0000	1.0000
30 / 70	1.0000	1.0000	1.0000
40 / 60	1.0000	1.0000	1.0000
50 / 50	1.0000	1.0000	1.0000
60 / 40	1.0000	1.0000	1.0000
70 / 30	1.0000	1.0000	1.0000
80 / 20	1.0000	1.0000	1.0000
90 / 10	1.0000	1.0000	1.0000

Table 4.15: R^2 values for effect of different orientations of LiH

4.1.5 Effect changing orientation of LiH

We have also propagated the simulation using different alignment of LiH molecule to the field. The result of this section is shown in Table 4.15, 4.16, and 4.17. For the small linear diatomic molecule such as LiH, the changing its alignment of bond axis in x, y and z axis to the perturbation with respect to the incoming field directed along the z-axis generally has no effect and performs good predictions.

4.1.6 Effect of including/removing delta kick values

We recall that our data are obtained from real-time TDHF simulation, which use a time-dependent perturbation or delta-kick function. We expect that applying a delta-kick, means that applying an electric field for a very short time at $t = 0$, equally excites all the optically allowed transitions. This instantaneous perturbation contains all frequencies. After the delta-kick perturbation, the system evolves without further perturbation, and the time dependent dipole moment of the system is recorded, and its Fourier transform will provides us the spectrum.[11] Thus it is interesting to investigate such effect on the fitting procedures. One may expect that the very first time steps of the time evolution are affected by this instantaneous perturbation.

The number of data removed are 0 (remove 0 values), 500 (-500 values) from the first 500 time steps and 1000 (-1000 values) data from the first 1000 time steps. We have to adjust the `kick_size` and `remove_kick`. For example we can choose to remain full dataset, remove first 500 time steps data or remove first 1000 time steps data. Then, the number of training data for three experiments will be 1000 (no data removed), 950 and 900 respectively at 10% training data. The results for H₂O molecule in Table 4.21,

Train (%) / test (%)	RMSE		
	z-axis	y-axis	x-axis
10 / 90	2.5703e-15	3.1005e-15	2.0882e-15
20 / 80	2.5651e-15	7.1957e-16	7.1372e-16
30 / 70	4.6838e-16	6.9349e-16	9.2730e-16
40 / 60	6.3370e-16	7.3439e-16	8.9264e-16
50 / 50	4.2805e-16	7.0324e-16	1.0604e-15
60 / 40	4.6337e-16	6.6837e-16	1.0948e-15
70 / 30	4.6371e-16	7.6146e-16	5.5240e-16
80 / 20	4.7790e-16	8.4679e-16	6.8802e-16
90 / 10	4.7687e-16	9.0468e-16	5.1977e-16

Table 4.16: RMSE values for effect of different orientations of LiH

Train (%) / test (%)	MAPE (%)		
	z-axis	y-axis	x-axis
10 / 90	5.9730e-11	0.0531	4.0356e-09
20 / 80	9.6785e-11	0.0639	3.3887e-09
30 / 70	3.3151e-11	0.0622	5.9054e-09
40 / 60	3.1370e-11	0.0440	4.8174e-09
50 / 50	3.1807e-11	0.0477	6.0926e-09
60 / 40	4.3634e-11	0.0527	6.9722e-09
70 / 30	4.2886e-11	0.0568	2.5358e-09
80 / 20	2.7191e-11	0.0586	3.3569e-09
90 / 10	2.3042e-11	0.0659	6.9096e-10

Table 4.17: MAPE values for effect of different orientations of LiH

Train / test (%)	R^2 (remove 0 values)	R^2 (- 500 values)	R^2 (- 1000 values)
10 / 90	1.0000	1.0000	1.0000
20 / 80	1.0000	1.0000	1.0000
30 / 70	1.0000	1.0000	1.0000
40 / 60	1.0000	1.0000	1.0000
50 / 50	1.0000	1.0000	1.0000
60 / 40	1.0000	1.0000	1.0000
70 / 30	1.0000	1.0000	1.0000
80 / 20	1.0000	1.0000	1.0000
90 / 10	1.0000	1.0000	1.0000

Table 4.18: R^2 values of removing different delta kick values effect in LiH dataset

4.22 and 4.23 and their plots in Figure 4.4 shows that the initial values are playing some roles in the prediction. However, the reason of the different values of accuracy needs further investigation. The results shown in this removing delta kick values also could be due to the presence of noise in the dataset, because the noise directly affect the prediction accuracy. This can be seen in the prediction of LiH datasets in Table 4.18, 4.19, 4.20 when it does not have significant noise present in the data, removing the initial values does not affect the accuracy of predictions. Thus, the prediction maintained good performance.

The number of data to remove is intuitive guess. In the work of Bhat [1], they removed the data from first two time steps for training, claiming to be avoiding large numerical time-derivatives associated with delta kick perturbation. However, our approach is different, and hence removing data with only two time steps will not affect the result. As shown in the results, when we increase the number of training data, the delta kick effect will not be important, and the results converge to the same value when we remove different number of initial values. This is expected as we include more training data, the learning is more effective.

Train / test (%)	RMSE (remove 0 values)	RMSE (- 500 values)	RMSE (- 1000 values)
10 / 90	2.5703e-15	1.2931e-14	2.3168e-14
20 / 80	2.5651e-15	6.8319e-16	6.4109e-16
30 / 70	4.6838e-16	4.9289e-16	6.2429e-16
40 / 60	6.3370e-16	5.0133e-16	8.4336e-16
50 / 50	4.2805e-16	6.6477e-16	8.8344e-16
60 / 40	4.6337e-16	4.8949e-16	5.1286e-16
70 / 30	4.6371e-16	4.8934e-16	5.0558e-16
80 / 20	4.7790e-16	4.7423e-16	4.7600e-16
90 / 10	4.7687e-16	4.7356e-16	4.8125e-16

Table 4.19: RMSE values of removing different delta kick values effect in LiH dataset

Train / test (%)	MAPE (%) (removed 0 values)	MAPE (%) (-500 values)	MAPE (%) (-1000 values)
10 / 90	5.9730e-11	2.4134e-10	3.5110e-10
20 / 80	9.6785e-11	4.2547e-11	5.0416e-11
30 / 70	3.3151e-11	2.5297e-11	5.2530e-11
40 / 60	3.1370e-11	4.2485e-11	5.3185e-11
50 / 50	3.1807e-11	4.1904e-11	5.7543e-11
60 / 40	4.3634e-11	5.1763e-11	6.0447e-11
70 / 30	4.2886e-11	4.6946e-11	4.8251e-11
80 / 20	2.7191e-11	2.5655e-11	2.5623e-11
90 / 10	2.3042e-11	2.3403e-11	2.3699e-11

Table 4.20: MAPE values of removing different delta kick values effect in LiH dataset

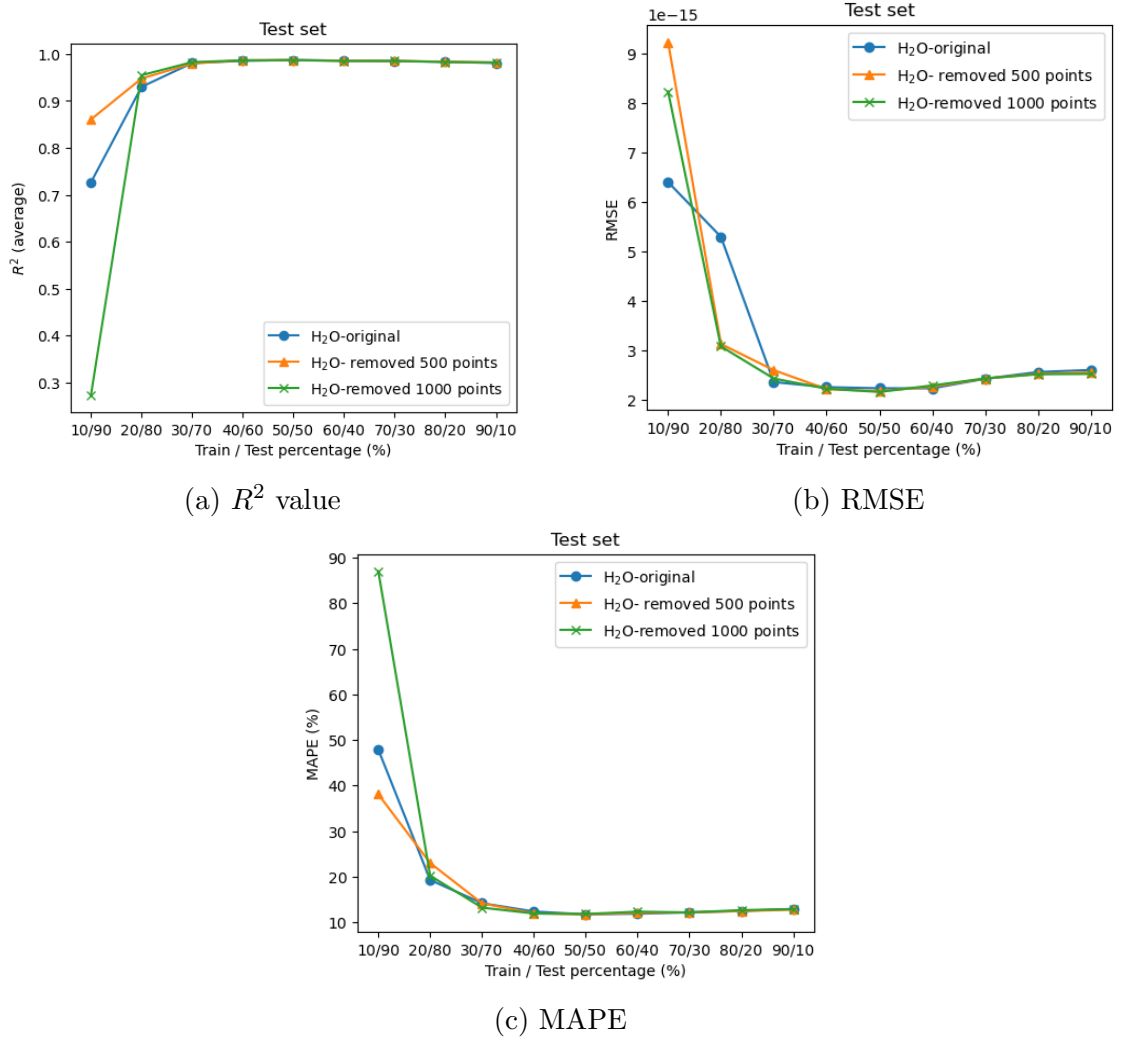
Train / test (%)	R^2 (remove 0 values)	R^2 (- 500 values)	R^2 (- 1000 values)
10 / 90	0.7263	0.8605	0.2727
20 / 80	0.9300	0.9478	0.9549
30 / 70	0.9807	0.9797	0.9827
40 / 60	0.9853	0.9866	0.9865
50 / 50	0.9871	0.9870	0.9870
60 / 40	0.9859	0.9857	0.9850
70 / 30	0.9846	0.9852	0.9855
80 / 20	0.9833	0.9836	0.9829
90 / 10	0.9806	0.9817	0.9821

Table 4.21: R^2 values of removing different delta kick values effect in H₂O dataset

Train / test (%)	RMSE (remove 0 values)	RMSE (- 500 values)	RMSE (- 1000 values)
10 / 90	6.3961e-15	9.2236e-15	8.2263e-15
20 / 80	5.2899e-15	3.1240e-15	3.0757e-15
30 / 70	2.3555e-15	2.5955e-15	2.4263e-15
40 / 60	2.2518e-15	2.2126e-15	2.2208e-15
50 / 50	2.2293e-15	2.1719e-15	2.1554e-15
60 / 40	2.2284e-15	2.2604e-15	2.2862e-15
70 / 30	2.4220e-15	2.4248e-15	2.4278e-15
80 / 20	2.5575e-15	2.5209e-15	2.5133e-15
90 / 10	2.5998e-15	2.5523e-15	2.5198e-15

Table 4.22: RMSE values of removing different delta kick values effect in H₂O dataset

Train / test (%)	MAPE (%) (remove 0 values)	MAPE (%) (- 500 values)	MAPE (%) (- 1000 values)
10 / 90	47.8891	38.0718	87.0299
20 / 80	19.2694	22.9850	20.2110
30 / 70	14.1691	14.1418	13.1986
40 / 60	12.3486	11.9410	11.9156
50 / 50	11.7226	11.7175	11.8068
60 / 40	11.8586	12.0615	12.3141
70 / 30	12.1611	12.1579	12.1450
80 / 20	12.4601	12.4331	12.6502
90 / 10	12.9319	12.7542	12.8635

Table 4.23: MAPE values of removing different delta kick values effect in H₂O datasetFigure 4.4: Plots of remove delta kick data for H₂O

Train / test (%)	R^2	
	original	filtered
10 / 90	0.7263	1.0000
20 / 80	0.9300	1.0000
30 / 70	0.9807	1.0000
40 / 60	0.9853	1.0000
50 / 50	0.9871	1.0000
60 / 40	0.9859	1.0000
70 / 30	0.9846	1.0000
80 / 20	0.9833	1.0000
90 / 10	0.9806	1.0000

Table 4.24: R^2 for H₂O when remove zeros

4.1.7 Effect of removing zeros

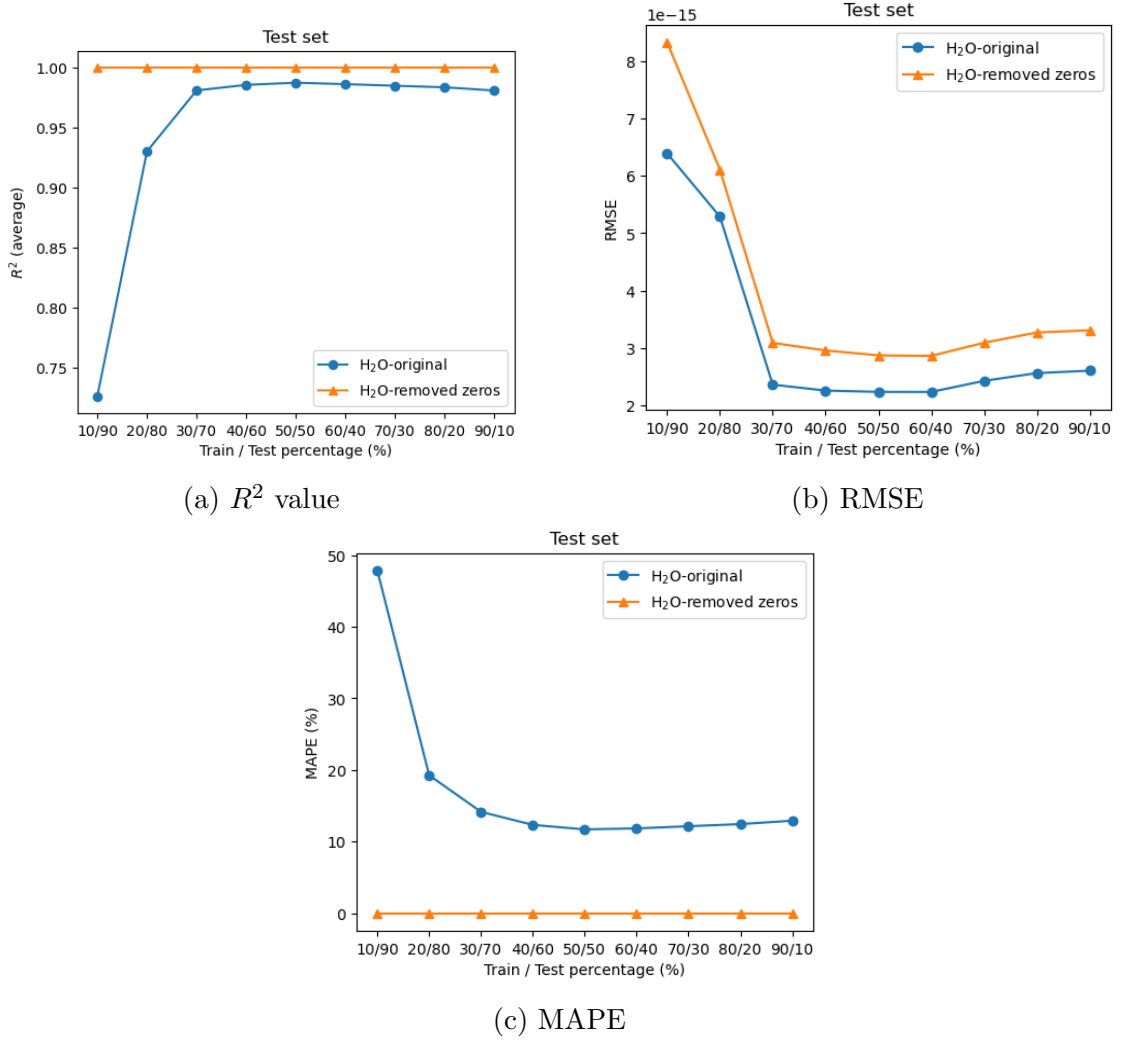
The parameters need to be adjusted to use in this section are `check_noise` and `filter_noise`. The results for finding out the features which are very small values are shown in Table 4.24, 4.25, and 4.26 with graphs presented in Figure 4.5. The values of the density matrices and also the Fock matrices which less than 1×10^{-13} are removed. We can see significant increase on the accuracy of the prediction, which shows that the inaccuracy of predictions are caused by the presence of zeros, or noise in the dataset during propagation. The machine learning learned those wrong data that affect our parameters values. By removing the zero arrays, we can also save computational cost.

Train/ test (%)	RMSE	
	original	filtered
10 / 90	6.3961e-15	8.3372e-15
20 / 80	5.2899e-15	6.1119e-15
30 / 70	2.3555e-15	3.0840e-15
40 / 60	2.2518e-15	2.9534e-15
50 / 50	2.2293e-15	2.8667e-15
60 / 40	2.2284e-15	2.8580e-15
70 / 30	2.4220e-15	3.0871e-15
80 / 20	2.5575e-15	3.2668e-15
90 / 10	2.5998e-15	3.3057e-15

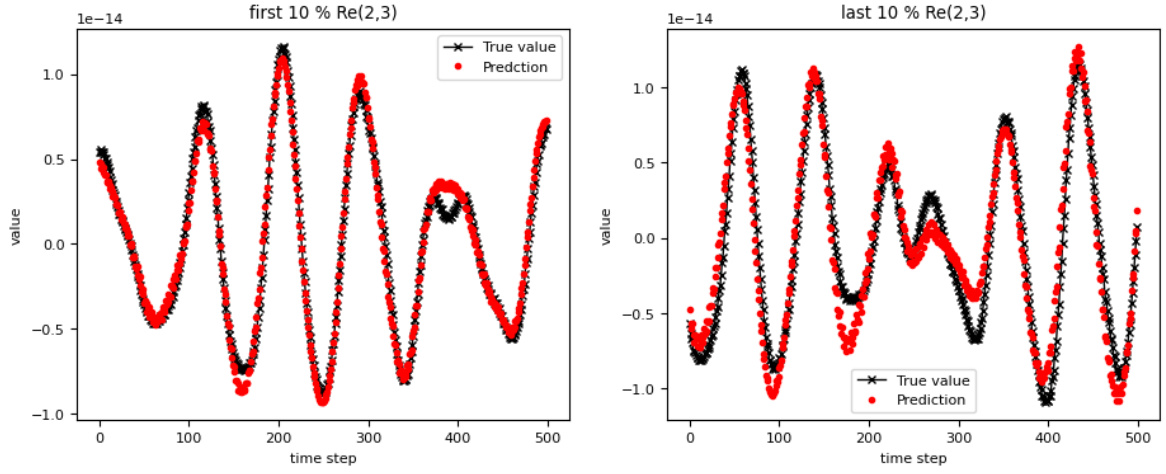
Table 4.25: RMSE for H₂O when remove zeros

Train/ test (%)	MAPE (%)	
	original	filtered
10 / 90	47.8891	5.3409e-09
20 / 80	19.2694	4.4545e-09
30 / 70	14.1691	9.6521e-10
40 / 60	12.3486	9.3540e-10
50 / 50	11.7226	8.1191e-10
60 / 40	11.8586	9.2086e-10
70 / 30	12.1611	1.1921e-09
80 / 20	12.4601	1.3824e-09
90 / 10	12.9319	1.4691e-09

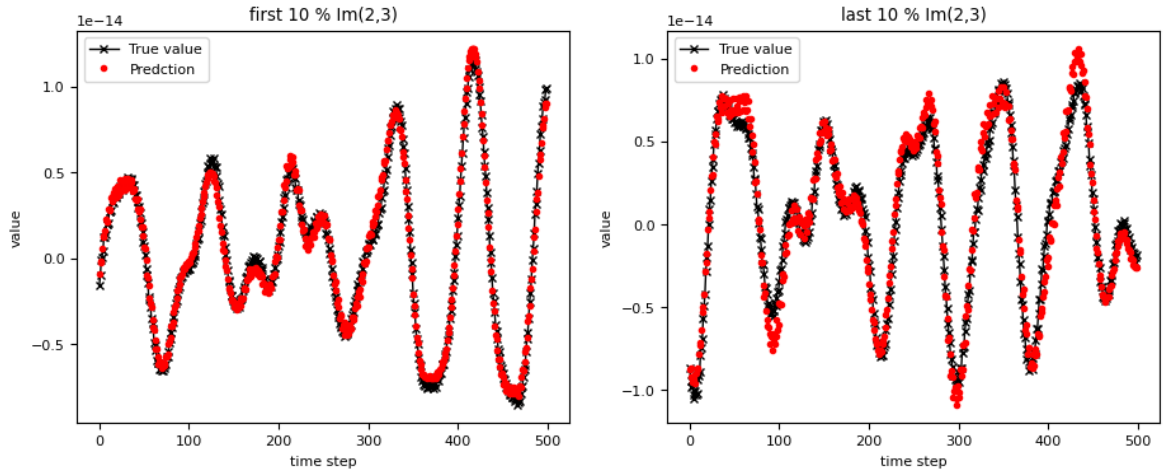
Table 4.26: MAPE for H₂O when remove zeros

Figure 4.5: Remove noise effect in H₂O dataset for linear regression method

In the Figure 4.6 Re(2,3) and Im(2,3) represents the time series plots of H₂O dataset with respect to the 2nd row and 3rd column of real and imaginary part of Fock matrices. The red dot represents the predicted value Fock matrix elements, and the black cross represents the true value of Fock matrix elements. We can see that the plotting of noise with small values, 1×10^{-14} in this case, do not fit well in our prediction. In contrast, the plotting 4.7 Re(2,6) and Im(2,6), which is the real values of 2nd row and 6th column Fock matrices prediction with non-zero values 4.7 fits well with the true Fock matrix value. The “first” and “last” 10% in the plot label means that we plot the first 500 data points, and last 500 data points, out of the 5000 data points.

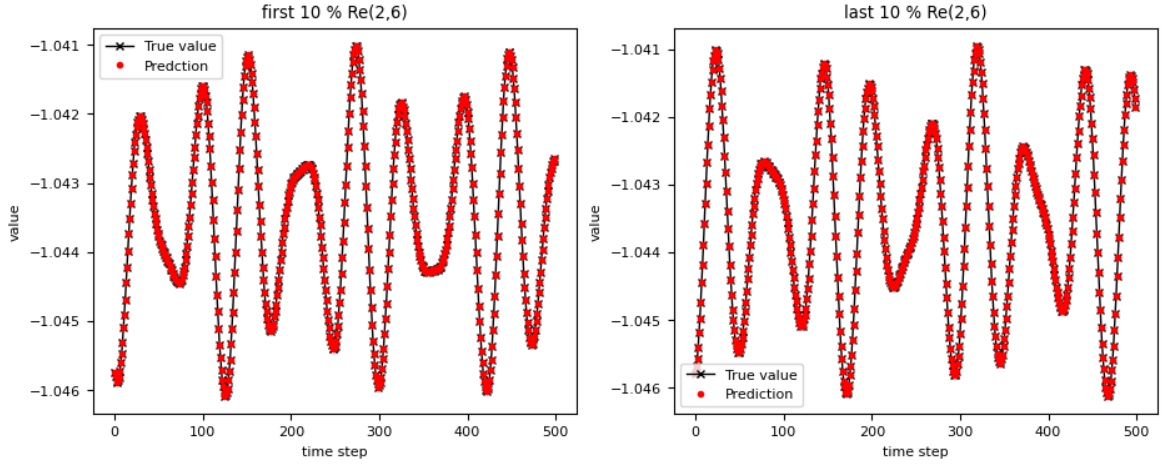


(a) First 10% (500 time steps) and last 10% (500 time steps) of real value of the Fock matrix element (2, 3)

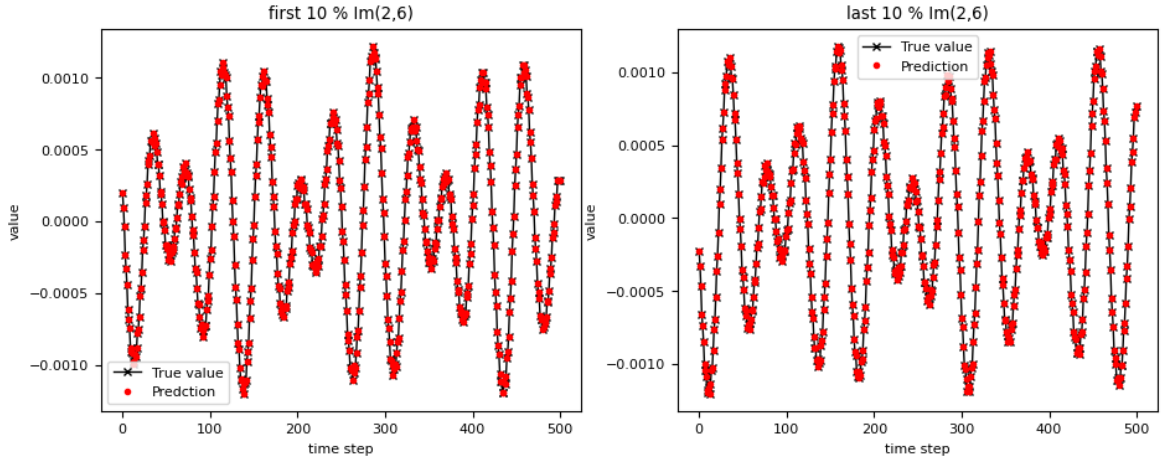


(b) First 10% (500 time steps) and last 10% (500 time steps) of imaginary value of Fock the matrix element (2, 3)

Figure 4.6: Prediction with actual value plots of (a) Real value and (b) imaginary value of the Fock matrix element (2, 3) on test set for identically zero values, across time propagation of an element matrix for H₂O linear regression method.



(a) First 10% (500 time steps) and last 10% (500 time steps) of real value of the Fock matrix element (2, 6)



(b) First 10% (500 time steps) and last 10% (500 time steps) of imaginary value of the Fock matrix element (2, 6)

Figure 4.7: Prediction with actual value plots of (a) Real value and (b) imaginary value of the Fock matrix element (2, 6) on test set for non-zero values, across time propagation of an element matrix for H₂O linear regression method.

4.2 Neural Network method

Next, we will investigate the implementation of neural network. We are interested to observe the effect of different number of hidden layers, number of neurons and using different types of optimizers. We also want to use grid search to find our best parameters. We also want to investigate the effect of removing small values in data in neural network. Throughout this section, some common hyperparameters are constant unless specified and investigated. Those adjustable hyperparameters that are fixed as constant in this thesis are the learning rate, number of epochs, and batch size.

Hidden layers	R^2	RMSE	MAPE (%)
0	0.9653	3.0896e-05	20.8026
1	0.9364	1.1012e-04	37.8441
2	0.9332	2.0219e-04	45.5113
3	0.9080	3.5229e-04	58.3728
4	0.8615	4.4750e-04	73.1377
5	0.8316	4.9396e-04	78.5049

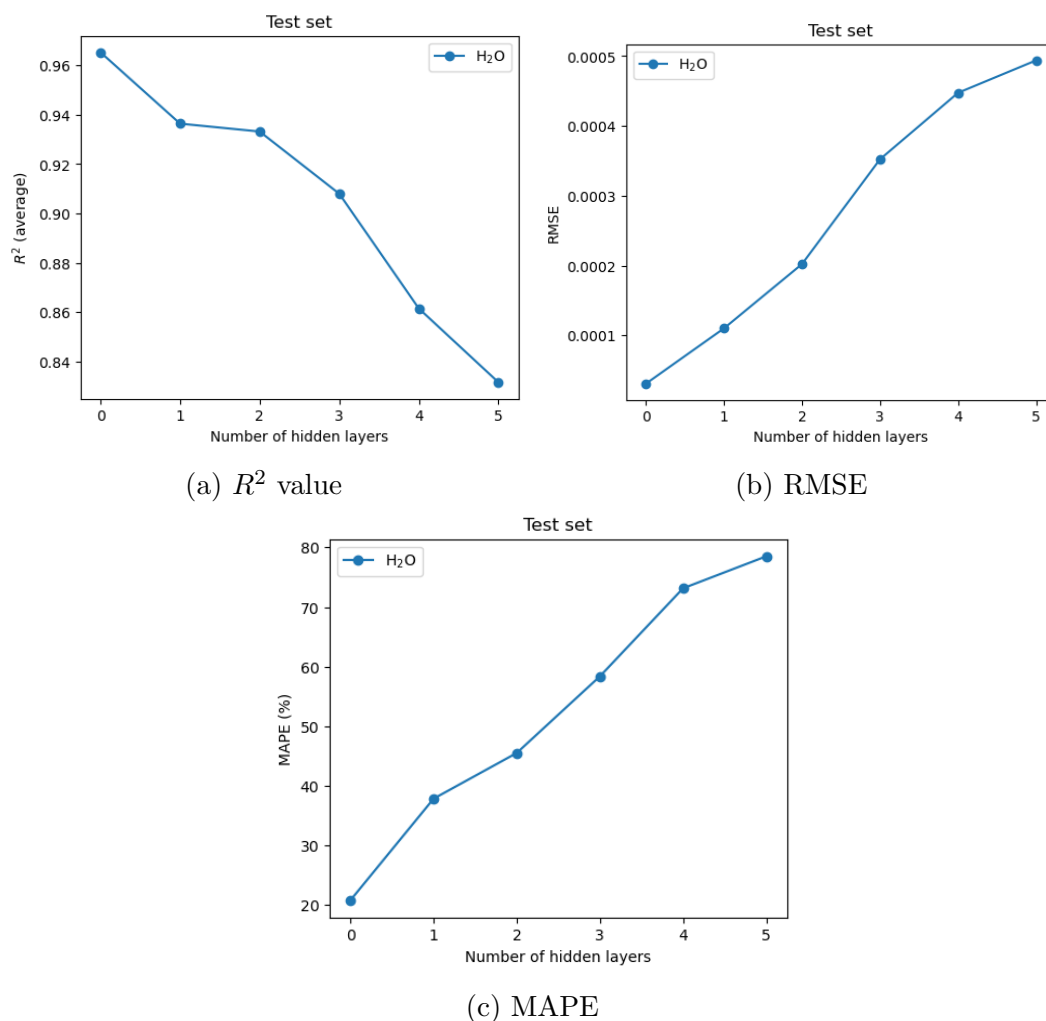
Table 4.27: Effect of changing number of hidden layers in neural network

4.2.1 Effect of changing number of hidden layers

The result is shown in Table 4.27. Changing of number of layers can be done manually by commenting out the lines that specify layers in the code. With an increasing number of layers, the prediction accuracy decreases. This is because the data is a linear regression task, and increasing non-linear layer will decrease the accuracy. Without hidden layer, the model performs as linear regression. Increasing the number of hidden layer, does not necessarily increase the prediction accuracy. In the neural network, we only need hidden layers if the data need to be explained non-linearly.

For no hidden layer, the framework can represent linear separable functions or decisions. For one hidden layers, we can approximate any function that contains a continuous mapping from one finite space to another. For two hidden layers, it can represent an approximate any smooth mapping to any accuracy. Although neural networks with two hidden layers can represent any kind of shape, the problems that require two hidden layers are rarely encountered, and we have no practical reason to use two or more hidden layers.[61]

In the hidden layer, we use the ReLU activation function, and the other parameters in the hidden layers are set for default values. Since the results become worse when we introduce non-linearity in our machine learning framework, we can verify that our entire dataset can be described by linear relationships.

Figure 4.8: Plots for effects of increase hidden layer of H₂O dataset

4.2.2 Effect of changing number of neurons

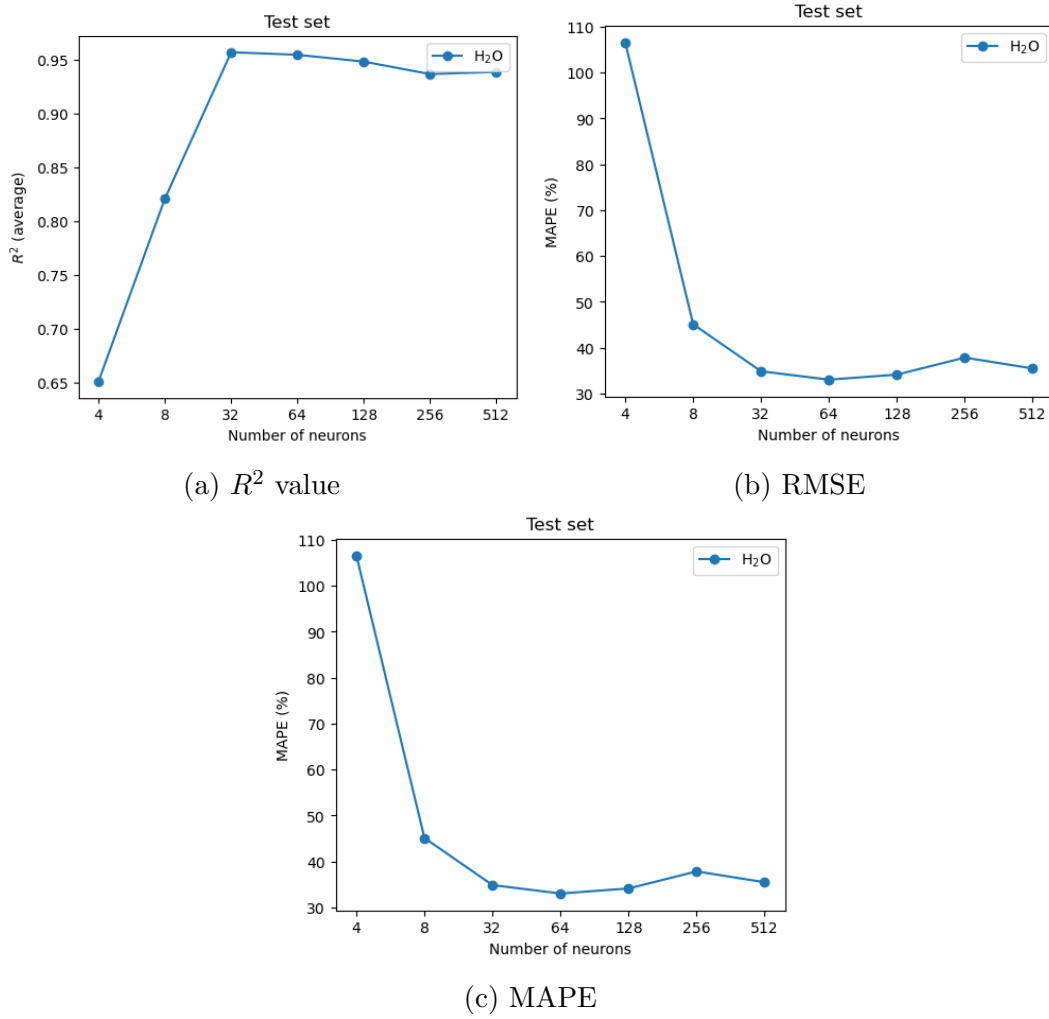
We can manually change the number of neurons by commenting out specific lines in the code. From the Table 4.28 and Figure 4.9, we can see the effect of different number of neurons on the prediction accuracy. The number of neurons in the input layer, is decided by the number of input parameters, similarly in the output layer that decided by number of output parameters. However, the number of neurons in the hidden layer is not fixed. Using few neurons in the hidden layers, will result underfitting, while too many neurons in the hidden layers will result in overfitting.[61]

In this section, we are using one hidden layer. Generally, we can see that the best number of neurons in the one hidden layer are around 32 in this case. This is consistent with the rule-of-thumb method that are used for determining the number of neurons in the hidden nodes.[62] The number of neurons in one hidden layer are 2/3 or 70% to 90% of the size of the input layer. In this study of H₂O of STO-3G basis set, the number of input features is 49, and $49 \times 2/3 \approx 32.67$. The slight decrease of accuracy when

Number of neurons	R^2	RMSE	MAPE
4	0.6513	6.1128e-04	106.5921
8	0.8207	3.2986e-04	45.1173
32	0.9567	1.2957e-04	34.8596
64	0.9543	1.0647e-04	33.0328
128	0.9480	1.2035e-04	34.1324
256	0.9364	1.1012e-04	37.8441
512	0.9384	1.1715e-04	35.4772

Table 4.28: Effect of changing number of neurons

increasing number of neurons might because of the presence of overfitting in training data, which cannot be generalized to the test set.

Figure 4.9: Effect of increasing number of neurons for H₂O dataset

4.2.3 Effect of different types of optimizer

We can change the types of optimizers by manually adjusting the parameter with name `optimizer_select` to `adam` or `sgd` in the code. The results of using different types of optimizer are shown in Table 4.29, 4.30, and 4.31 with graph shown in Figure 4.10. Overall, using Adam optimizer results in better prediction than SGD optimizer. We can see that Adam optimizer works effectively in small number of training data, while SGD only rise in the prediction accuracy with increasing training data. This might because Adam generally converges faster and more reliable than SGD, and is preferred choice due to its efficiency, while SGD may require more training iterations. This is because Adam combines two techniques compared to SGD, which are momentum and adaptive learning rates. Momentum can speeds up the training by accelerating gradients in the right directions, intuitively can say as rolling ball down the hill with increasing momentum.[63] The adaptive learning rates adapts the learning rate for each parameter for faster convergence, compared to SGD only use a fixed learning rate.[64]

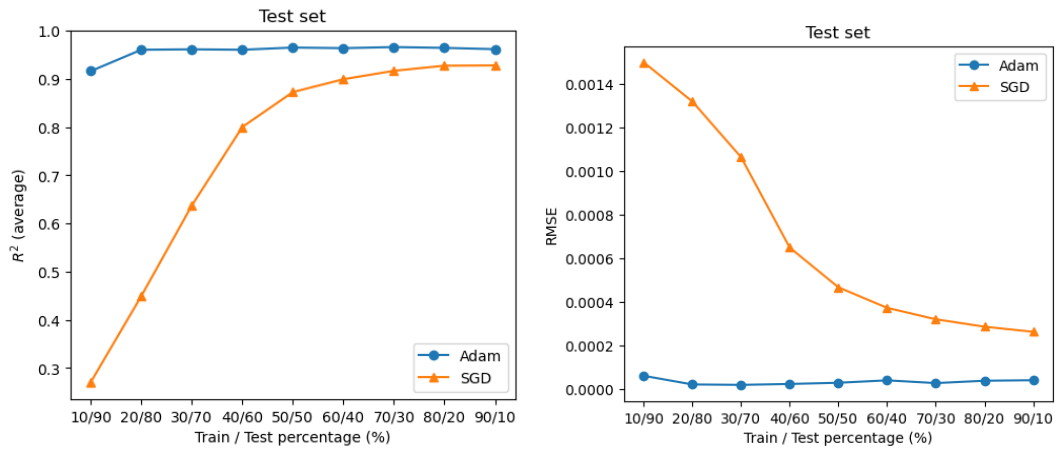
Train (%) / test (%)	R^2	
	Adam	SGD
10 / 90	0.9164	0.2708
20 / 80	0.9607	0.4495
30 / 70	0.9616	0.6375
40 / 60	0.9607	0.8004
50 / 50	0.9653	0.8729
60 / 40	0.9640	0.8995
70 / 30	0.9663	0.9170
80 / 20	0.9646	0.9276
90 / 10	0.9619	0.9282

Table 4.29: R^2 values for effect of different types of optimizer

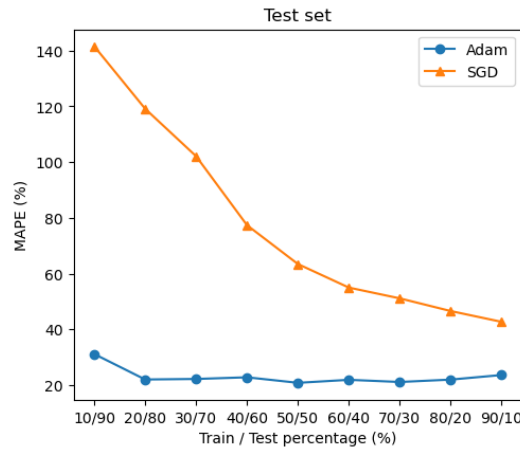
Train (%) / test (%)	RMSE	
	Adam	SGD
10 / 90	6.2699e-05	1.5001e-03
20 / 80	2.3160e-05	1.3201e-03
30 / 70	2.1228e-05	1.0650e-03
40 / 60	2.5414e-05	6.5039e-04
50 / 50	3.0896e-05	4.6840e-04
60 / 40	4.1969e-05	3.7433e-04
70 / 30	2.9355e-05	3.2192e-04
80 / 20	4.0133e-05	2.8795e-04
90 / 10	4.2502e-05	2.6443e-04

Table 4.30: RMSE values for effect of different types of optimizer on H₂O dataset

Train (%) / test (%)	MAPE (%)	
	Adam	SGD
10 / 90	31.1325	141.5725
20 / 80	21.9972	119.1432
30 / 70	22.1862	102.1256
40 / 60	22.7833	77.3742
50 / 50	20.8026	63.4196
60 / 40	21.8768	54.9761
70 / 30	21.1037	51.1071
80 / 20	21.9568	46.5602
90 / 10	23.6050	42.6785

Table 4.31: MAPE values for effect of different types of optimizer on H₂O dataset(a) R^2 value

(b) RMSE



(c) MAPE

Figure 4.10: Plots for effects of different optimizer on H₂O dataset

	Without grid search	With grid search
No. of hidden layers	0	0
Number of neurons	256	128
Types of optimizer	adam	sgd
R^2	1.0000	1.0000
RMSE	1.0526e-08	1.5555e-08
MAPE	8.6083e-06	1.2450e-04

Table 4.32: Comparison for results of using grid search and without grid search for H_2 dataset

4.2.4 Grid search

As the above hyperparameters configuration are intuitive guesses, we want to see if using a simple grid search would result in better predictions. Grid search is a model hyperparameter optimization technique that solves the difficulty of configuring the neural networks, because there are a lot of parameters to be set. A dictionary of hyperparameters are set, then the grid search will try each combination of the hyperparameters. We have defined the dictionary of hyperparameters of 0, 1 and 2 hidden layers, adam or sgd optimizers, and with 32, 64, 128, 256 number of neurons. Here we have used 3-fold cross validation, which splits data into 3 equal segments for training and testing. After the algorithm found out the best combination of hyperparameters, the hyperparameters will be passed down to the neural network.

In the results shown in Table 4.32, 4.33 and 4.34 for molecules H_2 , LiH and H_2O respectively, compared to the guessed default hyperparameters we have defined, we can say that we have good implementation of the three hyperparameters (number of hidden layers, number of neurons, types of optimizers) without using the grid search. This shows that learning the neural network theory and understanding the data we are working is important, to save time for tuning. The grid search has chosen the best hyperparameter value that gives the best result. The grid search method will become handy, when we do not know entirely the best hyperparameters to use, or when there are several number of hyperparameters we need to tune. The process of completing the grid search takes time, and once we have obtained the best result, we are not sure whether a change in parameters would give us better result. The only way to check is to change the parameter grid dictionary and run through the grid search again. Therefore, it is better to understand the patterns of underlying data, or we have to blindly try each combination of hyperparameters. We can also try another method which is random search, which randomly finds the combination of each hyperparameters when the number of tuning needed is large, so that we do not need to try each combinations iteratively like grid search.

	Without grid search	With grid search
No. of hidden layers	0	0
Number of neurons	256	64
Types of optimizer	adam	adam
R^2	0.9991	0.9836
RMSE	1.5794e-05	4.4476e-05
MAPE	3.9484	8.3663

Table 4.33: Comparison for results of using grid search and without grid search for LiH dataset

	Without grid search	With grid search
No. of hidden layers	0	0
Number of neurons	256	32
Types of optimizer	adam	adam
R^2	0.9653	0.9636
RMSE	3.0896e-05	2.8993e-05
MAPE	20.8026	20.3415

Table 4.34: Comparison for results of using grid search and without grid search for grid search H₂O dataset

Train (%) / test (%)	R^2	
	original	filtered
10 / 90	0.9164	0.9998
20 / 80	0.9607	0.9999
30 / 70	0.9616	0.9999
40 / 60	0.9607	0.9999
50 / 50	0.9653	0.9999
60 / 40	0.9640	0.9999
70 / 30	0.9663	0.9999
80 / 20	0.9646	0.9999
90 / 10	0.9619	0.9998

Table 4.35: R^2 values of effect of removing zeros for neural network

4.2.5 Effect of removing zeros

From the linear regression method, we know that the presence of identically zero values affect the learning. We also investigated the effect of removing noise from the data in neural network. The results are shown in the Table 4.35, 4.36, 4.37, and with the plots shown in Figure 4.11. The procedure of removing the features that are zeros is same as in the linear regression method, which is finding out the absolute and mean of the values for density and Fock matrices across time series that are less than 1×10^{-13} . We can see that the prediction accuracy rise significantly, shows that the noise will also directly affect the training of the model of neural network. This means that the presence of noise that caused the model to learn on wrong values and perform badly.

Train (%) / test (%)	RMSE	
	Original	filtered
10 / 90	6.2699e-05	3.9836e-05
20 / 80	2.3160e-05	2.2238e-05
30 / 70	2.1228e-05	2.3352e-05
40 / 60	2.5414e-05	1.7888e-05
50 / 50	3.0896e-05	3.5559e-05
60 / 40	4.1969e-05	2.4900e-05
70 / 30	2.9355e-05	2.6616e-05
80 / 20	4.0133e-05	2.8381e-05
90 / 10	4.2502e-05	4.1885e-05

Table 4.36: RMSE values of effect of removing zeros for neural network

Train (%) / test (%)	MAPE (%)	
	original	filtered
10 / 90	31.1325	2.4458
20 / 80	21.9972	1.8379
30 / 70	22.1862	1.6666
40 / 60	22.7833	1.6141
50 / 50	20.8026	2.3548
60 / 40	21.8768	3.3991
70 / 30	21.1037	2.5147
80 / 20	21.9568	2.7024
90 / 10	23.6050	3.0417

Table 4.37: MAPE values of effect of removing zeros for neural network

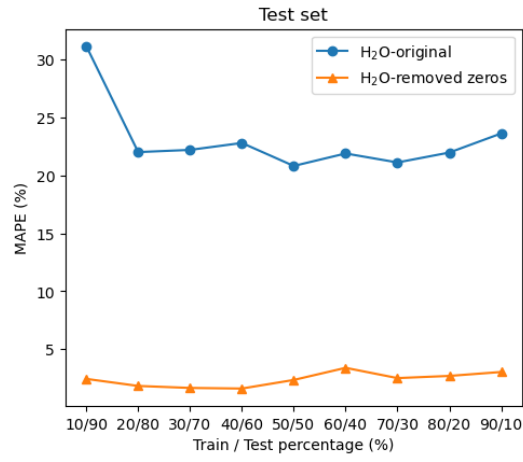
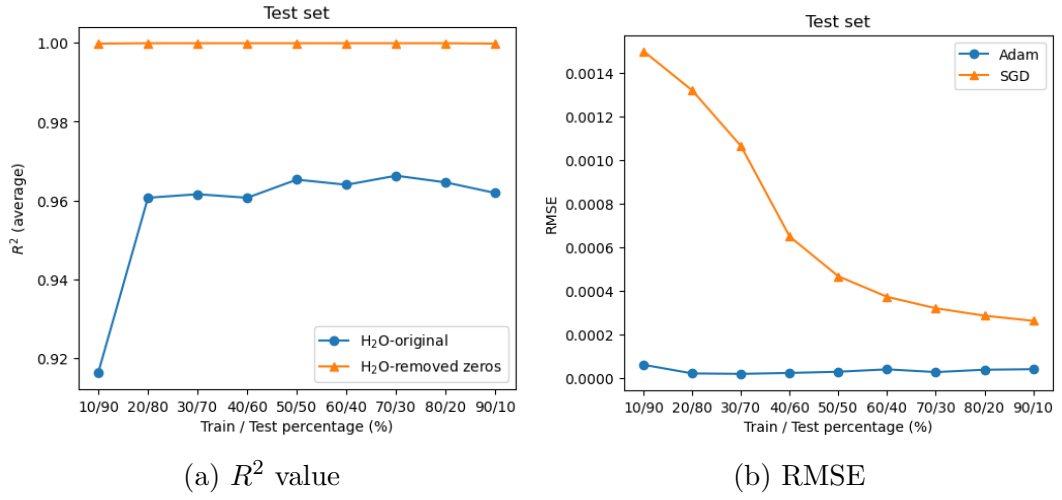


Figure 4.11: Plots for the effect of removing noise values for neural network in H₂O dataset

4.3 Discussion

In this study we have investigated the different machine learning approach and implementation methods, and their effect on the quality of Fock matrix prediction. The results showed that with different parameters and optimization techniques, we will get different outcome. Furthermore, the dataset also affects the result of prediction. In the machine learning there several factors that can influence the accuracy of the prediction, and the existence of noise is a major factor. The errors or irrelevant data are referred as noise. The largest factor that affects out prediction results are the presence of noise, which are the small values that approaching zeros in the dataset. These presence of small values make the machine learning algorithms to learn the wrong pattern in fitting. Therefore, we need to do data preprocessing in machine learning. This data preprocessing is important to improve the quality of data, we can do data cleaning, normalization or outlier elimination. We can remove the zeros to make the data smaller, so that we can reduce computational cost. We must be aware that the symmetry of the molecule causes this presence of small values in the matrix elements during the real-time propagation. We have also tested the different alignment of LiH molecule with respect to the pulse in z-axis, but the result is not significant with this small linear molecule.

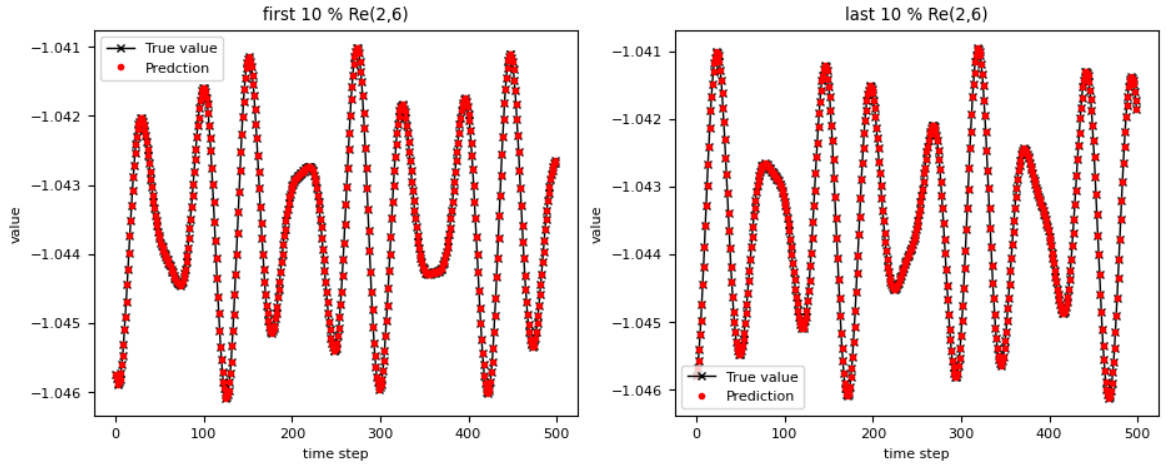
Computational complexity is the study of how much resources required to solve a problem change with its size.[65] In this discussion, we focus on the time complexity of our code. The normal equation method, compute the inverse of $\mathbf{X}^T\mathbf{X}$, and this is an $(n+1) \times (n+1)$ matrix, where n is the number of features. The computational complexity of inverting the matrix, is usually about $O(n^{2.4})$ to $O(n^3)$. The `LinearRegression` class is based on the `scipy.linalg.lstsq()` least squares function, that use singular value decomposition (SVD) to solve the parameter vector, that has complexity about $O(n^2)$, which is quadratic time. This means that if the feature is doubled, or in our case density matrices doubled, the computation time is scale by 4.[35] Meanwhile for neural network, when the number and size of the layers increase, the model complexity increase. It is usually measured by its number of hidden units and layers, connectivity, form of activation functions, and parameters of the algorithms.[66] Comparing to the Hartree-Fock algorithm, the evaluation and manipulation for two electron integrals explicitly, is $O(M^4)$ as the basis size M which dominates the runtime, and the algorithm scales somewhere between $O(M^2)$ and $O(M^3)$. [65] But in our linear regression implementation, when the size of the matrix elements is $M \times M$, the total input scaled as M^2 , which cause out total product of complexity to actually become $O(n^4)$, that is same as the usual Hartree-Fock calculation scheme. Therefore, an alternative implementation using machine learning framework to replace the dependence of the calculation of two-electron integrals is a possible way for the electrodynamics simulation, but we have to develop the program more efficiently with less time complexity. We should know the difference time complexity and actual training time. Time complexity is the computational time required for training, scales with input size. Actual training time is the duration of training, and it will be influenced by computer hardware capabilities, software optimization, and implementation efficiency. It will not change the complexity class. Therefore, to further reduce the training time, we can use GPU, TPU or

optimized libraries.[67]

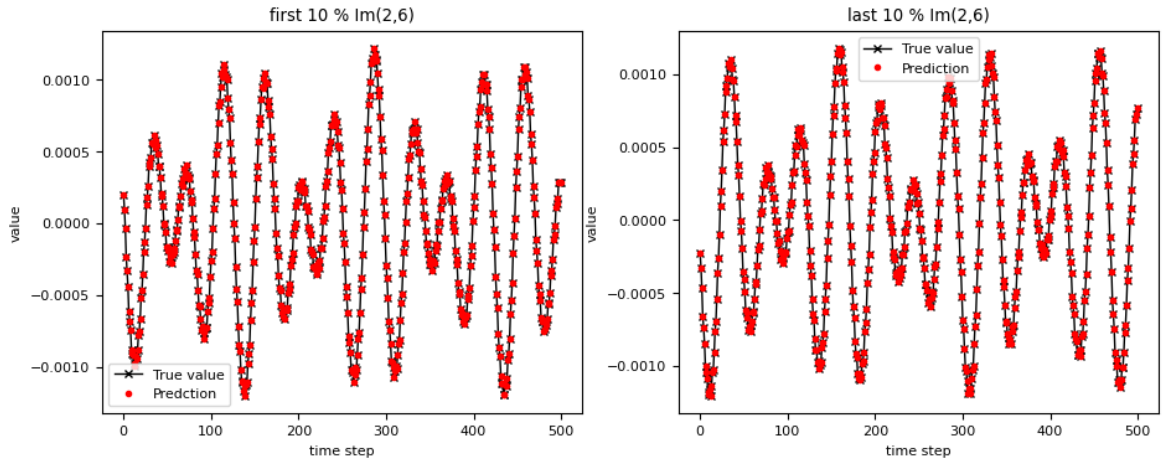
We can see that the increase in dataset size increases the prediction accuracy. In machine learning model, the size of dataset plays a critical role, because it impacts the model’s ability to generalize unseen data. A larger dataset will provides more examples of underlying patterns for the models to learn, reduce overfitting risk, and recognize broader trends. In this work, when we double the size of the data, the training data scaled as twice, hence perform better training and predictions. Increasing the molecular basis size do not have significant impact on the machine leaning

However, there is a cost to increase the dataset size and the basis size. The method presented in this work is not suitable for investigating a larger basis or larger dataset size, as well as large molecules. For example, the H_2O molecule we are investigating have size 7×7 for STO-3G basis set and 13×13 for 6-31G basis set. If we increase basis set size to aug-cc-pVDZ, the H_2O molecule will have parameter size of 41×41 , and increases to 92×92 for aug-cc-pVTZ basis. In this case we cannot inspect each matrix elements individually, and there will be too many propagation graphs to inspect, by using the implementation in this thesis. When the basis size becomes larger, the number of parameters needed to optimize for both linear regression and neural network increases very large. Hence, it will not enough to load all the data at once into the memory, once we increase the basis for training because the computer RAM capacity might not be enough. We need to resort to load the data in batch, or use high performance computer for training, and this might not be the best efficient way to perform machine-learned electrodynamic simulation. One dimensionality reduction technique is to reduce the number of model parameters by modelling only non-zero elements of the Hamiltonian matrix. Therefore, we have to develop an improvement or a new method, that do not rely only on linear regression parameters.

Generally, the data presented a linear regression relationship between the density and Fock matrices. Thus the linear regression technique by Scikit-Learn library performs better than the neural network method, as seen from the result of the performance metrics comparing two different methods. This might because the neural network have more hyperparameters that affect result, and a simple linear regression prediction only rely on the reduction of mean squared error. In the Figure 4.12 and Figure 4.13, we can compare the prediction of both methods on the same matrix element across time steps, and we can look at the performance metrics in Table 4.38.



(a) First 10% (500 time steps) and last 10% (500 time steps) of Real value of the Fock matrix element (2, 6)

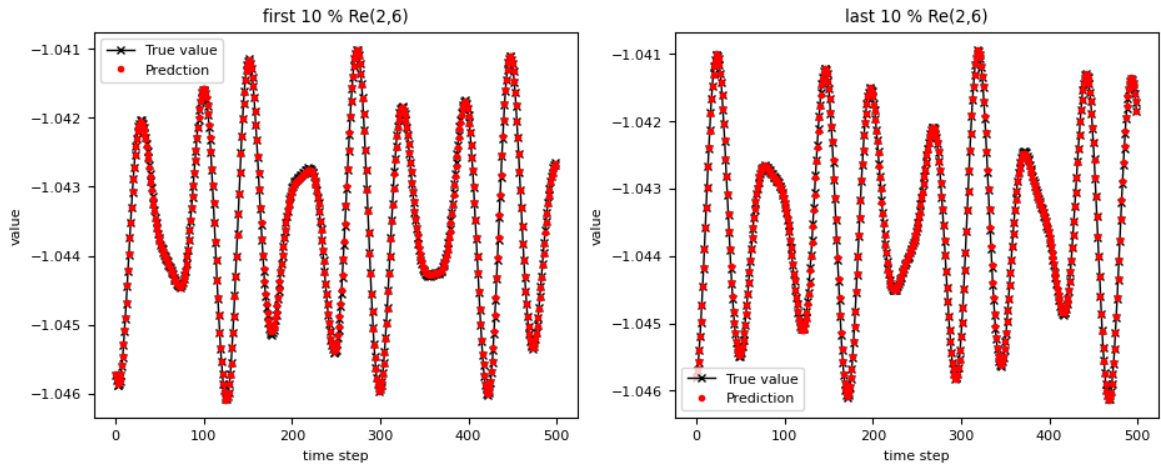


(b) First 10% (500 time steps) and last 10% (500 time steps) of Imaginary value of the Fock matrix element (2, 6)

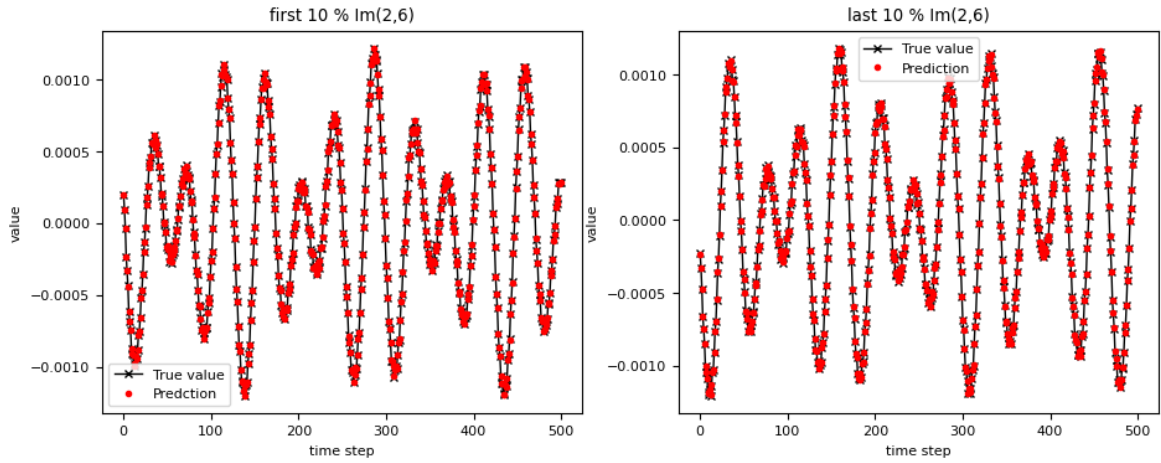
Figure 4.12: Prediction with actual value plots of (a) Real value and (b) imaginary value of the Fock matrix element (2, 6) on test set for non-zero values, across time propagation of an element matrix for H_2O linear regression method.

	Linear regression	Neural network
R^2	0.9871	0.9653
RMSE	2.2293e-15	3.0896e-05
MAPE (%)	11.7226	20.8026

Table 4.38: Comparison of linear regression and neural network result without filtered out zero, for H₂O dataset at 50% train-test value



(a) First 10% (500 time steps) and last 10% (500 time steps) of real value of the Fock matrix element (2, 6)



(b) First 10% (500 time steps) and last 10% (500 time steps) of Imaginary value of the Fock matrix element (2, 6)

Figure 4.13: Prediction with actual value plots of (a) real value and (b) imaginary value of the Fock matrix element (2, 6) on test set for non-zero values, across time propagation of an element matrix for H₂O neural network method.

We have tried with different configuration of neural network, in case the data has underlying non-linear relationship or patterns that we do not recognized. In this dataset, the introduction of non-linear parameters do not increase the accuracy of predictions. When we have different machine learning models, we should always choose the model that is simple and explainable, that gives us the best result.

In the above methods used and results shown, we have tested different approaches in machine learning and their effects in the training and testing. This is a first method to hypothesize what we will expect when we proceed further to learn the Fock matrix for larger basis sets and larger molecules.

In future work, we would like to extend this research to investigate the use of learned Fock matrix to apply in propagation. As shown in the work of Bhat and coworkers[1], the learned Hamiltonian can be used for propagation in both the field-off and field-on settings.

The method used in this work do not discover underlying physics because we individually analyze the element of the density and Fock matrices, and do not see beyond the linear regression relationship between the two variables, thus we need a new approach. One methods is that practitioners should push further to apply scientific machine learning (SciML), beyond the classic machine learning methods as presented in this work. One scientific machine learning technique is the Physics-Informed Neural Network, which solves problems involving Partial Differential Equations (PDEs) that represents the physics of the problem, other than deducing solution based solely on data.[68] Incorporating physical laws that govern the system into the data-driven machine learning methodologies will give us new insights on underlying fundamental laws.

Conclusion

In this thesis we have briefly summarised the theory of real-time time-dependent electronic structure methods and presented the basics of machine learning methods that are useful in this field of research. This thesis shows that the use of machine learning methods such as linear regression and neural network can be promising to learn and predict the Fock matrix given the time series of density matrices generated in a real-time simulation. We have also investigated the factors that affect the accuracy of the training and prediction, thus completing the previous literature works by Bhat and coworkers [1, 2, 3]. In general, machine learning is good at making predictions, and when more training data is included, the prediction accuracy increases. The presence of noise in the dataset is the main factor that affects the accuracy of prediction. The use of different combinations of hyperparameters in the neural network also affects the accuracy of the prediction. Therefore, the data set should be carefully analysed before performing machine learning to increase the accuracy.

Once the factors that affect the accuracy of the prediction are identified, this machine-learned Fock matrix can be effectively used for propagation instead of the exact Hamiltonian, which is costly. We also analysed the time complexity of the linear regression and neural network, which shows that it can potentially reduce the cost of computation compared to the usual evaluation of the two-electron integral. The Python programme implemented in this research uses the Python library scikit-learn and Keras, which are freely available to download online. Everyone can easily implement the machine learning for the tasks on hand, and the tutorials are freely available. The current version of the code is far to be optimal and clearly does not necessarily reduce the computational cost compared to usual propagation scheme, however, there are ways to optimise our code, such as running the neural network on a GPU.

This work shows that the machine learning can potentially be a useful tool to be applied in the electrodynamics simulations. Linear regression method is one of the earliest form of machine learning, it is easy to use and is effective in most applications. However, the limitation of using machine learning is the quality of the data, which has to be inspected, requiring pre-processing of data if necessary. Besides, in the case of neural network, we do not know the factor and underlying data patterns that give us the best combination of hyperparameters. We need to tune the hyperparameters for the best result, that will describe underlying relationship between the input and output data. However, if implemented correctly, machine learning are expected to save computational time compared to usual propagation.

We also need to wisely choose the machine learning algorithms, to effectively make

predictions because different algorithms are suited for different tasks. We can find that, due to the simplicity of linear regression, it is faster than neural network and more effective in this framework. Thus, if the simplest algorithm can perform the prediction accurately, we do not need to apply for more complex algorithms that consume larger resources.

As work in progress, we expect to investigate to apply the scheme worked out here to larger molecular system with extended basis set, and develop more effective machine learning methods when dealing with data that require large memory. In addition, we can use other regression methods such as ridge regression models, to test if there is an increase of prediction accuracy. Besides, we aim to incorporate the machine learning algorithms into the propagation code to execute the machine learning in real-time, instead of extracting out the density and Fock matrices independently. The next step for future work will be using this learned Fock matrix for the electron density evolution, because we are interested in knowing how good the predicted Fock matrix in the actual real-time propagation scheme. Furthermore, we can use the learned Fock matrix to simulate the electron dynamics in the presence or absence of an applied electric field, as shown in the previous literature. [1, 2, 3] Although there are many aspects to improve in this work, we have here demonstrated the potential of using machine learning in real-time time-dependent electronic structure methods. The real-time methods have well-developed theories, future developments should explore the methods beyond the usual framework, and we can use knowledge from different disciplines such as machine learning. In this work, we have shown the importance of studying machine learning methods. This brings the need to cultivate researchers with skills that interweaves chemistry, physics, and computer science to this emerging field. With the machine learning approach, we may expect some useful advancement of real-time electron dynamics towards more realistic systems and better descriptions of complex phenomena.

Bibliography

- [1] Harish Bhat, Karnamohit Ranka, and Christine Isborn. Machine learning a molecular hamiltonian for predicting electron dynamics. 07 2020.
- [2] Prachi Gupta, Harish Bhat, Karnamohit Ranka, and Christine Isborn. Statistical learning method for predicting density-matrix based electron dynamics, 07 2021.
- [3] Harish S. Bhat, Prachi Gupta, and Christine M. Isborn. Scalable learning of potentials to predict time-dependent hartree-fock dynamics, 2024.
- [4] Joshua J. Goings, Patrick J. Lestrangle, and Xiaosong Li. Real-time time-dependent electronic structure theory. *WIREs Computational Molecular Science*, 8(1):e1341, 2018.
- [5] Jacek Jakowski and Keiji Morokuma. Liouville-von neumann molecular dynamics. *The Journal of chemical physics*, 130:224106, 07 2009.
- [6] Xiaosong Li, Niranjana Govind, Christine Isborn, A. Eugene III DePrince, and Kenneth Lopata. Real-time time-dependent electronic structure theory. *Chemical Reviews*, 120(18):9951–9993, 2020. PMID: 32813506.
- [7] David Joshua. Tannor. *Introduction to quantum mechanics : a time-dependent perspective*. Univ. Science Books, 2007.
- [8] A.D. McLachlan. A variational solution of the time-dependent schrodinger equation. *Molecular Physics*, 8(1):39–44, 1964.
- [9] Seyed Mahdi Mastoor and Amirhossein Ahmadkhan Kordbacheh. Scalable machine learning models for predicting quantum transport in disordered 2d hexagonal materials, 2025.
- [10] Samat Tussupbayev, Niranjana Govind, Kenneth Lopata, and Christopher J. Cramer. Comparison of real-time and linear-response time-dependent density functional theories for molecular chromophores ranging from sparse to high densities of states. *Journal of Chemical Theory and Computation*, 11(3):1102–1109, 2015. PMID: 26579760.
- [11] Rajarshi Sinha-Roy, Pablo Garca-Gonzalez, Xchitl Lopez Lozano, Robert L. Whetten, and Hans-Christian Weissker. Identifying electronic modes by fourier transform from -kick time-evolution tddft calculations. *Journal of Chemical Theory and Computation*, 14(12):6417–6426, 2018. PMID: 30404453.

- [12] Sanguk Woo, Hyein Jung, and Yohan Yoon. Real-time uv/vis spectroscopy to observe photocatalytic degradation. *Catalysts*, 13:683, 03 2023.
- [13] K. Yabana and G. F. Bertsch. Time-dependent local-density approximation in real time: Application to conjugated molecules. *International Journal of Quantum Chemistry*, 75(1):55–66, 1999.
- [14] Miguel A. L. Marques, Xabier López, Daniele Varsano, Alberto Castro, and Angel Rubio. Time-dependent density-functional approach for biological chromophores: The case of the green fluorescent protein. *Phys. Rev. Lett.*, 90:258101, Jun 2003.
- [15] Christian Bressler and Majed Chergui. Ultrafast x-ray absorption spectroscopy. *Chemical Reviews*, 104(4):1781–1812, 2004. PMID: 15080712.
- [16] Ranelka G. Fernando, Mary C. Balhoff, and Kenneth Lopata. X-ray absorption in insulators with non-hermitian real-time time-dependent density functional theory. *Journal of Chemical Theory and Computation*, 11(2):646–654, 2015. PMID: 26579600.
- [17] Triet S. Nguyen, Joong Hoon Koh, Susan Lefelhocz, and John Parkhill. Black-box, real-time simulations of transient absorption spectroscopy. *The Journal of Physical Chemistry Letters*, 7(8):1590–1595, 2016. PMID: 27064028.
- [18] Umberto De Giovannini, Gustavo Brunetto, Alberto Castro, Jessica Walkenhorst, and Angel Rubio. Simulating pumpprobe photoelectron and absorption spectroscopy on the attosecond timescale with time-dependent density functional theory. *ChemPhysChem*, 14(7):1363–1376, 2013.
- [19] Alexey Gorlach, Ofer Neufeld, Nicholas Rivera, Oren Cohen, and Ido Kaminer. The quantum-optical nature of high harmonic generation. *Nature communications*, 11:4598, 09 2020.
- [20] Emanuele Coccia and Eleonora Luppi. Time-dependent ab initio approaches for high-harmonic generation spectroscopy. *Journal of Physics: Condensed Matter*, 34(7):073001, nov 2021.
- [21] Taeho Jo. *Machine Learning Foundations: Supervised, Unsupervised, and Advanced Learning*. 01 2021.
- [22] H J Kulik, T Hammerschmidt, J Schmidt, S Botti, M A L Marques, M Boley, M Scheffler, M Todorovi, P Rinke, C Oses, A Smolyanyuk, S Curtarolo, A Tkatchenko, A P Bartk, S Manzhos, M Ihara, T Carrington, J Behler, O Isayev, M Veit, A Grisafi, J Nigam, M Ceriotti, K T Schtt, J Westermayr, M Gastegger, R J Maurer, B Kalita, K Burke, R Nagai, R Akashi, O Sugino, J Hermann, F No, S Pilati, C Draxl, M Kuban, S Rigamonti, M Scheidgen, M Esters, D Hicks, C Toher, P V Balachandran, I Tamblyn, S Whitlam, C Bellinger, and L M Ghiringhelli. Roadmap on machine learning in electronic structure. *Electronic Structure*, 4(2):023004, aug 2022.

- [23] Types of Machine Learning - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/machine-learning/types-of-machine-learning/](https://www.geeksforgeeks.org/machine-learning/types-of-machine-learning/). [Accessed 01-09-2025].
- [24] John Keith, Valentin Vassilev-Galindo, Bingqing Cheng, Stefan Chmiela, Michael Gastegger, Klaus-Robert Müller, and Alexandre Tkatchenko. Combining machine learning and computational chemistry for predictive insights into chemical systems, 02 2021.
- [25] Regression in machine learning - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/machine-learning/regression-in-machine-learning/](https://www.geeksforgeeks.org/machine-learning/regression-in-machine-learning/). [Accessed 10-08-2025].
- [26] Ashok Kumar D and Nagarajan Kannathasan. A study and characterization of chemical properties of soil surface data using k-means algorithm. pages 264–270, 02 2013.
- [27] Reinforcement Learning - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/](https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/). [Accessed 02-09-2025].
- [28] Junwoong Yoon, Zhonglin Cao, Rajesh K Raju, Yuyang Wang, Robert Burnley, Andrew J Gellman, Amir Barati Farimani, and Zachary W Ulissi. Deep reinforcement learning for predicting kinetic pathways to surface reconstruction in a ternary alloy. *Machine Learning: Science and Technology*, 2(4):045018, 2021.
- [29] Beomchang Kang, Chaok Seok, and Juyong Lee. Prediction of molecular electronic transitions using random forests. *Journal of Chemical Information and Modeling*, 60(12):5984–5994, 2020. PMID: 33090804.
- [30] Sanggil Park, Herim Han, Hyungjun Kim, and Sunghwan Choi. Machine learning applications for chemical reactions. *Chemistry - An Asian Journal*, 17(14):e202200203, 2022.
- [31] Wenhong Yang, Timothy Tizhe Fidelis, and Wen-Hua Sun. Machine learning in catalysis, from proposal to practicing. *ACS Omega*, 5(1):83–88, 2020. PMID: 31956754.
- [32] Nicholas J. Boyer, Christopher Shepard, Ruiyi Zhou, Jianhang Xu, and Yosuke Kanai. Machine-learning electron dynamics with moment propagation theory: Application to optical absorption spectrum computation using real-time tddft. *Journal of Chemical Theory and Computation*, 21(1):114–123, 2025. PMID: 39729524.
- [33] Maxim Secor, Alexander V. Soudackov, and Sharon Hammes-Schiffer. Artificial neural networks as propagators in quantum dynamics. *The Journal of Physical Chemistry Letters*, 12(43):10654–10662, 2021. PMID: 34704767.

- [34] What is a Neural Network? - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/machine-learning/neural-networks-a-beginners-guide/](https://www.geeksforgeeks.org/machine-learning/neural-networks-a-beginners-guide/). [Accessed 10-08-2025].
- [35] Aurelien Geron. *Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Sebastopol, CA, 2017.
- [36] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released January 12, 2025.
- [37] Chayma Zatout. A Brief Introduction to Neural Networks: a Regression Problem — Towards Data Science — [towardsdatascience.com. https://towardsdatascience.com/a-brief-introduction-to-neural-networks-a-regression-problem-c58c26e18008/](https://towardsdatascience.com/a-brief-introduction-to-neural-networks-a-regression-problem-c58c26e18008/). [Accessed 23-08-2025].
- [38] Nadeem. Cost Function & Loss Function — [nadeemm.medium.com. https://nadeemm.medium.com/cost-function-loss-function-c3cab1ddffa4](https://nadeemm.medium.com/cost-function-loss-function-c3cab1ddffa4). [Accessed 01-09-2025].
- [39] Simeon Kostadinov. Understanding Backpropagation Algorithm — [medium.com. https://medium.com/data-science/understanding-backpropagation-algorithm-7bb3aa2f95fd](https://medium.com/data-science/understanding-backpropagation-algorithm-7bb3aa2f95fd). [Accessed 23-08-2025].
- [40] Jason Brownlee. Understand the Impact of Learning Rate on Neural Network Performance - MachineLearningMastery.com — [machinelearningmastery.com. https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/](https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/). [Accessed 02-09-2025].
- [41] Activation functions in Neural Networks - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/](https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/). [Accessed 10-08-2025].
- [42] Activation Functions in Neural Networks [12 Types & Use Cases] — [v7labs.com. https://www.v7labs.com/blog/neural-networks-activation-functions](https://www.v7labs.com/blog/neural-networks-activation-functions). [Accessed 10-08-2025].
- [43] ReLU Activation Function in Deep Learning - GeeksforGeeks — [geeksforgeeks.org. https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/](https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/). [Accessed 04-08-2025].
- [44] Thi-Lam-Thuy LE. How does ReLU enable Neural Networks to approximate continuous nonlinear functions? — Towards Data Science — [towardsdatascience.com. https://towardsdatascience.com/how-relu-enables-neural-networks-to-approximate-continuous-nonlinear-functions-f171b7859727/](https://towardsdatascience.com/how-relu-enables-neural-networks-to-approximate-continuous-nonlinear-functions-f171b7859727/). [Accessed 02-09-2025].

- [45] Abhishek Jaiswal. Understanding Optimizers in Neural Networks — abhishek.jaiswaal1810. <https://medium.com/@abhishek.jaiswaal1810/optimizers-in-neural-networks-4fb7adee4a63>. [Accessed 02-09-2025].
- [46] Adam Optimizer: Deep Learning — Ultralytics — ultralytics.com. <https://www.ultralytics.com/glossary/adam-optimizer>. [Accessed 02-09-2025].
- [47] What is Gradient Descent? — IBM — ibm.com. <https://www.ibm.com/think/topics/gradient-descent>. [Accessed 10-09-2025].
- [48] Dagang Wei. Demystifying the Adam Optimizer in Machine Learning — weidagang. <https://medium.com/@weidagang/demystifying-the-adam-optimizer-in-machine-learning-4401d162cb9e>. [Accessed 02-09-2025].
- [49] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 12 2014.
- [50] Mohamed Reyad, Amany Sarhan, and Mohammad Arafa. A modified adam algorithm for deep neural network optimization. *Neural Computing and Applications*, 35:1–18, 04 2023.
- [51] Vishal Yathish. Loss Functions and Their Use In Neural Networks — Towards Data Science — towardsdatascience.com. <https://towardsdatascience.com/loss-functions-and-their-use-in-neural-networks-a470e703f1e9/>. [Accessed 10-08-2025].
- [52] Juan Terven, Diana-Margarita Cordova-Esparza, Alfonzo Ramirez-Pedraza, and Edgar Chvez Urbiola. Loss functions and metrics in deep learning. a review, 07 2023.
- [53] Loss Functions in Deep Learning - GeeksforGeeks — geeksforgeeks.org. <https://www.geeksforgeeks.org/deep-learning/loss-functions-in-deep-learning/>. [Accessed 10-08-2025].
- [54] Omar Elharrouss, Yasir Mahmood, Yassine Bechqito, Mohamed Adel Serhani, Elarbi Badidi, Jamal Riffi, and Hamid Tairi. Loss functions in deep learning: A comprehensive review, 2025.
- [55] Haden Pelletier. Data Scaling 101: Standardization and Min-Max Scaling Explained — Towards Data Science — towardsdatascience.com. <https://towardsdatascience.com/data-scaling-101-standardization-and-min-max-scaling-explained-60789833e160/>. [Accessed 10-08-2025].
- [56] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [57] François Chollet et al. Keras. <https://keras.io>, 2015.

- [58] Double precision floating values in Python - AskPython — askpython.com. <https://www.askpython.com/python/examples/high-precision-numerical-calculations>. [Accessed 06-09-2025].
- [59] Prathik C. Should You Scale the Target Variable in Machine Learning? — prathik.codes. <https://medium.com/@prathik.codes/should-you-scale-the-target-variable-in-machine-learning-37fab15fe8d6>. [Accessed 28-08-2025].
- [60] Nima Beheshti. Cross Validation and Grid Search — Towards Data Science — towardsdatascience.com. <https://towardsdatascience.com/cross-validation-and-grid-search-efa64b127c1b/>. [Accessed 29-08-2025].
- [61] The Number of Hidden Layers — Heaton Research — web.archive.org. <https://web.archive.org/web/20140721050413/http://www.heatonresearch.com/node/707>. [Accessed 04-08-2025].
- [62] Saurabh Karsoliya and M. Abul Kalam Azad. Approximating number of hidden layer neurons in multiple hidden layer bpnn architecture. 2012.
- [63] Complete Guide to the Adam Optimization Algorithm — Built In — builtin.com. <https://builtin.com/machine-learning/adam-optimization>. [Accessed 07-09-2025].
- [64] Hey Amit. Stochastic Gradient Descent (SGD) and Adam — medium.com. <https://medium.com/data-scientists-diary/stochastic-gradient-descent-sgd-and-adam-4fe496ef1bbf>. [Accessed 07-09-2025].
- [65] James Daniel Whitfield, Peter John Love, and Aln Aspuru-Guzik. Computational complexity in electronic structure. *Phys. Chem. Chem. Phys.*, 15:397–411, 2013.
- [66] Data Science Beyond. Model Complexity in Machine Learning — ishanjainofficial.medium.com. <https://ishanjainofficial.medium.com/model-complexity-explained-intuitively-e179e38866b6>. [Accessed 13-08-2025].
- [67] What Is the Time Complexity for Training a Neural Network Using Backpropagation? — Baeldung on Computer Science — baeldung.com. <https://www.baeldung.com/cs/backpropagation-time-complexity>. [Accessed 13-08-2025].
- [68] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and whats next. *Journal of Scientific Computing*, 92, 07 2022.
- [69] Volker Blum, Ryoji Asahi, Jochen Autschbach, Christoph Bannwarth, Gustav Bihlmayer, Stefan Blgel, Lori A Burns, T Daniel Crawford, William Dawson, Wibe Albert de Jong, Claudia Draxl, Claudia Filippi, Luigi Genovese, Paolo

- Giannozzi, Niranjana Govind, Sharon Hammes-Schiffer, Jeff R Hammond, Benjamin Hourahine, Anubhav Jain, Yosuke Kanai, Paul R C Kent, Ask Hjorth Larsen, Susi Lehtola, Xiaosong Li, Roland Lindh, Satoshi Maeda, Nancy Makri, Jonathan Moussa, Takahito Nakajima, Jessica A Nash, Micael J T Oliveira, Pansy D Patel, Giovanni Pizzi, Geoffrey Pourtois, Benjamin P Pritchard, Eran Rabani, Markus Reiher, Lucia Reining, Xinguo Ren, Mariana Rossi, H Bernhard Schlegel, Nicola Seriani, Lyudmila V Slipchenko, Alexander Thom, Edward F Valeev, Benoit Van Troeye, Lucas Visscher, Vojtech Vlek, Hans-Joachim Werner, David B Williams-Young, and Theresa L. Windus. Roadmap on methods and software for electronic structure based simulations in chemistry and materials. *Electronic Structure*, 6(4):042501, nov 2024.
- [70] Andreas Dreuw and Martin Head-Gordon. Single-reference ab initio methods for the calculation of excited states of large molecules. *Chemical Reviews*, 105(11):4009–4037, 2005. PMID: 16277369.
- [71] Xiaosong Li, Stanley M. Smith, Alexei N. Markevitch, Dmitri A. Romanov, Robert J. Levis, and H. Bernhard Schlegel. A time-dependent hartreefock approach for studying the electronic optical response of molecules in intense fields. *Phys. Chem. Chem. Phys.*, 7:233–239, 2005.
- [72] Attila Szabo and Neil S. Ostlund. *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*. Dover Publications, Inc., Mineola, first edition, 1996.
- [73] Xiaosong Li, Stanley M. Smith, Alexei N. Markevitch, Dmitri A. Romanov, Robert J. Levis, and H. Bernhard Schlegel. A time-dependent hartreefock approach for studying the electronic optical response of molecules in intense fields. *Phys. Chem. Chem. Phys.*, 7:233–239, 2005.
- [74] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA, 2001.
- [75] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [76] Alberto Fabrizio, Andrea Grisafi, Benjamin Meyer, Michele Ceriotti, and Clemence Corminboeuf. Electron density learning of non-covalent systems. *Chem. Sci.*, 10:9424–9432, 2019.
- [77] Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, USA, 1st edition, 2019.

Appendix A

Machine learning program details

A.0.1 Input parameters

Table A.1 below listed the parameters to be adjusted before running the program. The parameters that are not listed here do not need to be adjusted, unless with new dataset or new commit on codes.

Parameter variable	Value	Description
molecule	Molecule dataset name	Select the dataset to investigate. Comment out other molecule names
kick_size, remove_kick	500, 1000, True / False	Select the first X value amount of data to remove to investigate kick effect
plot_front_end	True / False	Plot first X% and last X% of the training and testing data
plot_front_end_percentage	10,	
performance_metric_ front_end	False / True	Calculate performance metrics of first X% and last X% of the training and testing data
performance_metric_ front_end_percentage	10,	
scaler_type	MinMax, Std, None	Choose type of data scaling
plot_training_data	True / False	Plot training data
start_train , end_train	0 1000,	Define range of points to plot training data
plot_testing_data	True / False	Plot testing data
start_test , end_test	0 1000,	Define range of points to plot testing data
check_noise	True / False	Print which columns has very small values
filter_noise	True / False	Remove the columns that has very small values
scikit_LR	True / False	Use linear regression method. Set to False when not in use
Neural_Network	True / False	Use Neural network method. Set to False when not in use
grid_search	True / False	Use grid search method. Set to False when not in use
training_percentage	50,	Define amount of training data to use. To use when not running program in loop.
training_percentage2	List of percentage value	Loop through different value of training percentage at once.

Table A.1: Parameters list

Appendix B

Propagation details

B.1 Molecule coordinates

H₂

```
# mol.atom = '''
# H  0.0000000000000000  0.0000000000000000  -0.01997819186545
# H  0.0000000000000000  0.0000000000000000  0.72797819186545'''
```

LiH

```
# aligned along z axis
# mol.atom = '''
# Li  0.0000000000000000  0.0000000000000000  0.02959606653958
# H  0.0000000000000000  0.0000000000000000  1.64733393346042'''
```

```
# aligned along y axis
# mol.atom = '''
# Li  0.0000000000000000  0.02959606653958  0.0000000000000000
# H  0.0000000000000000  1.64733393346042  0.0000000000000000'''
```

```
# aligned along x axis
mol.atom = '''
Li  0.02959606653958  0.0000000000000000  0.0000000000000000
H  1.64733393346042  0.0000000000000000  0.0000000000000000'''
```

H₂O

```
# mol.atom = '''
# O      0.0000  0.0000  0.0000
# H     -0.6142 -0.4283  0.5786
# H      0.6142  0.4283  0.5786'''
```

B.2 Details of the simulation

Impulse: analytic, Fmax (a.u.): 0.02, Direction:z
dt: 0.1 a.u., Total num. steps: 10000
E0 in a.u. 2.000000e-02
Energy/(area*time) in a.u. 2.180994e-03
Energy/(area*time) in W/cm² 1.403688e+13